Algorithm Engineering Notes

Academic Year 2026/2027

Contents

1 Algorithm Design, Computational Models, and Memory Hierarchy **1**
- 1.1 Formal Definition of an Algorithm . . . 1
- 1.2 Efficiency Measurement and Asymptotic Analysis . . . 3
- 1.3 Computational Models: RAM and Real Hardware . . . 4
- 1.4 Locality and the Two-Level Model . . . 7
- 1.5 Worked Example: Linear Array Scan . . . 9

2 Strided Access, B-Trees, and the Impact of Virtual Paging **11**
- 2.1 Non-Sequential Access: The $A_{s,b}$ Algorithm . . . 11
- 2.2 Algorithmic Limits and Hardware Acceleration . . . 13
- 2.3 Searching Ordered Sets: Binary Search versus B-Trees . . . 14
- 2.4 I/O-Aware Algorithms and Virtual Paging . . . 16
- 2.5 Key Takeaways . . . 17

3 Permutation, Sorting, and Binary Merge Sort in External Memory **19**
- 3.1 The Permutation vs. Sorting Problem . . . 19
- 3.2 Permutation Complexity in External Memory . . . 24
- 3.3 External Memory Binary Merge Sort . . . 26
- 3.4 Asymptotic Equivalence of the External Sorting Cost . . . 29

4 Multiway Merge Sort, Snow Plow, and Disk Striping **31**
- 4.1 Multiway Merge Sort (k-Way Merge Sort) . . . 31
- 4.2 Optimizing Run Formation: The Snow Plow (*Replacement Selection*) . . . 34
- 4.3 Parallelism Across Multiple Disks: *Disk Striping* . . . 37

5 Lower Bounds for Sorting, Dual-Pivot Quicksort, and Bounded Quicksort **41**
- 5.1 RAM Model: Lower Bounds and Decision Trees . . . 41

5.2 Two-Level Memory: Aggarwal-Vitter Lower Bound 45
5.3 Quicksort: Duplicate Handling and Dual-Pivot Quicksort . . . 49
5.4 Bounded Quicksort (BQS) and Tail Recursion Elimination . . 56

6 Order Statistics, Multi-Pivot Trade-offs, and Average-Case Analysis of Quicksort 63
6.1 The Selection Problem (*Selection of the k-th Item*) 64
6.2 The Multi-Pivot Dilemma (*Why Not Many Pivots?*) 69
6.3 Quicksort: 3-Way Partitioning and Recursive Implications . . 72
6.4 Formal Mathematical Proof of Quicksort Average-Case ($\Theta(n \log_2 n)$) 75
6.5 Comparative Synthesis of Results 79

7 External Distribution Sort, Over-Sampling, and Random Sampling Foundations 82
7.1 The External Sorting Problem and Multi-Way Partitioning . . 82
7.2 The Over-Sampling Technique 84
7.3 Probabilistic Failure Analysis and Chernoff Bound 86
7.4 Introduction to Random Sampling and Non-Streaming Limits 91

8 Random Sampling Algorithms on Data Streams 95
8.1 Foundations of the Streaming Model and Random Sampling . 96
8.2 Sampling with Known Stream Length (n Known) 97
8.3 Sampling with Unknown Stream Length (n Unknown) 103
8.4 Formal Proof by Induction of Reservoir Sampling 107
8.5 Comparative Synthesis 110

9 Randomized Data Structures: The Dictionary Problem and Skip Lists 111
9.1 Introduction and Motivation: The Dictionary Problem 112
9.2 From Binary Search on Lists to Skip Lists 114
9.3 Skip Lists: Structure and Formal Probabilistic Analysis 116
9.4 Summary and Outlook: Toward Randomized Search Trees . . 120

10 Randomized Search Trees: Treaps, Dynamic Operations, and Geometric Applications 122
10.1 Introduction and Formal Treap Definition 123
10.2 Elementary Operations: Tree Rotations 125
10.3 Fundamental Dynamic Algorithms 127
10.4 Probabilistic Analysis 133
10.5 Geometric Isomorphism and Applications 134

11 Posting Lists Intersection Algorithms in Information Retrieval 143
11.1 Search Engine Architecture and Problem Formulation 144
11.2 Fundamental Algorithms and Comparative Analysis 147
11.3 Mutual Partition (Divide and Conquer) 150
11.4 Doubling Search (Exponential Search / Galloping Search) . . . 153
11.5 Block Partitioning / Skip Pointers (Practical Block Approach) 157

Chapter 1

Algorithm Design, Computational Models, and Memory Hierarchy

Lecture 1

Introduction, computational models, and memory hierarchy

Date: September 15, 2026

1.1 Formal Definition of an Algorithm

An *algorithm* is a finite and ordered sequence of unambiguous, elementary instructions that, starting from a specified set of input data, produces a corresponding set of output data within a finite amount of time.

In the classical formulation by Donald Knuth in *The Art of Computer Programming* [1], an algorithm must satisfy five fundamental properties:

Finiteness. The algorithm must always halt after a finite number of computation steps for every admissible input configuration.

Definiteness. Each instruction must be stated clearly, precisely, and without ambiguity; the next execution state must be uniquely determined.

Input. The algorithm receives zero or more quantities specified from outside before execution begins.

Output. The algorithm produces one or more output quantities that have a specified relationship with the input data. The output must be meaningful for the problem being solved.

Effectiveness. Every prescribed operation must be elementary and physically executable in finite time using a bounded amount of resources, such as paper and pencil or a sequential machine.

1.1.1 Exact Algorithms, Artificial Intelligence, and Heuristics

A classical exact algorithm is distinguished from an AI or heuristic method by the guarantees it provides and by how it explores the space of decisions.

Table 1.1: Exact algorithms compared with AI and heuristic methods.

Dimension	Classical exact algorithm	AI or heuristic method
Correctness guarantee	Produces a correct solution for every admissible instance; optimality is guaranteed when the problem specification asks for an optimum.	May prioritize empirical quality or practical efficiency and may lack a universal guarantee for every instance.
Termination	Can be proved in advance using potential functions or loop variants.	Often tied to heuristic stopping criteria, such as a maximum number of epochs or gradient convergence.
Decision space	Determined by a precisely specified procedure, which may be deterministic or randomized.	Explored through heuristic search, statistical inference, or loss-function optimization.

Heuristic programming does not seek a formal proof of absolute optimality over every admissible instance. Instead, it seeks an efficient trade-off for computationally intractable problems, such as NP-complete or NP-hard problems.

1.2 Efficiency Measurement and Asymptotic Analysis

1.2.1 Why Not Count the Exact Number of Instructions?

Defining the exact running cost $T_A(n)$ as the actual number of clock cycles or elementary instructions presents several difficulties:

1. **Architecture dependence:** the result would depend on the specific CPU, instruction set architecture (ISA), and clock frequency.
2. **Toolchain dependence:** it would vary with the compiler, optimization level, and operating system.
3. **Lack of generality:** it would obscure the intrinsic quality of the algorithmic idea.

For this reason, algorithm analysis uses asymptotic notation. It abstracts away implementation details and multiplicative constants, and studies the growth rate of the cost function as the input size tends to infinity: $n \to \infty$.

Big-$\mathcal{O}$ (asymptotic upper bound):

$$f(n) = \mathcal{O}(g(n)) \iff \exists c > 0,\ n_0 > 0 \text{ such that } 0 \leq f(n) \leq c\, g(n), \quad \forall n \geq n_0.$$

Big-Ω (asymptotic lower bound):

$$f(n) = \Omega(g(n)) \iff \exists c > 0,\ n_0 > 0 \text{ such that } 0 \leq c\, g(n) \leq f(n), \quad \forall n \geq n_0.$$

Big-Θ (tight asymptotic bound):

$$f(n) = \Theta(g(n)) \iff f(n) = \mathcal{O}(g(n)) \wedge f(n) = \Omega(g(n)).$$

1.2.2 Why Prefer Worst-Case Analysis?

- **Absolute guarantee:** an upper bound guarantees that the algorithm will never take more time than the theoretical bound. This is essential for real-time and mission-critical systems.
- **Simple formal derivation:** worst-case analysis does not require probabilistic assumptions about the input distribution, which are often needed for average-case analysis.

1.3 Computational Models: RAM and Real Hardware

An algorithm is not executed in an abstract vacuum: it runs on a machine with processors, memories, and data-transfer mechanisms. A computational model specifies which machine details are relevant to the analysis and which are deliberately ignored. The RAM model is the standard starting point; real hardware then explains when that abstraction becomes insufficient.

1.3.1 The Random Access Machine Model

The Random Access Machine (RAM) model is a sequential, word-based abstraction inspired by the von Neumann architecture. A RAM consists of a processor, a finite program, and an addressable memory of words. Its usual assumptions are:

- **Sequential execution:** one processor executes one elementary instruction at a time.
- **Word-sized data:** arithmetic, comparison, assignment, and address calculation on a constant number of words take constant time.
- **Uniform memory access:** reading or writing any address costs $\mathcal{O}(1)$, independently of the address and of the access history.
- **Sufficient memory:** the input and the data structures fit in the modeled address space; the cost of allocating the space is analyzed separately when it matters.

The RAM model is useful because it makes algorithmic ideas comparable. For example, a loop that examines n array entries performs $\Theta(n)$ RAM operations, regardless of the particular processor used to run it. The model does not claim that a real machine has infinite memory or truly constant latency; it states which costs are being abstracted away.

1.3.2 Real Hardware: Memory Hierarchy and the Memory Wall

On a modern machine, the processor is much faster than the memories that hold large datasets. This growing gap between computation and data movement is known as the *memory wall*. Moreover, local memory is not a

single uniform resource: it is organized into levels with different capacities, latencies, and transfer granularities.

A simplified hierarchy, ordered from the fastest and smallest level to the slowest and largest one, is:

$$\begin{aligned}\text{CPU registers} &\longrightarrow \text{L1 cache} \longrightarrow \text{L2 cache} \longrightarrow \text{L3 cache} \longrightarrow \text{DRAM} \\ &\longrightarrow \text{NVMe/SSD/disk} \longrightarrow \text{remote storage / cloud.}\end{aligned}$$

Registers and caches are part of the processor's fast memory system; DRAM is the usual main memory; SSDs and disks provide persistent storage. Remote storage and cloud services are included here only as a broader data-access hierarchy, because they are reached through a network rather than through the local memory bus.

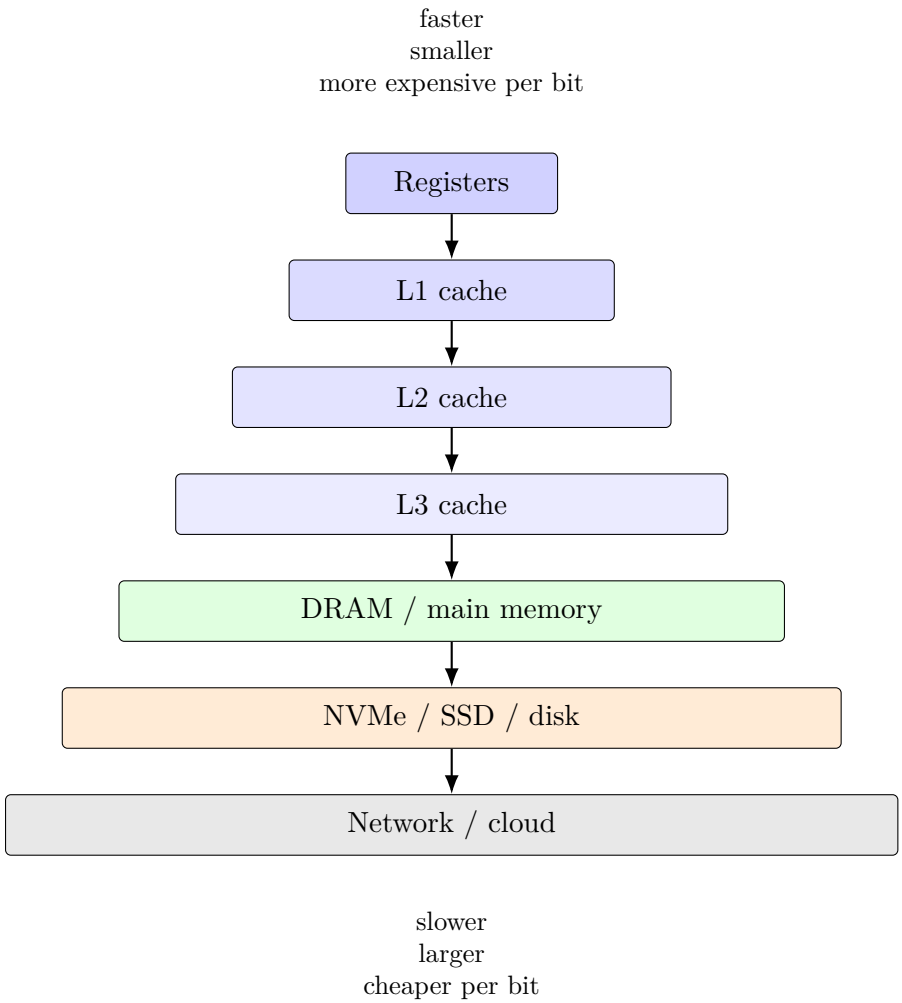


Figure 1.1: A hierarchical view of memory and storage.

Figure 1.1 describes the main local-memory levels: each lower layer trades lower access speed for greater capacity.

Moving downward in Figure 1.1, capacity generally grows while latency and transfer cost grow as well. The exact ratios depend on the hardware and workload, so values such as 100–200 times for DRAM relative to L1, or 10^4–10^6 times for disk relative to L1, should be read as order-of-magnitude examples rather than universal constants.

1.3.3 When the RAM Model Is Not Enough

The RAM model is a good first approximation when the computation is mostly sequential, the working set fits in main memory, and the cost of data

movement is not the dominant factor. A different model is preferable when one of the following effects controls performance:

GPU computing. SIMT (Single Instruction, Multiple Threads) execution uses many processing cores and distinguishes shared, local, and global memory spaces.

External memory. In out-of-core computation, the dataset does not fit in fast memory, so block transfers and I/O operations dominate the cost.

Parallel and distributed computation. Several processors or machines execute simultaneously, and synchronization and communication become explicit costs.

Quantum computing. Qubits, superposition, entanglement, and unitary gates require a model based on quantum operations rather than RAM words.

The engineering lesson is therefore not to discard the RAM model, but to use the simplest model that still captures the bottleneck of the target system.

1.4 Locality and the Two-Level Model

Modern systems exploit two statistical properties of real programs:

1. **Temporal locality:** if a memory location is referenced at time t, it is likely to be referenced again soon, at $t + \Delta t$.
2. **Spatial locality:** if a memory location x is referenced, nearby locations such as $x + 1$, $x + 2$, and so on are likely to be referenced soon.

1.4.1 The Two-Level Memory Model

When processing gigabytes or terabytes of data, the multilevel hierarchy can be abstracted as a two-level model, associated with the external-memory model of Aggarwal and Vitter [2], also known as the Disk Access Machine model.

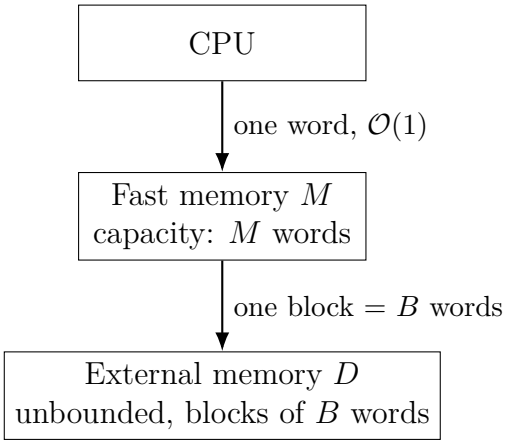


Figure 1.2: Two-level memory model with fast memory M and external memory D.

Figure 1.2 describes the two-level abstraction: the CPU operates on words in fast memory M, while transfers between M and the external memory D occur in blocks of B words.

Model Parameters

N**:** total number of problem elements to process.

M**:** size of internal fast memory, measured as the maximum number of elements that can be held simultaneously.

B**:** size of the block transferred in one I/O operation, with $M \geq 2B$.

Cost Metrics

1. **I/O complexity (primary metric):** number of transfers of blocks of size B between slow and fast memory.
2. **CPU time (secondary metric):** number of logical and arithmetic operations performed after data have been loaded into M.
3. **Space:** number of blocks allocated on secondary storage.

This abstraction is fractal: it can model both RAM–disk transfers and L3-cache–RAM transfers by redefining the meaning of M and B.

1.5 Worked Example: Linear Array Scan

Consider a contiguous array of N integers scanned in sequence, for example to compute the sum $\sum_{i=1}^{N} A[i]$.

Listing 1.1: Sequential array scan

```
long sum = 0;
for (int i = 0; i < N; i++) {
    sum += A[i];
}
```

1.5.1 Analysis in the Classical RAM Model

- Memory accesses: N.
- Addition instructions: N.
- Asymptotic computation time: $\Theta(N)$.

1.5.2 Analysis in the Two-Level Model

Assume that the secondary memory stores array A consecutively in blocks of size B and that $A[0]$ is aligned with a block boundary. Without this alignment assumption, the scan may require at most one additional block transfer, without changing the asymptotic bound:

1. At the first access to $A[0]$, a miss occurs. The system transfers the block containing $A[0], \ldots, A[B-1]$ into fast memory with one I/O.
2. The following $B-1$ accesses find their data already in fast memory, requiring no additional I/O. This exploits spatial locality.
3. A new I/O transfer is needed only when the index crosses a block boundary, that is, once every B elements.

Therefore, the total I/O cost of a linear scan is

$$\mathrm{Scan}(N) = \left\lceil \frac{N}{B} \right\rceil = \Theta\left(\frac{N}{B}\right) \text{ I/O operations.}$$

1.5.3 Linear Access versus Sparse Access

- **Sequential access (high spatial locality):** reading N elements requires N/B I/O operations. Since B typically ranges from 64 bytes for a cache line to hundreds of kilobytes or megabytes for pages and disk blocks, the speedup over the naive model can be a factor of approximately B.
- **Random access or stride $k > B$ (no spatial locality):** every requested element may reside in a different block, causing N independent I/O operations. The cost degrades to $\Theta(N)$ I/Os, which is asymptotically slower by a factor of B.

Chapter 2

Strided Access, B-Trees, and the Impact of Virtual Paging

Lecture 2

Strided access, binary search versus B-Trees, and virtual paging

Date: September 16, 2026

This chapter studies how an algorithm's access pattern interacts with the transfer granularity of a memory system. The same arithmetic work can have very different costs depending on whether accesses preserve spatial locality, match the block size, or exceed the capacity of fast memory.

2.1 Non-Sequential Access: The $A_{s,b}$ Algorithm

Let S be an array containing n elements. The storage system transfers data in physical blocks of size B (a disk block or cache line). We also define a logical block size b, where $b \leq B$ and b divides B. The corresponding numbers of logical and physical blocks are

$$N' = \frac{n}{b}, \qquad N_{\text{phys}} = \frac{n}{B}.$$

The algorithm $A_{s,b}$ computes the sum of the elements by visiting logical blocks in a stride pattern. The logical block visited at step j is

$$\text{block}(j) = (js) \bmod N', \qquad j = 0, 1, \ldots, N' - 1.$$

2.1.1 Coprimality Condition

The sequence visits every logical block exactly once if and only if

$$\gcd\left(s, \frac{n}{b}\right) = 1.$$

In that case, multiplication by s is a permutation of the residues modulo N'. If the greatest common divisor is $d > 1$, the sequence repeats after $N'/d = n/(bd)$ distinct logical blocks and therefore covers only a fraction of the array.

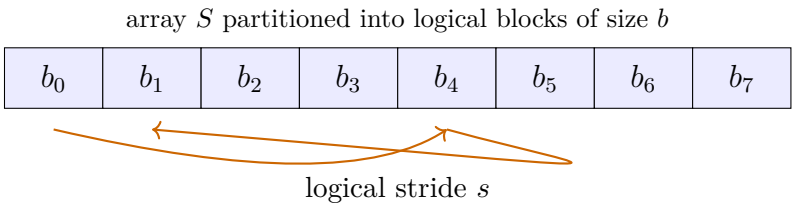


Figure 2.1: A modular stride visits logical blocks in a non-sequential order.

Figure 2.1 illustrates the logical access order. The physical cost depends on how this order maps to blocks of size B, rather than only on the number of arithmetic operations.

2.1.2 I/O Cost

A useful first approximation compares the physical distance between successive logical accesses, sb, with the physical block size B.

Table 2.1: Approximate I/O behavior of the strided access pattern.

Condition	Locality and behavior	I/O cost
$sb < B$	Several logical accesses can share one physical block; partial spatial locality is preserved.	$\Theta(n/B)$
$sb \geq B$	Successive accesses typically fall on different physical blocks. If $M < n$, the working set cannot remain entirely in fast memory.	$\Theta(n/b)$
$b = 1$ and $s \geq B$	Each logical access is one element and the stride destroys the benefit of block transfers.	$\Theta(n)$

The bounds in Table 2.1 describe the usual worst-case or capacity-miss regime. Exact constants depend on alignment, replacement policy, and the interaction between the modular sequence and physical blocks. In the last case, the cost is a factor of roughly B worse than a linear scan, which can exploit every transferred element.

2.2 Algorithmic Limits and Hardware Acceleration

Fix a maximum admissible execution time T_0 and assume that hardware or parallelism accelerates the computation by a factor K. The accelerated machine can therefore afford an original running time of

$$\frac{T(n)}{K} = T_0 \qquad \Longleftrightarrow \qquad T(n) = KT_0.$$

The effect on the largest solvable input depends on the growth rate of T.

Table 2.2: Effect of a K-fold hardware acceleration on solvable input size.

Algorithm	Complexity	Base hardware	Accelerated hardware	Effective gain
A_1	cn	$n_1 = T_0/c$	$n_1' = Kn_1$	Multiplicative, K
A_2	cn^2	$n_2 = \sqrt{T_0/c}$	$n_2' = \sqrt{K}\, n_2$	Sublinear, $\sqrt{K}$
A_3	$c2^n$	$n_3 = \log_2(T_0/c)$	$n_3' = n_3 + \log_2 K$	Additive, $+\log_2 K$

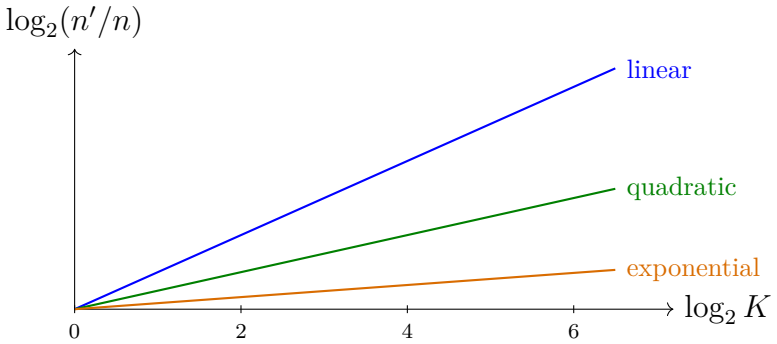


Figure 2.2: Relative benefit of hardware acceleration for representative growth rates.

Figure 2.2 shows why hardware improvements do not remove an asymptotically inefficient algorithm. For an exponential algorithm, even a 1000-fold speedup increases the solvable input size by only $\log_2(1000) \approx 10$ elements.

2.3 Searching Ordered Sets: Binary Search versus B-Trees

2.3.1 Binary Search in Secondary Memory

For a sorted sequence of n keys stored in blocks of size B, binary search performs $\Theta(\log_2 n)$ comparisons on the CPU. Its I/O cost is larger than its comparison count suggests: during the first iterations, the examined positions are separated by $n/2$, $n/4$, and so on. Until the search interval shrinks to at most B elements, each probe can address a different physical block. Thus,

$$\mathrm{I/O}_{\text{binary search}} \approx \log_2 n - \log_2 B = \log_2\left(\frac{n}{B}\right).$$

The approximation ignores alignment and the possibility that a block remains cached, but captures the dominant external-memory behavior.

2.3.2 B-Trees and B$^+$-Trees

A B-Tree chooses the node size to match one physical block. Each node stores $\Theta(B)$ keys and child pointers, so its fan-out is $\Theta(B)$. A B$^+$-Tree uses the same principle and normally stores records or record pointers in its leaves, making it especially suitable for range queries.

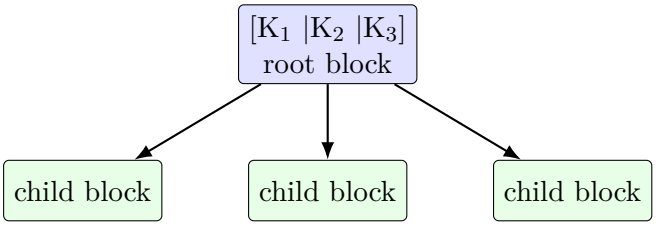


Figure 2.3: A block-oriented B-Tree with high fan-out.

Figure 2.3 shows the central layout decision: one I/O loads an entire node and supplies many routing keys and pointers. The height, and therefore the number of block reads for a point query, is approximately

$$h = \log_B\left(\frac{n}{B}\right) = \frac{\log_2(n/B)}{\log_2 B}.$$

More precisely, implementation details introduce ceilings and occupancy factors. The internal-node space is small compared with the leaves because

$$\text{internal nodes} \le \frac{n}{B^2} + \frac{n}{B^3} + \cdots = \frac{n}{B(B-1)} \ll \frac{n}{B}.$$

2.3.3 Quantitative Comparison

Ignoring ceilings, the I/O speedup of a B-Tree over binary search is

$$\frac{\mathrm{I/O}_{\text{binary search}}}{\mathrm{I/O}_{\text{B-Tree}}} = \frac{\log_2(n/B)}{\log_2(n/B)/\log_2 B} = \log_2 B.$$

Table 2.3: Illustrative comparison for a large ordered dataset.

Parameter	Value
Dataset size	$n = 2^{40}$ elements, approximately 10^{12} records
Block size	$B = 2^{14} = 16\,384$ records
Binary-search I/O	$40 - 14 = 26$ block accesses
B-Tree I/O	$(40 - 14)/14 \approx 1.85$, hence about 2 block accesses

For millisecond-scale storage latency, the difference between approximately 26 and 2 block accesses changes a point query from hundreds of milliseconds to only a few milliseconds, subject to caching and hardware details.

2.4 I/O-Aware Algorithms and Virtual Paging

There are two complementary ways to manage transfers between fast and slow memory:

- **I/O-aware design (explicit):** The algorithm models B and the fast memory capacity M directly, and manages buffers, data layout, and transfers programmatically.
- **Virtual memory and paging (implicit):** The operating system and the hardware transparently move pages using page tables and replacement policies such as LRU approximations.

2.4.1 Paging with an Oversized Working Set

Assume that physical memory has capacity M and is divided into M/B frames. If the application requires

$$n = (1 + \varepsilon)M = M + \varepsilon M, \qquad \varepsilon > 0,$$

then at least εM units of the working set must reside in secondary storage. Let α be the fraction of instructions that perform memory operations. A typical instruction profile is:

- fraction $1 - \alpha$: arithmetic or logical operations, with cost approximately one CPU cycle;
- fraction α: `LOAD` or `STORE` operations;

- probability $1 - P(\varepsilon)$: the referenced data are resident in RAM, with access time t_m;
- probability $P(\varepsilon)$: the access causes a page fault, with service time t_{disk}.

The resulting average time per instruction is

$$T_{\text{avg}} = (1 - \alpha) \cdot 1 + \alpha \left[(1 - P(\varepsilon)) t_m + P(\varepsilon) t_{\text{disk}} \right].$$

Since $t_m \ll t_{\text{disk}}$, the page-fault term can dominate:

$$T_{\text{avg}} \approx \alpha P(\varepsilon) t_{\text{disk}}.$$

This approximation is a warning about sensitivity, not an identity: when $P(\varepsilon)$ is zero, the baseline CPU and RAM terms still matter.

2.4.2 Numerical Thrashing Example

Take $\alpha = 0.3$, a page-fault penalty of $t_{\text{disk}} = 10^5$ clock cycles, and a fault probability of only $P(\varepsilon) = 10^{-3}$. Then the expected penalty is

$$T_{\text{penalty}} \approx 0.3 \cdot 10^{-3} \cdot 10^5 = 30 \text{ cycles per instruction.}$$

One page fault every thousand memory accesses can therefore add about 30 cycles to every instruction on average. Relative to a one-cycle baseline, this can produce an order-of-magnitude slowdown, potentially in the $10\times$–$30\times$ range depending on the ordinary RAM latency and the actual working-set behavior. When the fault rate rises further, the process enters *thrashing*: the system spends most of its time servicing page faults instead of progressing with useful computation.

2.5 Key Takeaways

1. A modular stride visits all logical blocks only when the stride and the number of logical blocks are coprime.
2. Block-aware layouts and access patterns can change I/O complexity by a factor of B, even when CPU work is unchanged.
3. B-Trees reduce external-memory search depth by matching their fan-out to the physical block size.

4. Hardware acceleration improves linear and polynomial algorithms in different ways, but gives only an additive improvement to exponential algorithms.

5. Virtual memory is convenient, but an oversized working set can turn a small page-fault probability into a dominant performance cost.

Chapter 3

Permutation, Sorting, and Binary Merge Sort in External Memory

Lecture 3

Permutation vs. Sorting, Two-Level Memory Model, and Binary Merge Sort

Date: September 21, 2026

In this chapter, we examine the fundamental relationship between the permutation problem and the sorting problem in the two-level memory hierarchy, contrasting the characteristics of the RAM model with block-based storage systems. We analyze the permuting algorithm via sorting and scanning, evaluating its I/O bounds. Finally, we explore external Binary Merge Sort, highlighting its internal memory underutilization bottleneck and establishing the asymptotic equivalence of the sorting I/O bound between logarithmic arguments n/M and n/B.

3.1 The Permutation vs. Sorting Problem

3.1.1 Problem Formulation

We are given two vectors of size n:

- A data array $S[1 \dots n]$.
- A permutation vector $\pi[1 \dots n]$, containing a bijective permutation of the indices $\{1, 2, \dots, n\}$.

The objective is to rearrange the elements of S into a new array S' such that each original element $S[i]$ is placed into the destination position specified by $\pi[i]$:

$$S'[\pi[i]] = S[i], \qquad \forall i \in \{1, \ldots, n\}. \tag{3.1}$$

Example 3.1.1 (Index Permutation)**.** Consider an array of 4 elements and its corresponding permutation vector:

$$\begin{aligned} \text{Indices:} \quad & 1 \quad 2 \quad 3 \quad 4 \\ S = \quad & [\,A, \quad B, \quad C, \quad D\,] \\ \pi = \quad & [\,3, \quad 1, \quad 2, \quad 4\,] \end{aligned}$$

Applying the definition element by element:

$$\begin{aligned} S'[\pi[1]] &= S'[3] = A, \\ S'[\pi[2]] &= S'[1] = B, \\ S'[\pi[3]] &= S'[2] = C, \\ S'[\pi[4]] &= S'[4] = D. \end{aligned}$$

The resulting permuted array is therefore:

$$S' = [\,B, \quad C, \quad A, \quad D\,].$$

In Figure 3.1, the mapping from the source array to the destination array is illustrated visually.

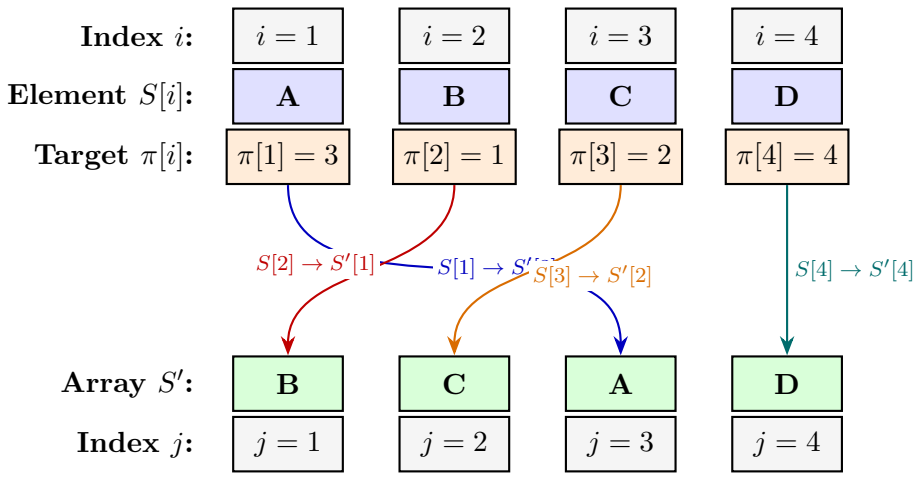


Figure 3.1: Permutation mapping: each element $S[i]$ is relocated to the destination index specified by $\pi[i]$ in array S'.

3.1.2 Naive Implementation on the RAM Model

In the standard RAM model, where any read or write access to arbitrary memory locations costs $\mathcal{O}(1)$ time, the permutation is implemented in a single linear pass:

Algorithm 1: Direct Permutation on RAM

Input: Array $S[1 \dots n]$, permutation vector $\pi[1 \dots n]$
Output: Permuted array S in place

```
for i ← 1 to n do
    S'[π[i]] ← S[i]
S ← copy(S')
```

The time complexity in the RAM model is strictly linear:

$$T_{\text{RAM}}(n) = \Theta(n).$$

Therefore, in internal memory, permuting is asymptotically faster than sorting by comparisons, which requires $\Omega(n \log n)$ operations.

3.1.3 Simulation Principle (RAM to Two-Level Memory)

Theorem 3.1.1 (Simulation Theorem)**.** *Any sequential algorithm formulated for the RAM model with time complexity $T(n)$ incurs at most $\mathcal{O}(T(n))$ block transfers (I/Os) in the two-level memory model with parameters (M, B).*

Proof Sketch. In the worst case, every single RAM operation accessing memory addresses a memory cell that resides in a block not currently resident in internal cache M. Each such access triggers a block transfer from disk, resulting in exactly 1 I/O operation. Multiplying the total step count $T(n)$ by the upper bound of 1 I/O per step yields the trivial bound of $\mathcal{O}(T(n))$ I/Os. □

Although the simulation principle guarantees that any RAM algorithm can be ported to external memory, the bound $\mathcal{O}(T(n))$ is often thoroughly sub-optimal because it fails to exploit the block aggregation factor B.

3.1.4 The Relationship Between Sorting and Permutation

In the two-level memory hierarchy, the complexity relationship changes fundamentally, as summarized in Table 3.1:

Problem	RAM Model	Two-Level Memory (I/O)
Permutation $\mathrm{Perm}(n)$	$\Theta(n)$	$\Theta\Big(\min\Big\{n, \mathrm{Sort}(n, M, B)\Big\}\Big)$
Sorting $\mathrm{Sort}(n)$	$\Theta(n \log_2 n)$	$\Theta\left(\frac{n}{B} \log_{M/B} \frac{n}{B}\right)$

Table 3.1: Asymptotic complexity comparison between RAM and External Memory models.

To understand this equivalence in block-based storage, consider the two reductions:

1. **Sorting via Permutation:** Given an array S, sorting requires computing the permutation π_S that puts keys in order, and then applying π_S to S.

2. **Permuting via Sorting (Pair Reduction Algorithm):** Permuting an array S can be converted into an external sorting problem via the following three-step procedure:

   - **Step 1 (Scan and Pair Creation):** Perform parallel linear scans over S and π to construct a set of n records containing $\langle S[i], \pi[i] \rangle$. Since this requires only streaming sequential I/O, the cost is:

   $$\mathrm{I/O}_{\text{Step 1}} = \mathcal{O}\left(\frac{n}{B}\right).$$

   - **Step 2 (Sorting by Destination):** Sort the n records using the second component $\pi[i]$ as the sorting key. The cost is identical to external sorting:

   $$\mathrm{I/O}_{\text{Step 2}} = \mathrm{Sort}(n, M, B).$$

   - **Step 3 (Scan and Extraction):** Since the records are now sorted by increasing target index, a simple sequential scan extracts the first coordinate $S[i]$ into the destination array S'. The cost is:

   $$\mathrm{I/O}_{\text{Step 3}} = \mathcal{O}\left(\frac{n}{B}\right).$$

The overall I/O cost of this reduction is:

$$\mathrm{I/O}_{\text{Sort-based}} = \mathcal{O}\left(\frac{n}{B}\right) + \mathrm{Sort}(n, M, B) + \mathcal{O}\left(\frac{n}{B}\right) = \Theta(\mathrm{Sort}(n, M, B)). \quad (3.2)$$

Example 3.1.2 (Step-by-Step Pair Reduction Trace on Concrete Arrays)**.** To visualize the reduction with concrete values, consider the instance from Example 3.1.1:

$$S = [A, B, C, D], \qquad \pi = [3, 1, 2, 4].$$

The algorithm executes across three sequential states of the record array:

- **Step 1 (Pairing):** Sequentially pair each item with its designated target index:

$$P = \Big[\langle A, 3\rangle,\ \ \langle B, 1\rangle,\ \ \langle C, 2\rangle,\ \ \langle D, 4\rangle \Big].$$

- **Step 2 (External Sorting on $\pi[i]$):** Sort the record array using the second coordinate (target destination) as the sorting key:

$$P_{\text{sorted}} = \Big[\langle B, \mathbf{1}\rangle,\ \ \langle C, \mathbf{2}\rangle,\ \ \langle A, \mathbf{3}\rangle,\ \ \langle D, \mathbf{4}\rangle \Big].$$

- **Step 3 (Sequential Extraction):** Stream through the sorted records and project their first coordinates to form the destination array:

$$S' = [B, C, A, D],$$

  which correctly fulfills the required permutation since $S'[1] = B$, $S'[2] = C$, $S'[3] = A$, $S'[4] = D$.

The pair reduction pipeline is illustrated in Figure 3.2.

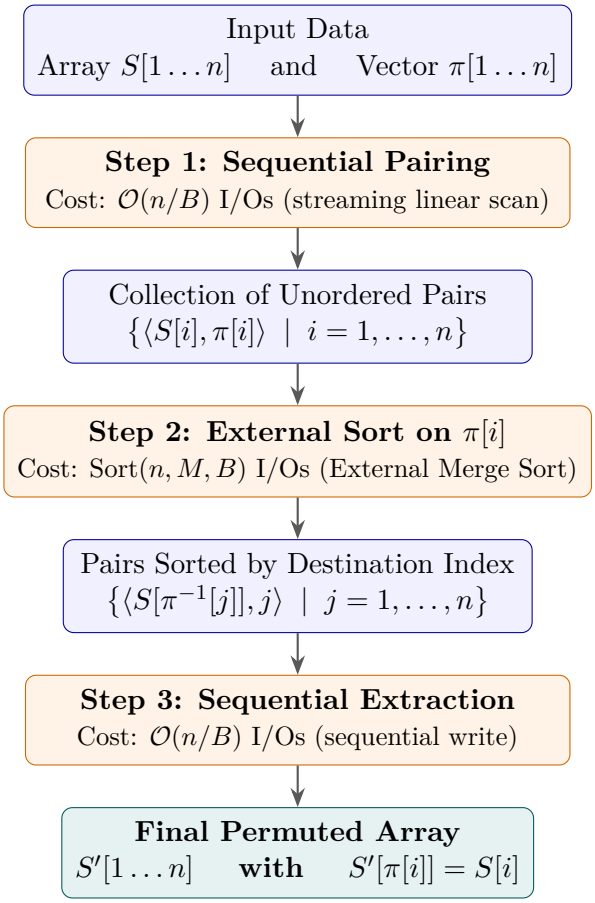


Figure 3.2: Operational workflow of the pair-reduction permutation algorithm.

3.2 Permutation Complexity in External Memory

The fundamental lower bound for permutation in the two-level memory model was established by Aggarwal and Vitter [2]:

$$\mathrm{Perm}(n, M, B) = \Theta\left(\min\left\{n, \frac{n}{B}\log_{M/B}\left(\frac{n}{B}\right)\right\}\right). \tag{3.3}$$

Letting $L = \log_{M/B}(n/B)$ denote the number of merge passes required by multiway merge sort:

$$\mathrm{Sort}(n, M, B) = \Theta\left(\frac{n}{B} \cdot L\right). \tag{3.4}$$

3.2.1 Corner Case Analysis

Comparing the direct naive algorithm with the sorting-based reduction yields two distinct regimes determined by the relationship between block size B and pass count L:

- **Case 1 ($n \geq \frac{n}{B} \cdot L \iff B \geq L$):** The block size B is greater than or equal to the depth of the merge tree. In this setting, sorting the pairs requires fewer I/Os than random single-element access:

$$\text{Perm}(n, M, B) = \Theta(\text{Sort}(n, M, B)).$$

- **Case 2 ($n < \frac{n}{B} \cdot L \iff B < L$):** If the block size were pathologically small compared to the overall dataset ($B < L$), direct element-by-element random writing (at most n I/Os) would be asymptotically cheaper than sorting:

$$\text{Perm}(n, M, B) = \Theta(n).$$

3.2.2 Numerical Verification on Realistic Architectural Parameters

To establish which regime governs real-world computing systems, consider extreme yet realistic parameters:

- Fast internal memory: $M = 16$ GB $= 2^4 \cdot 2^{30} = 2^{34}$ elements.
- Disk block size: $B = 32$ KB $= 2^5 \cdot 2^{10} = 2^{15}$ elements.
- Input dataset size: $n = 2^{60}$ elements (Exabyte scale).

We calculate the merge depth L:

$$L = \log_{M/B}\left(\frac{n}{M}\right) = \frac{\log_2(n/M)}{\log_2(M/B)}.$$

Substituting the exponential values:

$$\frac{n}{M} = \frac{2^{60}}{2^{34}} = 2^{26} \implies \log_2\left(\frac{n}{M}\right) = 26,$$
$$\frac{M}{B} = \frac{2^{34}}{2^{15}} = 2^{19} \implies \log_2\left(\frac{M}{B}\right) = 19.$$

Hence:

$$L = \frac{26}{19} \approx 1.36 < 2.$$

Because $B = 2^{15} = 32\,768$ and $L \leq 2$, the condition $B \geq L$ holds by more than four orders of magnitude ($32\,768 \gg 2$). Consequently, in all practical systems, permutation is strictly bound by sorting complexity:

$$\text{Perm}(n) = \Theta(\text{Sort}(n)).$$

3.3 External Memory Binary Merge Sort

The classical recursive Merge Sort algorithm extends naturally to external memory by recursively dividing arrays, ordering runs, and merging sorted sub-arrays:

Algorithm 2: Binary Merge Sort

Input: Array S, boundaries i, j
Output: Sorted sub-array $S[i \dots j]$

```
1 if i < j then
2     m ← ⌊(i + j)/2⌋
3     MS(S, i, m)
4     MS(S, m + 1, j)
5     MERGE(S, i, m, j)
```

3.3.1 Buffer Architecture in the Merge Procedure

Given two consecutive sorted runs of length l, the merge procedure produces a single consolidated run of length $2l$ into an auxiliary array S'.

- **In the RAM Model:** The operation performs at most $2l$ comparisons and $2l$ memory writes, costing $\Theta(l)$ CPU steps.
- **In the Two-Level Model:** To minimize block transfers, internal memory M is partitioned into 3 dedicated buffers, each having size exactly equal to one block B:
    1. **Input Buffer 1:** holds one block (size B) of the first sorted run.
    2. **Input Buffer 2:** holds one block (size B) of the second sorted run.
    3. **Output Buffer:** accumulates merged elements in sorted order until full (B elements).

Figure 3.3 illustrates the internal memory allocation and streaming I/O pattern during the binary merge.

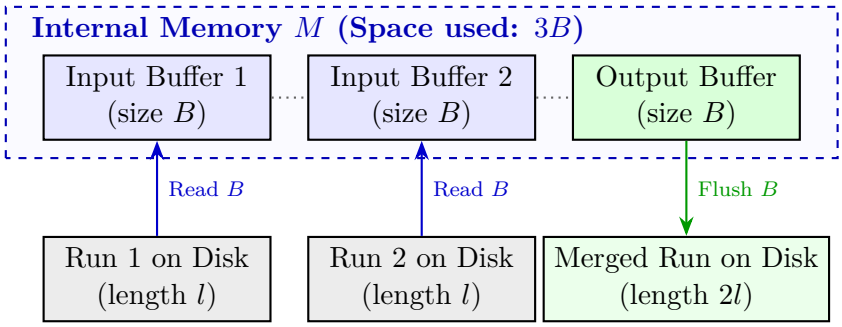


Figure 3.3: Internal memory buffer organization during binary external merge.

The execution protocol proceeds as follows:

1. The leading elements of the two input buffers are compared in CPU; the smaller key is appended to the output buffer.
2. Whenever an input buffer is exhausted, a disk read loads the next block of that run ($\frac{l}{B}$ I/Os per input run).
3. As soon as the output buffer fills up with B items, it is flushed to disk in a single write operation ($\frac{2l}{B}$ write I/Os overall).

The total I/O cost of merging two runs of length l is:

$$\mathrm{I/O}_{\mathrm{Merge}}(l) = \frac{l}{B} + \frac{l}{B} + \frac{2l}{B} = \frac{4l}{B} = \Theta\left(\frac{l}{B}\right). \tag{3.5}$$

3.3.2 Recursion Tree Complexity

Recursion terminates once sub-array sizes fit entirely within internal memory M. At this base level, each chunk of size M is read into memory, sorted using an in-memory algorithm (e.g., QuickSort or HeapSort) with zero additional I/O, and written back to disk using $\mathcal{O}(M/B)$ I/Os.

This yields:

$$N_{\mathrm{run}} = \frac{n}{M} \text{ initial sorted runs of size } M.$$

The binary merge tree has height:

$$h = \log_2\left(\frac{n}{M}\right) \text{ levels.} \tag{3.6}$$

Figure 3.4 depicts the recursion structure of binary external merge sort.

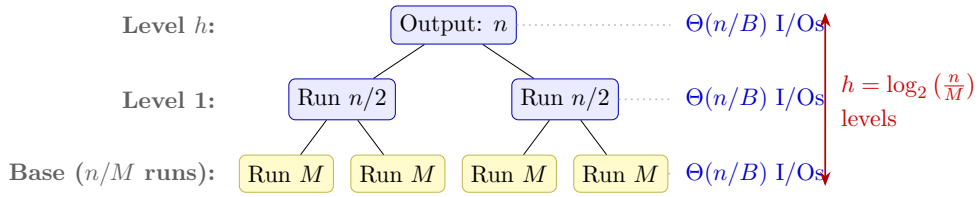


Figure 3.4: Recursion tree of external memory Binary Merge Sort with level I/O costs.

At each level of the tree, all n elements are streamed exactly once, requiring $\Theta(n/B)$ I/Os per level. Multiplying by tree height gives the overall complexity:

$$\mathrm{I/O}_{\mathrm{BinaryMergeSort}} = \mathcal{O}\left(\frac{n}{B}\log_2\left(\frac{n}{M}\right)\right). \tag{3.7}$$

3.3.3 Sub-optimality and Bottlenecks

Although significantly more efficient than naive approaches, this design suffers from a major structural bottleneck:

- **Drastic internal memory underutilization:** During merging, only 3 blocks ($3B$ elements) are occupied in fast memory M. The remaining capacity ($M - 3B$) sits completely idle.
- **Fixed logarithm base:** The base of the logarithm is fixed at 2, whereas memory M could accommodate up to:

$$k \approx \frac{M}{B} - 1 \text{ concurrent input streams.}$$

- **Loss of the $\log_2(M/B)$ convergence factor:** By failing to perform k-way merging (*Multiway Merge Sort*), the algorithm executes substantially more passes, falling short of the optimal lower bound:

$$\mathcal{O}\left(\frac{n}{B}\log_{M/B}\left(\frac{n}{B}\right)\right).$$

Figure 3.5 visually contrasts this internal memory underutilization against full buffer saturation.

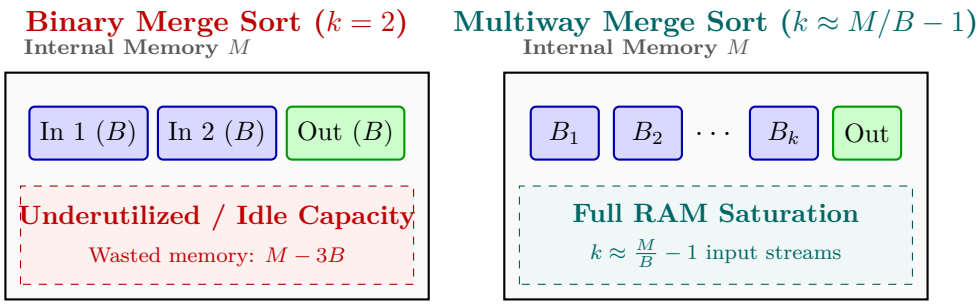


Figure 3.5: Internal memory allocation efficiency: severe underutilization in Binary Merge Sort versus full buffer saturation in Multiway Merge Sort.

Overcoming this structural limitation leads naturally to the multiway merging architecture, which we investigate in detail in the following sections.

3.4 Asymptotic Equivalence of the External Sorting Cost

In the computational analysis of secondary memory algorithms, the I/O complexity of external sorting is expressed in the literature [2] using two seemingly distinct formulations:

$$\Theta\left(\frac{n}{B}\log_{M/B}\left(\frac{n}{M}\right)\right) \qquad \text{or} \qquad \mathcal{O}\left(\frac{n}{B}\log_{M/B}\left(\frac{n}{B}\right)\right). \tag{3.8}$$

These two expressions are asymptotically equivalent for any problem instance where $n \geq M$.

To establish this equivalence formally, we apply the logarithmic product rule to expand the logarithmic term $\log_{M/B}(n/B)$:

$$\begin{aligned}\log_{M/B}\left(\frac{n}{B}\right) &= \log_{M/B}\left(\frac{n}{M}\cdot\frac{M}{B}\right)\\ &= \log_{M/B}\left(\frac{n}{M}\right) + \log_{M/B}\left(\frac{M}{B}\right)\\ &= \log_{M/B}\left(\frac{n}{M}\right) + 1.\end{aligned} \tag{3.9}$$

Multiplying the entire equation by the common scale factor $\frac{n}{B}$, we obtain:

$$\frac{n}{B}\log_{M/B}\left(\frac{n}{B}\right) = \frac{n}{B}\log_{M/B}\left(\frac{n}{M}\right) + \frac{n}{B}. \tag{3.10}$$

Algorithmically, the additional linear scan term $\frac{n}{B}$ corresponds precisely to the initial pass (Phase 0) that creates the initial sorted runs by loading chunks of size M into RAM, sorting them internally, and flushing them back to disk.

Whenever $n \gg M$, we have $\log_{M/B}(n/M) \geq 1$, which implies that the additive $+1$ term is readily subsumed by the hidden constant within asymptotic Θ-notation:

$$\Theta\left(\frac{n}{B}\log_{M/B}\left(\frac{n}{B}\right)\right) = \Theta\left(\frac{n}{B}\left(\log_{M/B}\left(\frac{n}{M}\right)+1\right)\right) = \Theta\left(\frac{n}{B}\log_{M/B}\left(\frac{n}{M}\right)\right). \tag{3.11}$$

Consequently, one may freely choose either $\frac{n}{M}$ or $\frac{n}{B}$ as the argument of the logarithm without altering the asymptotic validity of the result.

Chapter 4

Multiway Merge Sort, Snow Plow, and Disk Striping

Lecture 4

Multiway Merge Sort, Data Compression Impact, Snow Plow, and Disk Striping

Date: September 23, 2026

In this chapter, we develop in detail the *Multiway Merge Sort* (k-Way Merge Sort) on secondary storage, which overcomes the memory underutilization bottleneck of Binary Merge Sort and matches the Aggarwal-Vitter asymptotic lower bound. We evaluate the impact of data compression on I/O reduction and investigate two essential Algorithm Engineering techniques: doubling the expected length of initial sorted runs via the *Snow Plow* algorithm (*Replacement Selection*) and parallelizing storage throughput across multiple independent disks using *Disk Striping*.

4.1 Multiway Merge Sort (k-Way Merge Sort)

In the external *Binary Merge Sort* presented in section 3.3, only 3 blocks of internal memory were concurrently active, wasting the remaining $M - 3B$ elements of fast storage. The **Multiway Merge Sort** (or k-way merge sort) overcomes this severe bottleneck by utilizing the entire capacity M to merge k sorted runs simultaneously in a single pass (*fan-in* equal to k).

4.1.1 Internal Memory Configuration and Layout

Primary memory M is partitioned into blocks of transfer size B:

- $k = \frac{M}{B} - 1$ **input buffers**, each having capacity B, statically associated with one active run on external disk storage.
- 1 **output buffer**, also of capacity B, dedicated to incrementally accumulating sorted elements before performing an I/O flush to disk.
- An auxiliary in-memory data structure: a **Min-Heap** (or priority queue) of cardinality k. Each heap node stores the current leading key of one of the k active input buffers along with an identifier indicating its source buffer.

Figure 4.1 illustrates the architectural buffer layout and the data flow coordinated by the Min-Heap.

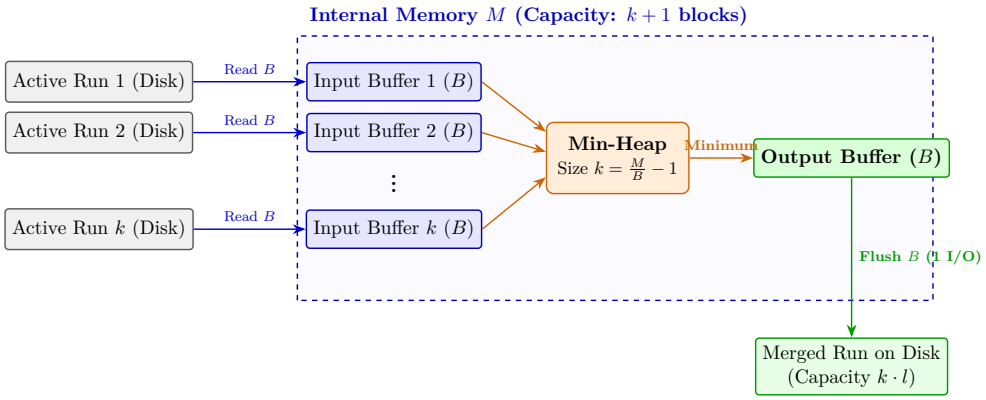


Figure 4.1: Internal memory layout during multiway merging (k-Way Merge Sort) with coordinating Min-Heap.

The Min-Heap allows extracting the global minimum among all k streams in $\mathcal{O}(\log_2 k)$ internal CPU comparisons. When the input buffer supplying the minimum is exhausted, a single disk read I/O is performed to load the next block of B elements from that run. Similarly, each time the output buffer accumulates B sorted elements, a single physical write I/O flushes the block to secondary storage.

4.1.2 Algorithm Execution and Tree Height Evolution

The execution proceeds across two distinct phases:

1. **Phase 0 (Initial Run Formation):** The unsorted array of n elements stored on disk is streamed in chunks of size M. Each chunk of M elements is brought into RAM, sorted using an optimal in-memory algorithm (e.g., QuickSort or HeapSort), and flushed back to disk:

- Number of initial sorted runs:

$$N_0 = \frac{n}{M}. \tag{4.1}$$

- I/O cost of Phase 0: full read and write scan across all n elements:

$$\mathrm{I/O}_{\mathrm{RunFormation}} = 2 \cdot \frac{n}{B} = \mathcal{O}\left(\frac{n}{B}\right). \tag{4.2}$$

2. **Subsequent Multiway Merging Passes (k-Way Merge):** At each level of the recursion tree, existing runs are partitioned into groups of size k. Each group of k runs, each of current length l, is merged simultaneously to yield a single consolidated run of size $k \cdot l$:
   - I/O cost for merging a single group of runs: $\mathcal{O}\left(\frac{k \cdot l}{B}\right)$.
   - Total I/O cost per merge level: because every element belongs to exactly one group per level, all n elements are read and written exactly once per pass, resulting in:

$$\mathrm{I/O}_{\mathrm{Level}} = 2 \cdot \frac{n}{B} = \mathcal{O}\left(\frac{n}{B}\right). \tag{4.3}$$

At the i-th level of the multiway merge tree, every run has size $k^i \cdot M$. The merge process terminates when only a single consolidated run of size n remains:

$$k^i \cdot M = n \implies k^i = \frac{n}{M} \implies i = \log_k\left(\frac{n}{M}\right) \text{ levels.} \tag{4.4}$$

Setting the fan-in to its maximum feasible capacity $k \approx \frac{M}{B} - 1 = \Theta\left(\frac{M}{B}\right)$, we obtain:

$$\text{Number of Merge Passes} = \mathcal{O}\left(\log_{M/B}\left(\frac{n}{M}\right)\right). \tag{4.5}$$

In Figure 4.2, we illustrate the hierarchy of k-way merge levels, highlighting the expanded fan-in and per-level I/O scan costs throughout the recursion tree.

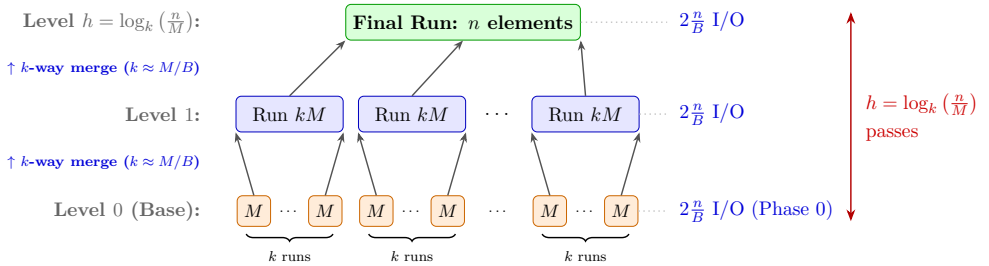


Figure 4.2: Level evolution and k-way merge tree in Multiway Merge Sort.

Combining both phases gives the overall I/O complexity:

$$\begin{aligned} \mathrm{I/O}_{\mathrm{Total}} = & \underbrace{\mathcal{O}\left(\frac{n}{B}\right)}_{\text{Run Formation}} + \underbrace{\mathcal{O}\left(\frac{n}{B}\log_{M/B}\left(\frac{n}{M}\right)\right)}_{\text{Merging Passes}} \\ & = \mathcal{O}\left(\frac{n}{B}\log_{M/B}\left(\frac{n}{B}\right)\right) \text{ I/O operations.} \end{aligned} \tag{4.6}$$

This matches the asymptotic lower bound for hierarchical memory sorting established by Aggarwal and Vitter [2] (whose rigorous formal proof via the I/O decision tree is developed in detail in Lecture 5, section 5.2), proving that Multiway Merge Sort is asymptotically optimal.

4.2 Optimizing Run Formation: The Snow Plow (*Replacement Selection*)

While standard Multiway Merge Sort creates initial runs of length bounded by memory capacity M, the tree depth $\lceil \log_{M/B}(n/M) \rceil$ depends directly on the initial number of runs. Producing runs substantially longer than M flattens the merge tree and can eliminate an entire I/O pass across petabyte-scale datasets.

A foundational technique designed to achieve this is **Replacement Selection**, historically known by the vivid metaphor of the **Snow Plow** [1].

4.2.1 Operational Mechanism of Replacement Selection

Rather than statically filling memory M, sorting it, and flushing it in bulk, the algorithm keeps fast memory M fully occupied at all times by dynamically splitting it into two logical partitions:

- **Active Elements (Current Min-Heap):** elements whose keys are greater than or equal to the last element emitted to the current run on disk. These elements are valid candidates to append to the active run.
- **Frozen Elements (U):** newly ingested elements whose keys are strictly smaller than the last element emitted. Appending them would violate the sorted invariant; hence they are "frozen" in RAM, waiting for the subsequent run.

Figure 4.3 shows the dynamic memory partitioning during the snow plow sweep.

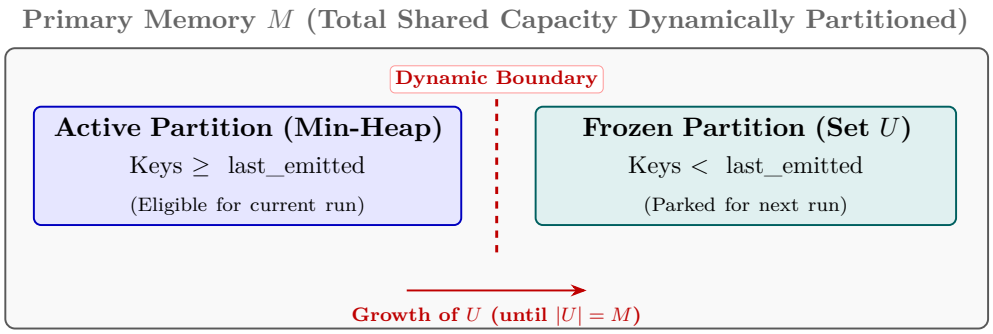


Figure 4.3: Dynamic partitioning of memory M during Replacement Selection (Snow Plow).

The algorithm executes the following element-wise loop:

1. Extract the minimum active element from the root of the Min-Heap, write it to the output buffer, and record its key as last_emitted.
2. Ingest the next item x from the unsorted input stream:
   - If $x \geq$ last_emitted, insert x into the active Min-Heap.
   - If $x <$ last_emitted, assign x to the frozen set U.
3. The active run terminates when the active heap becomes empty, i.e., when all M memory slots are occupied by frozen elements ($|U| = M$). At this moment, set U is transformed into the new active Min-Heap, a new run file is opened on disk, and the cycle repeats.

4.2.2 Step-by-Step Execution Trace

Consider internal memory capacity $M = 3$ and an input stream:

$$S = [1, 7, 5, 3, 2, 9, 0, 4, \ldots].$$

Table 4.1 traces the step-by-step state of memory and the output stream.

Table 4.1: Execution trace of Replacement Selection with capacity $M = 3$.

Step	Active	Frozen (U)	Action Performed	Emitted	Run
0	$\{1, 5, 7\}$	$\emptyset$	Initial load of $M = 3$ items	—	[ ]
1	$\{5, 7\}$	$\emptyset$	Emit 1; read 3. Since $3 \geq 1$, remains active	**1**	[1]
2	$\{5, 7\}$	$\emptyset$	Emit 3; read 2. Since $2 < 3$, 2 **goes to** U	**3**	[1, 3]
3	$\{7, 9\}$	$\{2\}$	Emit 5; read 9. Since $9 \geq 5$, remains active	**5**	[1, 3, 5]
4	$\{9\}$	$\{2, 0\}$	Emit 7; read 0. Since $0 < 7$, 0 **goes to** U	**7**	[1, 3, 5, 7]
5	$\emptyset$	$\{2, 0, 4\}$	Emit 9; read 4. Since $4 < 9$, 4 **goes to** U	**9**	[1, 3, 5, 7, 9]

At Step 5, active memory is completely depleted and frozen items fill all M slots: $|U| = M = 3$. The current run closes with a length of **5 items**—significantly exceeding physical memory capacity $M = 3$. The frozen set $\{0, 2, 4\}$ immediately forms the seed heap for the subsequent run.

Note on Production Systems. In production databases and file systems, transfers are performed using blocks of size B rather than single elements, utilizing prefetching buffers and write accumulation buffers (*double buffering*) without altering the asymptotic I/O behavior.

4.2.3 Average Run Length Theorem

Theorem 4.2.1 (Average Run Length in Snow Plow)**.** *Assuming that input keys are independent and identically distributed random variables drawn from a continuous uniform distribution, the average run length produced by Replacement Selection is asymptotically* $2M$*.*

Heuristic Proof. We analyze the steady-state stochastic process:

1. At the start of a run, memory holds M elements.
2. Let τ denote the number of additional items read from the input before the run ends. The total run length is $M + \tau$.
3. The run concludes when the frozen buffer reaches full capacity: $|U| = M$.

4. By distributional symmetry and statistical independence, each incoming element has probability $P = \frac{1}{2}$ of being smaller than the last emitted element (landing in U) and probability $P = \frac{1}{2}$ of being greater than or equal to it (joining the active run).

5. The expected number of items placed in U after reading τ elements is:

$$\mathbb{E}[|U|] = \tau \cdot P(\text{placed into } U) = \tau \cdot \frac{1}{2}. \tag{4.7}$$

6. Equating this expectation to the termination threshold $\mathbb{E}[|U|] = M$:

$$\frac{\tau}{2} = M \implies \tau = 2M. \tag{4.8}$$

Consequently, the expected average run length is approximately $2M$. □

Doubling the average run length ($\approx 2M$) cuts the initial number of runs in half $\left(\frac{n}{2M}\right)$, directly decreasing the required number of full I/O passes across the dataset.

4.3 Parallelism Across Multiple Disks: *Disk Striping*

In modern data centers and high-performance computing clusters, secondary storage is distributed across D independent physical disk drives or SSD units.

4.3.1 Parallel External Sorting Lower Bound

When a storage architecture incorporates D parallel disk units, each capable of transferring one block B per I/O cycle, the Aggarwal-Vitter [2] lower bound scales with an ideal factor D in the denominator:

$$\text{Sort}_D(n, M, B) = \Omega\left(\frac{n}{D \cdot B} \log_{M/B}\left(\frac{n}{M}\right)\right) \text{ parallel I/O steps.} \tag{4.9}$$

However, designing an algorithm that achieves this lower bound using asynchronous independent disks is remarkably intricate: one must continuously prevent disk contention and bottlenecking where access concentrates on a single drive while $D - 1$ drives sit idle.

4.3.2 The Disk Striping Technique

To exploit physical parallelism without introducing complex asynchronous disk schedulers, systems employ **Disk Striping** (analogous to RAID 0 architecture):

- The D physical disks $d_1, d_2, \ldots, d_D$ are logically aggregated into a single unified volume.
- A **logical super-block** of aggregate size:

$$B' = D \cdot B \tag{4.10}$$

  is defined by combining one physical block of size B stored at the identical track or cylinder position on each of the D drives.
- Every I/O transfer operates on an entire super-block of size B', distributing the I/O load synchronously and uniformly across all D channels in a single parallel transfer cycle.

Figure 4.4 illustrates the mapping of super-block B' onto the D underlying physical disks.

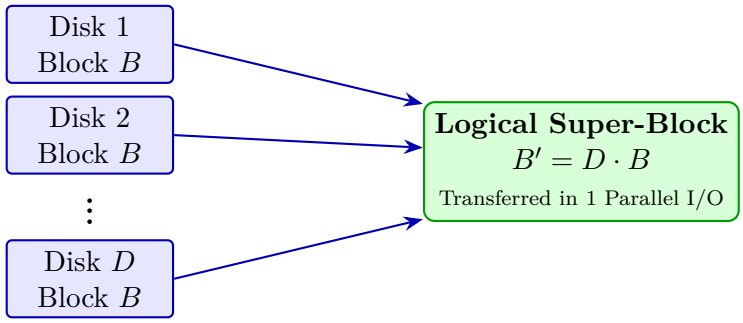


Figure 4.4: Disk striping layout: aggregating D physical blocks into a single logical super-block B'.

Executing Multiway Merge Sort using logical block size B', the number of super-blocks is $\frac{n}{B'} = \frac{n}{D \cdot B}$, while the memory fan-in capacity becomes $\frac{M}{B'} = \frac{M}{D \cdot B}$. The parallel I/O complexity is:

$$\mathrm{I/O}_{\mathrm{Striping}} = \mathcal{O}\left(\frac{n}{B'} \log_{M/B'}\left(\frac{n}{M}\right)\right) = \mathcal{O}\left(\frac{n}{D \cdot B} \log_{\frac{M}{D \cdot B}}\left(\frac{n}{M}\right)\right). \tag{4.11}$$

4.3.3 Slowdown Analysis

Although Disk Striping achieves full bandwidth utilization across all D channels, it reduces the available merging fan-in from $\frac{M}{B}$ down to $\frac{M}{D \cdot B}$, because each open run requires a buffer of size $B' = D \cdot B$.

To quantify this loss of efficiency relative to the optimal independent-disk lower bound, we compute the ratio of their I/O complexities (*Slowdown*):

$$\text{Slowdown} = \frac{\text{I/O}_{\text{Striping}}}{\text{I/O}_{\text{Ideal}}} = \frac{\frac{n}{D \cdot B} \log_{\frac{M}{D \cdot B}} \left(\frac{n}{M}\right)}{\frac{n}{D \cdot B} \log_{\frac{M}{B}} \left(\frac{n}{M}\right)} = \frac{\log_{\frac{M}{D \cdot B}} \left(\frac{n}{M}\right)}{\log_{\frac{M}{B}} \left(\frac{n}{M}\right)}. \tag{4.12}$$

Changing the logarithm base to 2 using $\log_a x = \frac{\log_2 x}{\log_2 a}$:

$$\text{Slowdown} = \frac{\frac{\log_2(n/M)}{\log_2(M/(D \cdot B))}}{\frac{\log_2(n/M)}{\log_2(M/B)}} = \frac{\log_2(M/B)}{\log_2(M/(D \cdot B))} = \frac{\log_2(M/B)}{\log_2(M/B) - \log_2 D}. \tag{4.13}$$

Dividing numerator and denominator by $\log_2(M/B)$:

$$\text{Slowdown} = \frac{1}{1 - \frac{\log_2 D}{\log_2(M/B)}}. \tag{4.14}$$

4.3.4 Asymptotic Regimes and Physical Limitations

The behavior of the slowdown factor is governed entirely by the dimensionless ratio:

$$\rho = \frac{\log_2 D}{\log_2(M/B)}. \tag{4.15}$$

Two limiting operating regimes arise:

1. **Ample Memory Regime ($M \to \infty$ or $D \ll \frac{M}{B}$):** When RAM is substantially larger than the number of attached disk drives, the memory fan-in factor dominates:

$$\rho = \frac{\log_2 D}{\log_2(M/B)} \to 0 \implies \text{Slowdown} \to \frac{1}{1 - 0} = 1. \tag{4.16}$$

   In this regime, Disk Striping suffers virtually no degradation and attains the asymptotic performance of the theoretical independent-disk system, representing an optimal engineering tradeoff due to its implementation simplicity.

2. **Memory Saturation Regime ($\log_2 D \to \log_2(M/B)$):** When the drive count D approaches the maximum number of blocks that fit into internal memory ($D \approx \frac{M}{B}$), the ratio ρ approaches 1. The denominator of (4.14) tends to zero, causing the expression to diverge:

$$\lim_{\rho \to 1^-} \frac{1}{1-\rho} = +\infty. \tag{4.17}$$

Physical interpretation: If $D \geq \frac{M}{B}$, internal memory cannot accommodate even a single super-block $B' = D \cdot B$ per active input stream, rendering striping physically impossible and causing the multiway merge to collapse.

Chapter 5

Lower Bounds for Sorting, Dual-Pivot Quicksort, and Bounded Quicksort

Lecture 5
Lower Bounds (RAM vs Two-Level Model), Dual-Pivot Quicksort, and Bounded Quicksort

Date: September 28, 2026

This chapter investigates the theoretical and engineering analysis of complexity lower bounds for the fundamental problem of comparison-based sorting, contrasting the classical RAM model with the two-level memory model of Aggarwal and Vitter. Subsequently, the focus shifts to modern architectural developments of the *Quicksort* algorithm: we analyze the duplicate keys problem and the *Dual-Pivot Quicksort* algorithm introduced by Yaroslavskiy, followed by a formal treatment of the *Bounded Quicksort* (BQS) variant. We rigorously prove how tail recursion elimination and bounded partitioning guarantee an auxiliary stack space strictly bounded by $\Theta(\log_2 n)$, while achieving optimal cache efficiency via an *Insertion Sort* cutoff.

5.1 RAM Model: Lower Bounds and Decision Trees

In the standard RAM (*Random Access Machine*) model of computation, a comparison-based algorithm cannot inspect the underlying bitwise representation of keys to extract digits or radix components (as done in *Radix Sort*

algorithms). Instead, it may only acquire information regarding the relative ordering of elements via binary comparisons of the form:

$$A < B, \quad A = B, \quad \text{or} \quad A > B. \tag{5.1}$$

Assuming, without loss of generality for lower bound derivations, that all input keys are pairwise distinct, each elementary comparison yields a strictly dichotomous outcome ($<$ or $>$).

5.1.1 The Decision Tree Model

The family of deterministic comparison-based algorithms is formally modeled through the abstract mathematical structure of a **Decision Tree**:

- **Internal nodes:** Each internal node models a single elementary comparison between a pair of input elements or positions, conventionally denoted by $[A : B]$. Since all keys are distinct, exactly two directed edges leave the node, corresponding to the two mutually exclusive outcomes: the left edge associated with $A < B$ and the right edge with $A > B$. Consequently, the decision tree is a proper binary tree.
- **Root-to-leaf path:** For any given input instance of size n, the deterministic algorithm traces a unique directed path from the root down to a leaf. Each node visited along this trajectory corresponds to a comparison executed by the algorithm; the length of the path (the number of traversed edges) reflects the exact comparison count for that specific configuration. The length of the longest root-to-leaf path corresponds to the **height** t of the decision tree and quantifies the worst-case running time $T(n)$.
- **Leaves:** Each leaf corresponds to algorithmic termination and encapsulates the final solution or output configuration of the computational problem (e.g., the permutation of indices that sorts the array, or the index at which the target key is located).

Figure 5.1 illustrates the general topological structure of a binary decision tree for a generic comparison-based problem.

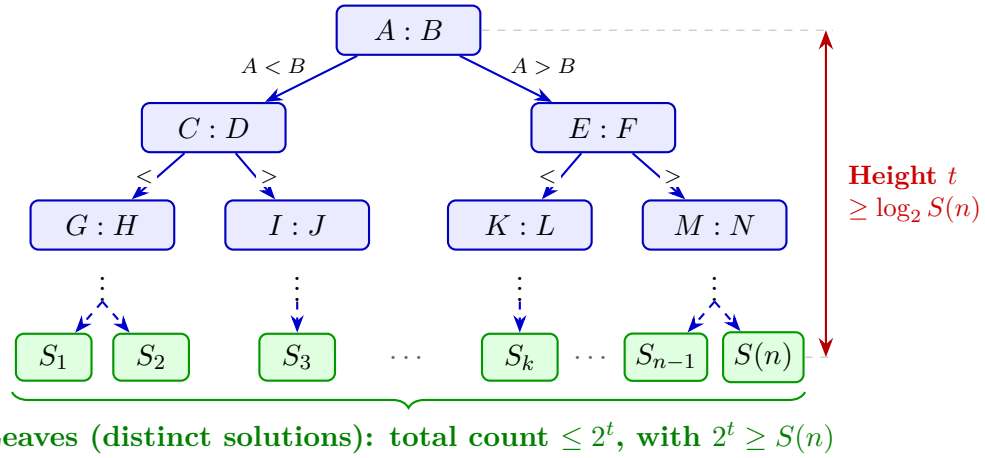


Figure 5.1: Binary decision tree model for comparison-based algorithms. Internal nodes represent elementary pairwise comparisons; leaves represent admissible distinct solutions $(S_1, S_2, \ldots, S(n))$. The longest path determines the worst-case running time.

5.1.2 The General Information-Theoretic Lower Bound

The fundamental relationship between the number of distinct solutions of a problem and the minimum number of comparisons necessary to solve it is established by the following information-theoretic theorem.

Theorem 5.1.1 (General Information-Theoretic Lower Bound)**.** *Let P be a computational problem solvable via a deterministic comparison-based algorithm. If for an input of size n the problem admits $S(n)$ distinct possible solutions (each of which must be returned by the algorithm for suitable input configurations), then any deterministic comparison-based algorithm requires in the worst case:*

$$T(n) = \Omega(\log_2 S(n)). \tag{5.2}$$

Proof. Consider the binary decision tree associated with an arbitrary correct deterministic algorithm for problem P. Let t denote the height of the tree, representing the maximum length of any root-to-leaf path.

Because comparisons are strictly binary (assuming distinct keys), every internal node has at most 2 children. By induction on height, a binary tree of height t possesses at most L leaves, bounded by:

$$L \leq 2^t. \tag{5.3}$$

For the algorithm to be correct on every valid input instance, the decision tree must be capable of reaching each of the $S(n)$ distinct solutions. Consequently,

each distinct solution must be associated with at least one distinct leaf in the tree (if two distinct solutions mapped to the same leaf, the algorithm would fail to discriminate between them, outputting an erroneous answer for at least one of the instances). Hence, we must have:

$$2^t \geq L \geq S(n). \tag{5.4}$$

Applying the base-2 logarithm (a strictly monotonically increasing function) to both sides yields:

$$t \geq \log_2 S(n). \tag{5.5}$$

Since the worst-case time complexity $T(n)$ is defined as the height t of the tree, we conclude that:

$$T(n) \geq \log_2 S(n) \implies T(n) = \Omega(\log_2 S(n)). \tag{5.6}$$

□

5.1.3 Fundamental Applications: Searching and Sorting

Theorem 5.1.1 directly yields canonical lower bounds for two core computational problems in internal memory:

1. **Searching a Key (*Searching*):** Given a sorted array of n elements and a target key K, the algorithm must determine whether K resides in the array and, if so, return its index $i \in \{1, 2, \ldots, n\}$; if absent, it returns an explicit failure symbol $\perp$. The set of distinct valid outputs is:

   $$S(n) = \{1, 2, \ldots, n\} \cup \{\perp\} \implies |S(n)| = n + 1. \tag{5.7}$$

   Applying the theorem:

   $$T_{\text{Search}}(n) \geq \log_2(n + 1) = \Omega(\log_2 n). \tag{5.8}$$

   This lower bound establishes the asymptotic optimality of *Binary Search*, which operates in exactly $\Theta(\log_2 n)$ comparisons.

2. **Sorting (*Sorting*):** Given an array of n distinct elements, sorting is equivalent to identifying which of the $n!$ permutations of indices rearranges the keys in non-decreasing order. Since the set $\{1, 2, \ldots, n\}$ admits exactly $n!$ distinct permutations, the number of possible solutions is:

   $$S(n) = n!. \tag{5.9}$$

Applying the lower bound theorem:

$$T_{\text{Sort}}(n) \geq \log_2(n!). \tag{5.10}$$

To express this bound in explicit asymptotic form, we invoke **Stirling's approximation** for the factorial:

$$n! \approx \sqrt{2\pi n}\left(\frac{n}{e}\right)^n \implies \ln(n!) = n\ln n - n + \mathcal{O}(\ln n). \tag{5.11}$$

Converting to base-2 logarithm:

$$\log_2(n!) = n\log_2 n - n\log_2 e + \Theta(\log_2 n) = \Theta(n\log_2 n). \tag{5.12}$$

Alternatively, using an elementary lower bound over the upper half of terms:

$$n! \geq \prod_{i=\lceil n/2\rceil}^{n} i \geq \left(\frac{n}{2}\right)^{n/2}, \tag{5.13}$$

$$\log_2(n!) \geq \frac{n}{2}\log_2\left(\frac{n}{2}\right) = \frac{n}{2}(\log_2 n - 1) = \Omega(n\log_2 n). \tag{5.14}$$

Therefore:

$$T_{\text{Sort}}(n) = \Omega(n\log_2 n). \tag{5.15}$$

Any deterministic or randomized comparison-based sorting algorithm (such as *Merge Sort*, *Heap Sort*, or the expected time of *Quicksort*) cannot surpass the asymptotic barrier $\Omega(n\log_2 n)$ in the RAM model.

5.2 Two-Level Memory: Aggarwal-Vitter Lower Bound

In the two-level memory model introduced by Aggarwal and Vitter [2], the primary computational cost is not measured by elementary CPU comparisons, but by the total number t of I/O operations, each transferring a contiguous block of B elements between slow external storage and internal RAM of capacity M items.

5.2.1 The Nature of I/O and Free In-Memory Comparisons

The standard binary decision tree cannot be directly applied to external memory analysis:

- A single disk read operation simultaneously brings an entire block of B elements into fast internal memory.
- Once these B elements reside in RAM alongside the remaining $M - B$ elements already present, the CPU can execute **an arbitrary number of in-memory comparisons at zero I/O cost**.
- These "free" internal comparisons permit sorting all M items currently in RAM, drastically pruning the set of candidate global permutations consistent with the current knowledge.

Consequently, to establish an external memory lower bound, one must formulate an **I/O Decision Tree**, where each node represents an I/O read operation (loading a block of size B), and outgoing edges correspond to all possible relative arrangements resulting from interleaving the B retrieved elements with the $M - B$ elements already residing in RAM.

5.2.2 Branching Factor (Fan-out) Calculation of the I/O Tree

When a block of B elements is transferred from disk into internal memory (accommodating M elements total, consisting of $M - B$ pre-existing elements and B newly loaded items), we distinguish two operational scenarios:

1. **Reading a "New Block" (*New Block*):** The block is transferred from disk for the very first time: its elements have never been observed before by any previous step of the algorithm.
   - The mutual relative order of the B elements is completely unknown: they can appear in any of the $B!$ possible permutations.
   - Once placed in RAM, the B new items must be interleaved with the $M - B$ pre-existing items to establish their combined sorted order within the M slots. The number of ways to interleave B elements into M positions is given by the binomial coefficient:

$$\binom{M}{B} = \frac{M!}{B!\,(M-B)!}. \tag{5.16}$$

   Multiplying the $B!$ internal ordering possibilities by the $\binom{M}{B}$ interleaving positions, the branching factor (*fan-out*) X for reading a new block is:

$$X = (B!) \cdot \binom{M}{B}. \tag{5.17}$$

2. **Re-reading an "Old Block" (*Old Block*):** The block was previously read, internally sorted, and written back to disk during an earlier phase of computation (such as an intermediate run generated during multiway merging).

   - The internal relative order among its B elements is already fully known; hence, no new information is acquired regarding their intrinsic ordering (a factor of 1 rather than $B!$).
   - The only new information gained is how these B elements interleave with the other $M - B$ items currently in RAM.

   The branching factor (*fan-out*) Y for re-reading an old block is therefore:

$$Y = \binom{M}{B}. \tag{5.18}$$

Figure 5.2 summarizes the logical dichotomy and branch cardinalities for both I/O transfer types.

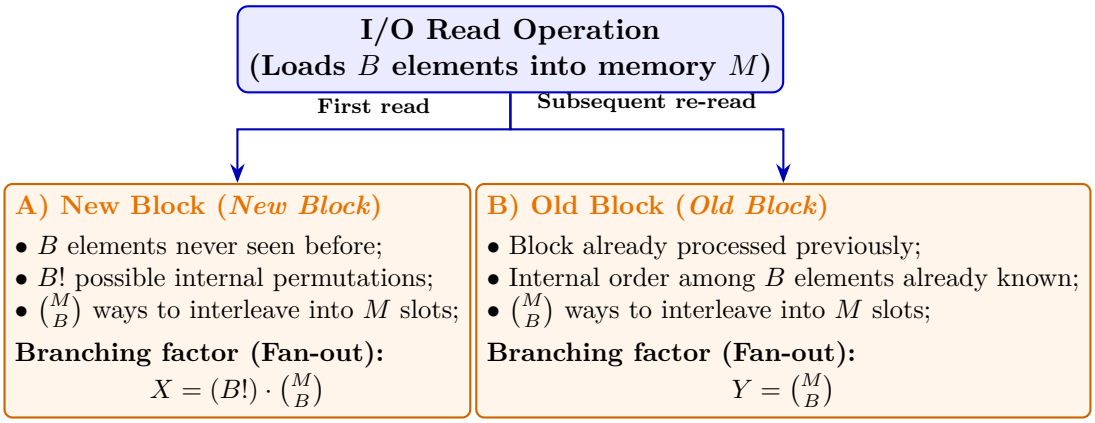


Figure 5.2: Branching factors (*fan-out*) in the I/O decision tree under the two-level memory model: reading a new block yields internal order information ($B!$) and interleaving information ($\binom{M}{B}$); re-reading an old block yields only interleaving information.

5.2.3 Formal Derivation of the Aggarwal-Vitter Lower Bound

Consider an input file consisting of n distinct elements stored across $N = \frac{n}{B}$ disk blocks. In order to sort the file, any correct algorithm must inspect each

element at least once; consequently, it must execute at least $\frac{n}{B}$ "New Block" read operations.

If the algorithm executes a total of t I/O operations in the worst case, then:

- Exactly $\frac{n}{B}$ operations will be of type "New Block", each with fan-out at most X.
- The remaining $t - \frac{n}{B}$ operations will be (at most) of type "Old Block", each with fan-out at most Y.

The total number of leaves in the I/O decision tree represents the maximum number of global permutations distinguishable by this sequence of transfers. For the algorithm to correctly sort any valid instance, this leaf count must be at least equal to $n!$:

$$X^{\frac{n}{B}} \cdot Y^{\left(t-\frac{n}{B}\right)} \geq n!. \tag{5.19}$$

Substituting the explicit formulas for X and Y from equations (5.17) and (5.18):

$$\left[(B!) \cdot \binom{M}{B}\right]^{\frac{n}{B}} \cdot \left[\binom{M}{B}\right]^{t-\frac{n}{B}} \geq n!. \tag{5.20}$$

Collecting powers sharing the common base $\binom{M}{B}$:

$$(B!)^{\frac{n}{B}} \cdot \binom{M}{B}^{\frac{n}{B}} \cdot \binom{M}{B}^{t-\frac{n}{B}} \geq n! \implies (B!)^{\frac{n}{B}} \cdot \binom{M}{B}^{t} \geq n!. \tag{5.21}$$

Taking the base-2 logarithm on both sides of inequality (5.21):

$$\log_2\left((B!)^{\frac{n}{B}}\right) + \log_2\left(\binom{M}{B}^{t}\right) \geq \log_2(n!), \tag{5.22}$$

which yields:

$$\frac{n}{B}\log_2(B!) + t\log_2\binom{M}{B} \geq \log_2(n!). \tag{5.23}$$

To expand the logarithmic expressions, we use standard asymptotic bounds for the factorial and the binomial coefficient:

- By Stirling's approximation: $\log_2(a!) \approx a\log_2 a - a\log_2 e \approx a\log_2 a$.
- For the binomial coefficient, recalling that $\binom{a}{b} \leq \left(\frac{e\cdot a}{b}\right)^b$:

$$\log_2\binom{M}{B} \approx B\log_2\left(\frac{M}{B}\right) + \mathcal{O}(B). \tag{5.24}$$

Substituting these expressions into equation (5.23):

$$\frac{n}{B} \cdot (B \log_2 B) + t \cdot \left(B \log_2 \frac{M}{B} \right) \geq n \log_2 n. \tag{5.25}$$

Simplifying the factor B in the first term:

$$n \log_2 B + t \cdot B \log_2 \left(\frac{M}{B} \right) \geq n \log_2 n. \tag{5.26}$$

Isolating the term containing the unknown t:

$$t \cdot B \log_2 \left(\frac{M}{B} \right) \geq n \log_2 n - n \log_2 B = n \left(\log_2 n - \log_2 B \right) = n \log_2 \left(\frac{n}{B} \right). \tag{5.27}$$

Dividing both sides by the strictly positive quantity $B \log_2 \left(\frac{M}{B} \right)$:

$$t \geq \frac{n \log_2 \left(\frac{n}{B} \right)}{B \log_2 \left(\frac{M}{B} \right)} = \frac{n}{B} \cdot \frac{\log_2(n/B)}{\log_2(M/B)}. \tag{5.28}$$

Applying the change of base formula for logarithms ($\frac{\log_2 a}{\log_2 b} = \log_b a$), we formally obtain the renowned Aggarwal-Vitter lower bound theorem.

Theorem 5.2.1 (Aggarwal-Vitter Lower Bound for External Sorting)**.** *Under the two-level memory model, any deterministic or randomized comparison-based sorting algorithm requires in the worst case:*

$$I/O_{Sort}(n) = \Omega \left(\frac{n}{B} \log_{M/B} \left(\frac{n}{B} \right) \right) = \Omega \left(\frac{n}{B} \frac{\log(n/B)}{\log(M/B)} \right) \ I/Os. \tag{5.29}$$

Property 5.2.1 (Asymptotic Optimality of Multiway Merge Sort)**.** *The Aggarwal-Vitter lower bound demonstrates that the* ***Multiway Merge Sort*** *(k-Way Merge Sort with merge degree $k = \frac{M}{B} - 1$) analyzed in Lecture 4 (section 4.1) is* ***asymptotically optimal****: the total number of I/O operations required to sort the array matches the fundamental theoretical lower bound dictated by information theory in external memory.*

5.3 Quicksort: Duplicate Handling and Dual-Pivot Quicksort

Quicksort is a *Divide and Conquer* algorithm based on distribution (*distribution-based*): rather than rigidly partitioning the array by index positions (as in *Merge Sort*) and then merging the sorted subproblems, Quicksort partitions the data based on the relative magnitude of one or more privileged keys termed **pivots**, delegating all sorting work to the partitioning phase.

5.3.1 The Equal Values Problem

In traditional textbook implementations of Quicksort employing standard two-way partitioning (*2-way partitioning*, such as Lomuto or Hoare schemes):

- The array is divided into two parts: elements $\leq p$ on one side and elements $> p$ on the other.
- Whenever the array contains many identical keys equal to pivot p (in the extreme case, an array containing only identical keys, $S[i] = c, \forall i$), such a scheme places all duplicates into the same partition.
- As a consequence, the subproblem shrinks by only a single element per recursive call (1 versus $n - 1$), collapsing the recursion tree into a degenerate chain of height $\Theta(n)$ and the overall running time to quadratic complexity:

$$T(n) = T(n-1) + \Theta(n) = \Theta(n^2). \tag{5.30}$$

Definition 5.3.1 (Three-Way Partitioning (Dutch National Flag))**.** The classical remedy for duplicate keys, formulated by Edsger Dijkstra [3] as the Dutch National Flag problem, partitions the array into **three contiguous regions**:

$$[\,< p \quad | \quad = p \quad | \quad > p\,]. \tag{5.31}$$

Elements strictly equal to the pivot ($= p$) are already in their final sorted positions and are **excluded from further recursive invocations**, which are executed solely on the $[< p]$ and $[> p]$ subranges. On arrays with all equal keys, this achieves linear running time $\mathcal{O}(n)$.

5.3.2 3-Way Partitioning Algorithm and Numerical Execution Example

To practically implement three-way partitioning (*Dutch National Flag*) in Quicksort, the linear-time algorithm designed by Dijkstra and formalized by Sedgewick [4] is widely adopted.

Given a sub-array $S[\mathrm{lo} \ldots \mathrm{hi}]$ with the first element chosen as the pivot $p = S[\mathrm{lo}]$, the algorithm maintains three operational indices during sequential scanning:

- **Pointer** lt **(*less than*):** delimits elements strictly smaller than the pivot on the left. The invariant guarantees that $S[k] < p$ for all $\mathrm{lo} \leq k < lt$. Initially, $lt = \mathrm{lo}$.

- **Pointer i (scan cursor):** scans the array elements. The invariant ensures that $S[k] = p$ for all $lt \leq k < i$. Initially, $i = \text{lo} + 1$.
- **Pointer gt (*greater than*):** delimits elements strictly greater than the pivot on the right. The invariant guarantees that $S[k] > p$ for all $gt < k \leq \text{hi}$. Initially, $gt = \text{hi}$.
- The sub-array $S[i \dots gt]$ contains elements that have not yet been examined.

At each iteration, as long as $i \leq gt$, the current element $S[i]$ is compared with the pivot p:

1. **If $S[i] < p$:** swap $S[lt]$ and $S[i]$, then increment both indices ($lt \leftarrow lt+1$, $i \leftarrow i + 1$). Because by invariant cell $S[lt]$ held an element equal to p, swapping moves the smaller element to the left and places a pivot duplicate at $S[i]$, allowing i to advance.
2. **If $S[i] > p$:** swap $S[i]$ and $S[gt]$, then decrement only gt ($gt \leftarrow gt - 1$). The index i **does not advance**, because the element just swapped from $S[gt]$ has not yet been compared.
3. **If $S[i] == p$:** the element is already in the correct central region; increment only the scan cursor ($i \leftarrow i + 1$).

Upon termination ($i > gt$), the array is partitioned into three distinct contiguous regions: $S[\text{lo} \dots lt-1] < p$, $S[lt \dots gt] = p$, and $S[gt+1 \dots \text{hi}] > p$. Subsequent recursive calls are issued **exclusively** on $S[\text{lo} \dots lt - 1]$ and $S[gt + 1 \dots \text{hi}]$, bypassing the entire duplicate block. The formal dimensional invariants of these partitions and their integration with the order statistics selection algorithm (*Quickselect*) and expected-time recurrence relation are explored in depth in Lecture 6 (section 6.3).

Table 5.1 details the step-by-step progress of the three pointers and array contents throughout the execution with pivot $p = 4$.

Step	i	lt	gt	Comparison	Action	Array State
Init	1	0	7	Pivot $p = 4$	Initialization	$[4, 7, 2, 4, 1, 4, 8, 3]$
1	1	0	7	$S[1] = 7 > 4$	SWAP$(S[1], S[7])$, $gt \leftarrow 6$	$[4, 3, 2, 4, 1, 4, 8, 7]$
2	1	0	6	$S[1] = 3 < 4$	SWAP$(S[0], S[1])$, $lt \leftarrow 1$, $i \leftarrow 2$	$[3, 4, 2, 4, 1, 4, 8, 7]$
3	2	1	6	$S[2] = 2 < 4$	SWAP$(S[1], S[2])$, $lt \leftarrow 2$, $i \leftarrow 3$	$[3, 2, 4, 4, 1, 4, 8, 7]$
4	3	2	6	$S[3] == 4$	Advance $i \leftarrow 4$	$[3, 2, 4, 4, 1, 4, 8, 7]$
5	4	2	6	$S[4] = 1 < 4$	SWAP$(S[2], S[4])$, $lt \leftarrow 3$, $i \leftarrow 5$	$[3, 2, 1, 4, 4, 4, 8, 7]$
6	5	3	6	$S[5] == 4$	Advance $i \leftarrow 6$	$[3, 2, 1, 4, 4, 4, 8, 7]$
7	6	3	6	$S[6] = 8 > 4$	SWAP$(S[6], S[6])$, $gt \leftarrow 5$	$[3, 2, 1, 4, 4, 4, 8, 7]$
End	**6**	**3**	**5**	$i > gt$	**Termination**	$[\mathbf{3, 2, 1} \mid \mathbf{4, 4, 4} \mid \mathbf{8, 7}]$

Table 5.1: Detailed step-by-step trace of 3-way partitioning with pivot $p = 4$. At termination, the three identical keys equal to 4 occupy indices $lt \dots gt$ $(3 \dots 5)$ and are bypassed by subsequent recursive calls.

5.3.3 Architecture of Dual-Pivot Quicksort (Yaroslavskiy)

In 2009, Russian engineer Vladimir Yaroslavskiy proposed an innovative algorithm called **Dual-Pivot Quicksort** [5]. Following extensive empirical benchmarks conducted in collaboration with Joshua Bloch and Robert Sedgewick, the algorithm replaced classic single-pivot Quicksort in Java's standard library ('java.util.Arrays.sort') for all primitive data types.

Unlike classical Quicksort, Dual-Pivot Quicksort chooses **two pivots** P and Q. It first ensures $P \leq Q$ (swapping them if necessary). The active subarray range $S[\text{left} \dots \text{right}]$ is dynamically divided into four regions managed via three operational indices (l, k, g):

- l (*less*): upper bound of elements strictly smaller than P ($S[i] < P$ for all $\text{left} \leq i < l$).
- k: current scanning pointer ($l \leq k \leq g$). Elements between l and $k - 1$ satisfy the central invariant: $P \leq S[i] \leq Q$.
- g (*great*): lower bound of elements strictly greater than Q ($S[i] > Q$ for all $g < i \leq \text{right}$).
- The region between k and g holds items not yet examined.

Figure 5.3 illustrates the array layout during the execution of dual-pivot partitioning.

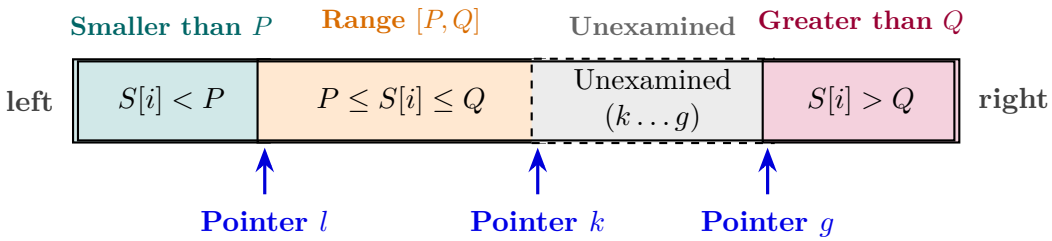


Figure 5.3: Dual-Pivot Quicksort partitioning schema (Yaroslavskiy). The array is partitioned into four dynamic regions: elements $< P$, elements between P and Q, unexamined elements between k and g, and elements $> Q$.

5.3.4 Invariants and Transition Logic

The partitioning procedure maintains the invariant until pointer k crosses g ($k \leq g$). At each step, the element $S[k]$ is evaluated:

1. **Case 1:** $S[k] < P$**:** The element belongs to the leftmost partition. It is swapped with $S[l]$, and both pointers are incremented:

$$\text{SWAP}(S[k], S[l]), \quad l \leftarrow l + 1, \quad k \leftarrow k + 1. \tag{5.32}$$

   Since by invariant the element formerly at $S[l]$ was within the middle range $[P, Q]$, the element moved to position k is guaranteed to belong to $[P, Q]$, allowing k to safely advance.

2. **Case 2:** $S[k] > Q$**:** The element belongs to the rightmost partition. Before swapping, pointer g is moved backward while $S[g] > Q$ (skipping elements already correctly placed on the right). Once an element $S[g] \leq Q$ is found, a swap is performed:

$$\text{SWAP}(S[k], S[g]), \quad g \leftarrow g - 1. \tag{5.33}$$

   The element now at $S[k]$ (retrieved from g) has not yet been compared against P; hence it is **re-examined on the next step without advancing** k.

3. **Case 3:** $P \leq S[k] \leq Q$**:** The element is already in its proper position in the middle partition. No swap is required, and k simply advances:

$$k \leftarrow k + 1. \tag{5.34}$$

When scanning terminates ($k > g$), the two pivots P and Q are swapped into their final positions $l-1$ and $g+1$, respectively. Three independent recursive calls are then launched on the three remaining subranges:

$$[\text{left} \dots l-2], \quad [l \dots g], \quad [g+2 \dots \text{right}]. \tag{5.35}$$

Algorithm 3: Dual-Pivot Partitioning (Yaroslavskiy)

Input: Array S, start index left, end index right with left $<$ right
Output: Final pivot positions (p_1, p_2)

```
1  if S[left] > S[right] then
2      SWAP(S[left], S[right])
3  P ← S[left]
4  Q ← S[right]
5  l ← left + 1
6  g ← right − 1
7  k ← l
8  while k ≤ g do
9      if S[k] < P then
10         SWAP(S[k], S[l])
11         l ← l + 1
12         k ← k + 1
13     else if S[k] > Q then
14         while S[g] > Q and k < g do
15             g ← g − 1
16         SWAP(S[k], S[g])
17         g ← g − 1
18         if S[k] < P then
19             SWAP(S[k], S[l])
20             l ← l + 1
21         k ← k + 1
22     else
23         k ← k + 1
24 l ← l − 1
25 g ← g + 1
26 SWAP(S[left], S[l])
27 SWAP(S[right], S[g])
28 return (l, g)
```

5.3.5 Execution Example and Trace of Dual-Pivot Quicksort

To illustrate the mechanics of the dual-pivot partitioning scheme described in algorithm 3, consider an array of 8 elements:

$$S = [3,\ 8,\ 2,\ 5,\ 1,\ 4,\ 9,\ 7]. \tag{5.36}$$

Pivots are selected from the boundary positions: $P = S[0] = 3$ and $Q = S[7] = 7$. Since $P \leq Q$ ($3 \leq 7$), the preliminary pivot ordering condition holds. The cursors are initialized as: $l = 1$, $k = 1$, and $g = 6$.

In Table 5.2, the step-by-step operation sequence executed by Dual-Pivot Quicksort is summarized, highlighting the pointer movements and the final relocation of pivots P and Q.

Step	k	l	g	**Element**	**Action**	**Array State**
Init	1	1	6	Pivots $P = 3, Q = 7$	Initialize cursors	$[3, 8, 2, 5, 1, 4, 9, 7]$
1	1	1	6	$S[1] = 8 > 7$	Find $g = 5$; SWAP$(S[1], S[5])$, $g \leftarrow 4, k \leftarrow 2$	$[3, 4, 2, 5, 1, 8, 9, 7]$
2	2	1	4	$S[2] = 2 < 3$	SWAP$(S[1], S[2])$, $l \leftarrow 2, k \leftarrow 3$	$[3, 2, 4, 5, 1, 8, 9, 7]$
3	3	2	4	$3 \leq S[3] = 5 \leq 7$	Advance $k \leftarrow 4$	$[3, 2, 4, 5, 1, 8, 9, 7]$
4	4	2	4	$S[4] = 1 < 3$	SWAP$(S[2], S[4])$, $l \leftarrow 3, k \leftarrow 5$	$[3, 2, 1, 5, 4, 8, 9, 7]$
End	5	3	4	$k > g$	Stop scan	$[3, 2, 1, 5, 4, 8, 9, 7]$
Pivot	–	2	5	Relocation	SWAP$(S[0], S[2])$, SWAP$(S[7], S[5])$	$[\mathbf{1}, \mathbf{2} \mid \mathbf{3} \mid \mathbf{5}, \mathbf{4} \mid \mathbf{7} \mid \mathbf{8}, \mathbf{9}]$

Table 5.2: Step-by-step trace of Dual-Pivot partitioning with pivots $P = 3$ and $Q = 7$. Upon termination, both pivots reside at their definitive index locations (2 and 5) and recursive calls process the three sub-regions independently.

Property 5.3.1 (Architectural Benefits of Dual-Pivot Quicksort)**.** *Deploying two pivots provides substantial algorithm engineering advantages:*

- ***Reduced Recursion Tree Depth:*** *Dividing the array into* 3 *sub-problems instead of* 2 *reduces the average recursion tree height from* $\log_2 n \approx 1.44 \ln n$ *down to* $\log_3 n \approx 0.91 \ln n$*, directly lowering the total number of memory scans.*

- ***Cache Efficiency and CPU Pipelining:*** *While the theoretical comparison count increases slightly compared to single-pivot Quicksort (*$1.9n \ln n$ *vs* $2n \ln n$*), the total number of cache line invalidations and memory writes drops significantly, making superior use of modern memory hierarchies and CPU pipeline architectures.*

5.4 Bounded Quicksort (BQS) and Tail Recursion Elimination

Despite its optimal expected time complexity $\mathcal{O}(n \log_2 n)$, standard recursive Quicksort exhibits a severe vulnerability concerning auxiliary memory usage on the call stack.

5.4.1 Call Stack Vulnerability

Upon each recursive invocation, the runtime environment allocates a call stack frame containing function parameters, local variables, and the return address:

- In balanced or near-optimal partitioning scenarios, the call stack depth is $\Theta(\log_2 n)$, consuming negligible extra space.
- However, under severely unbalanced partitions (worst-case scenario: pivot located at an extreme, yielding 1 versus $n-1$), stack depth grows linearly with input size:

$$\text{Stack Depth} = \Theta(n). \tag{5.37}$$

  For large inputs ($n \geq 10^6$), linear stack growth inevitably causes an unrecoverable system abort due to a *Stack Overflow Exception*.

5.4.2 Bounded Quicksort Algorithm (BQS)

Bounded Quicksort (BQS) resolves this critical design flaw through two established software engineering principles:

1. **Tail Recursion Elimination (*Tail Call Elimination*):** The algorithm never performs two recursive calls within the same activation frame. It invokes recursion **strictly on the partition of size at most half the current subproblem span** ($\leq \frac{j-i}{2}$). The remaining partition (the larger half) is handled iteratively by updating indices i or j inside a enclosing ‘while‘ loop.

2. **Insertion Sort Cutoff for Small Subproblems:** When the subarray length falls below a predetermined threshold m_0 (typically $m_0 \in [16, 32]$), recursion ceases and sorting is completed using insertion sort.

Algorithm 4: Bounded Quicksort (BQS)

Input: Array S, start index i, end index j, cutoff threshold m_0
Output: Sorted subarray $S[i \dots j]$

```
while (j − i) ≥ m_0 do
    r ← RANDOM_POSITION(i, ..., j)
    SWAP(S[r], S[i])   // Random pivot moved to the beginning
    p' ← PARTITION(S, i, j)            // p' is the final pivot
     position
    if p' ≤ ⌊(i + j)/2⌋ then
        // LEFT partition is smaller or equal:  recurse
        BQS(S, i, p' − 1, m_0)
        // Tail recursion elimination on the RIGHT:
           iterate
        i ← p' + 1
    else
        // RIGHT partition is smaller or equal:  recurse
        BQS(S, p' + 1, j, m_0)
        // Tail recursion elimination on the LEFT: iterate
        j ← p' − 1
INSERTION_SORT(S, i, j)     // Process small base cases of
   size < m_0
```

Figure 5.4 depicts the operational control flow of BQS, highlighting the branch between bounded recursion and iterative continuation.

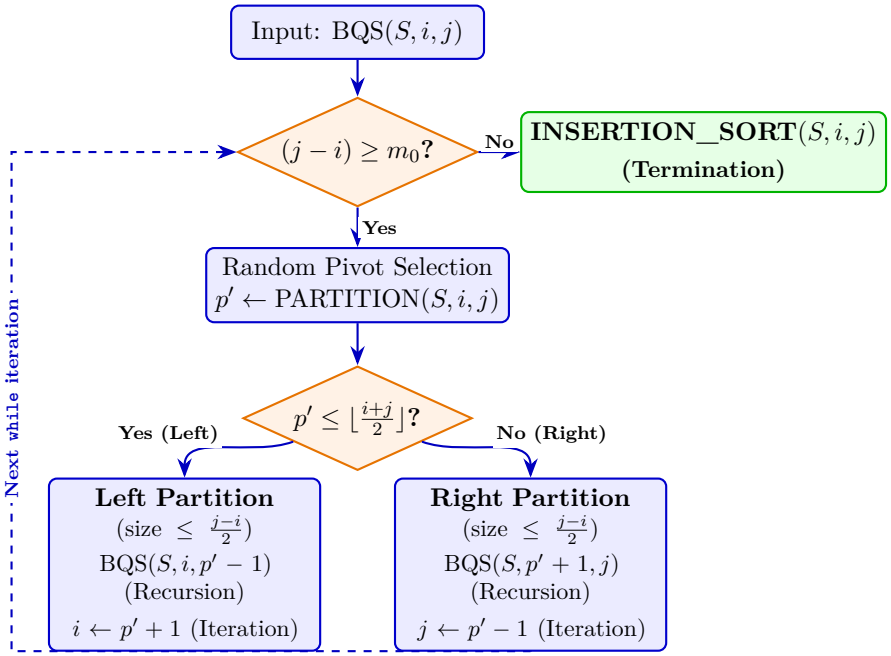


Figure 5.4: Control flow of Bounded Quicksort (BQS). Recursive invocations are strictly bounded to the smaller partition ($\leq (j - i)/2$), while the larger half is evaluated iteratively inside the `while` loop. Partitions smaller than m_0 are sorted directly via Insertion Sort.

5.4.3 Execution Example and Call Stack Management in Bounded Quicksort

To gain an in-depth understanding of the stack bounding mechanism and tail call elimination, consider the execution of algorithm 4 on an 8-element array with cutoff threshold $m_0 = 3$:

$$S = [7, 2, 8, 4, 1, 6, 3, 5]. \tag{5.38}$$

In an unfavorable scenario where partitions are repeatedly skewed, standard Quicksort pushes a new call frame for each subproblem, driving stack depth linearly to $\Theta(n)$. Conversely, Bounded Quicksort processes the larger subproblem purely iteratively within the existing stack frame (by updating boundary index i or j), invoking recursion exclusively on the subarray whose size does not exceed half the current range ($\leq L/2$).

In Figure 5.5, the execution sequence and the structural comparison between the call stack of standard Quicksort and Bounded Quicksort are illustrated.

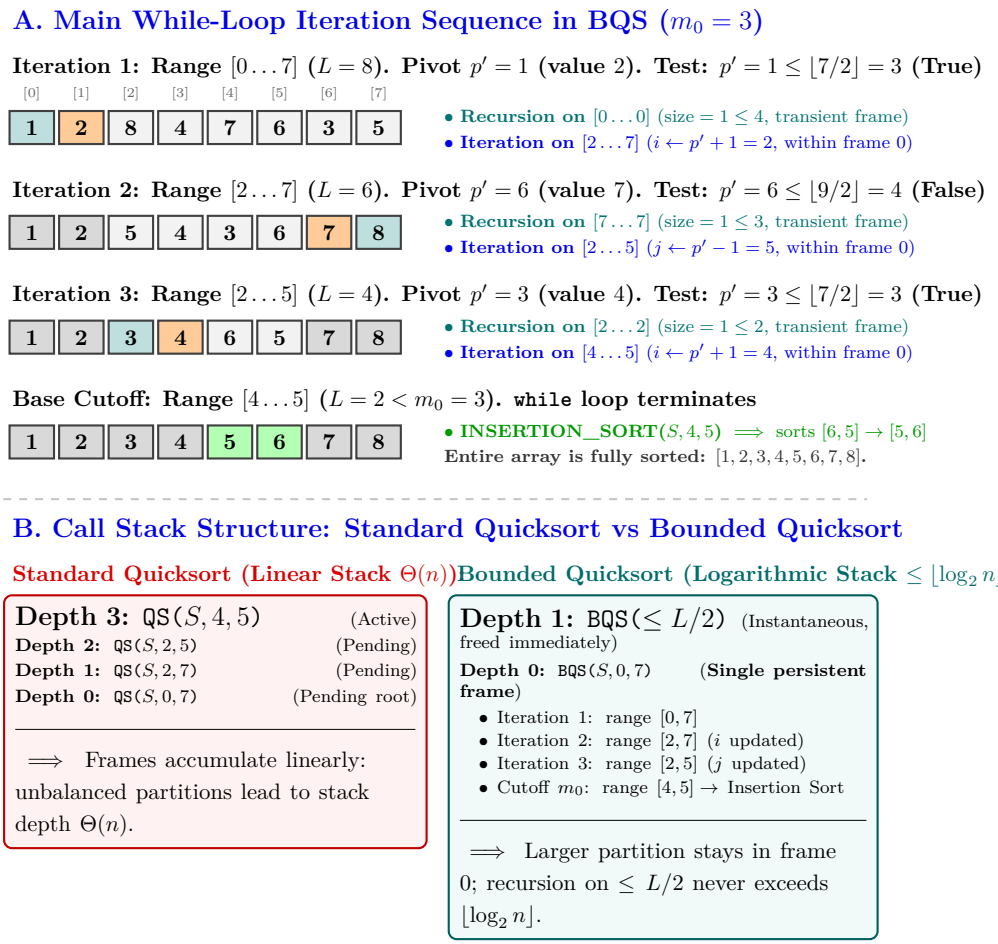


Figure 5.5: Execution trace of Bounded Quicksort (BQS) with threshold $m_0 = 3$ and call stack memory comparison. (**A**) At each iteration, the larger partition is processed within the same stack frame by updating i or j, while only the halved partition ($\leq L/2$) triggers a short-lived recursive call. (**B**) Standard Quicksort accumulates call frames linearly; BQS strictly guarantees logarithmic stack depth.

In Table 5.3, the step-by-step iteration sequence is detailed, highlighting the smaller-partition test, the tail recursion elimination update, and the instantaneous maximum stack depth.

Step	$[i, j]$	L	Pivot Test	Action (Recursion & Iteration)	Array State	Stack
Init	$[0, 7]$	8	–	Root frame BQS$(S, 0, 7)$	$[7, 2, 8, 4, 1, 6, 3, 5]$	0
1	$[0, 7]$	8	$p' = 1 \leq 3$	**Rec:** $[0, 0]$ (size ≤ 4, instant return) **Iter:** $i \leftarrow 2$ on $[2, 7]$ (same frame)	$[1 \mid \mathbf{2} \mid 8, 4, 7, 6, 3, 5]$	1
2	$[2, 7]$	6	$p' = 6 > 4$	**Rec:** $[7, 7]$ (size ≤ 3, instant return) **Iter:** $j \leftarrow 5$ on $[2, 5]$ (same frame)	$[1, 2 \mid 5, 4, 3, 6 \mid \mathbf{7} \mid 8]$	1
3	$[2, 5]$	4	$p' = 3 \leq 3$	**Rec:** $[2, 2]$ (size ≤ 2, instant return) **Iter:** $i \leftarrow 4$ on $[4, 5]$ (same frame)	$[1, 2 \mid 3 \mid \mathbf{4} \mid 6, 5 \mid 7 \mid 8]$	1
Cutoff	$[4, 5]$	2	$L < m_0$	**Exit:** Exit `while` loop **Sort:** Insertion Sort on $[4, 5]$	$[1, 2, 3, 4, \mathbf{5}, \mathbf{6}, 7, 8]$	0

Table 5.3: Step-by-step trace of Bounded Quicksort (BQS) with cutoff threshold $m_0 = 3$. In each iteration, the subproblem of size $\leq L/2$ is resolved with a bounded recursive call, while the larger subproblem is processed iteratively within the current frame, bounding stack depth to ≤ 1 in this trace ($\leq \lfloor \log_2 n \rfloor$ in general).

5.4.4 Rigorous Analysis of the $\Theta(\log_2 n)$ Extra Space Bound

The cornerstone of BQS lies in its absolute mathematical guarantee on memory consumption:

Theorem 5.4.1 (Auxiliary Space Guarantee of Bounded Quicksort)**.** *For any input of size n, Bounded Quicksort consumes an auxiliary call stack space strictly bounded by:*

$$\textit{Auxiliary Space} = \Theta(\log_2 n). \tag{5.39}$$

The maximum stack depth never exceeds $\lfloor \log_2 n \rfloor$ under any worst-case scenario.

Proof. Let $L = j - i + 1$ denote the length of the active subarray at the entry of a BQS call. Upon partitioning around index p', the algorithm inspects the lengths of the two subranges:

- The left subrange has length $L_{\text{left}} = (p' - 1) - i + 1 = p' - i$.
- The right subrange has length $L_{\text{right}} = j - (p' + 1) + 1 = j - p'$.

The condition $p' \leq \lfloor \frac{i+j}{2} \rfloor$ implies:

$$p' - i \leq \frac{j - i}{2} < \frac{L}{2}. \tag{5.40}$$

Consequently, when this condition holds, recursion is invoked exclusively on the left partition, whose size is strictly bounded by:

$$L_{\text{recursive}} \leq \left\lfloor \frac{L}{2} \right\rfloor. \tag{5.41}$$

Conversely, when $p' > \lfloor \frac{i+j}{2} \rfloor$, the right subrange satisfies $j - p' < \frac{L}{2}$, and the recursive call is invoked on the right partition, fulfilling the exact same inequality.

The remaining subproblem has size $L_{\text{iterative}} \geq \frac{L}{2}$, but **does not spawn any recursive call**: the algorithm simply mutates $i \leftarrow p' + 1$ or $j \leftarrow p' - 1$ and loops within the existing activation frame without allocating any new stack space.

Therefore, the input size passed to any subsequent stack frame decreases by at least a factor of 2 relative to its parent frame:

$$L^{(k)} \leq \frac{n}{2^k}, \tag{5.42}$$

where k denotes the current stack depth. The recursion can continue at most while $L^{(k)} \geq m_0 \geq 1$:

$$\frac{n}{2^k} \geq 1 \implies 2^k \leq n \implies k \leq \lfloor \log_2 n \rfloor. \tag{5.43}$$

Since each activation record requires constant memory $\mathcal{O}(1)$, the total auxiliary stack space is strictly bounded by:

$$\text{Extra Space} = \mathcal{O}(1) \cdot \lfloor \log_2 n \rfloor = \Theta(\log_2 n). \tag{5.44}$$

□

5.4.5 Hybrid Optimization with Insertion Sort Cutoff

Introducing a threshold $m_0 \in [16, 32]$ for handing off small subproblems to *Insertion Sort* directly leverages hardware caching characteristics and execution profile:

- **Elimination of Recursive Overhead:** When subarray size drops below m_0, the overhead of function call frames, parameter passing, and partition pointer management surpasses the computational cost of direct comparisons.
- **Low Constant Factors of Insertion Sort:** *Insertion Sort* has exceptionally low constant factors: it requires no extra heap or stack space, runs tight inner loops, and easily undergoes compiler vectorization and loop unrolling.

- **Optimal L1 Cache Locality:** A block of $16 \div 32$ items requires approximately $128 \div 256$ bytes (for 64-bit keys), fitting completely within one or two 64-byte L1 cache lines. All comparisons and shifts occur without generating any cache misses.
- **Nearly Sorted Behavior:** Because Quicksort's macro-partitions have already placed elements near their final positions, small subarrays are almost sorted. On nearly sorted inputs, Insertion Sort operates in linear time $\mathcal{O}(n)$ rather than quadratic time.

In conclusion, Bounded Quicksort represents an ideal synthesis of mathematical rigor and algorithm engineering, guaranteeing optimal expected time complexity $\mathcal{O}(n \log_2 n)$, absolute immunity to *Stack Overflow* via $\Theta(\log_2 n)$ auxiliary space, and exceptional empirical throughput on modern computer architectures.

Chapter 6

Order Statistics, Multi-Pivot Trade-offs, and Average-Case Analysis of Quicksort

Lecture 6

Selection of the k-th Item, Multi-Pivot Trade-offs, and Quicksort Average-Case

Date: September 29, 2026

In this chapter, we investigate the fundamental problem of **order statistic selection** (k-th smallest element), comparing global sorting, incremental window insertion, and bounded priority queue (*Bounded Max-Heap*) approaches. Subsequently, our analysis extends to the **multi-pivot dilemma**, uncovering the architectural mechanisms explaining why deploying multiple pivots does not unconditionally yield runtime improvements due to element-classification overhead, CPU register pressure, cache line thrashing, and branch misprediction penalties on modern processors. We then examine the **3-way partitioning scheme** (*Dutch National Flag*) and its structural role within recursive flows: connecting with the operational mechanics established in Lecture 5, we analyze its dimensional invariants, its direct application to the *Quickselect* algorithm for linear selection with early exit, and its impact on subproblem size reduction. Finally, we provide a complete **step-by-step formal algebraic proof of the expected running time of Quicksort** ($\Theta(n \log_2 n)$), solving the full recurrence via the integrating-factor technique and harmonic numbers.

6.1 The Selection Problem (*Selection of the k-th Item*)

The selection problem is a cornerstone primitive in theoretical computer science and algorithm engineering. Unlike sorting, where the primary objective is to globally rearrange an entire sequence in non-decreasing order, selection focuses on retrieving a single element defined by its positional rank without paying the unnecessary cost of sorting the entire collection.

6.1.1 Formal Problem Formulation

Definition 6.1.1 (k-th Order Statistic)**.** Let $S = [s_1, s_2, \ldots, s_n]$ be an unsorted array or sequence of n elements drawn from a totally ordered universe $(\mathcal{U}, \leq)$, and let $k \in \{1, 2, \ldots, n\}$ be a given target index. The **k-th order statistic** of S, commonly denoted by $S_{(k)}$ or $\text{Select}(S, k)$, is the element that would occupy index k if S were sorted in non-decreasing order:

$$S_{\text{sorted}} = [s_{(1)}, s_{(2)}, \ldots, s_{(n)}] \quad \text{with } s_{(1)} \leq s_{(2)} \leq \cdots \leq s_{(n)}. \tag{6.1}$$

Consequently, $\text{Select}(S, k) = S_{\text{sorted}}[k]$.

Example 6.1.1 (Instructive Array Instance)**.** Consider an unsorted array S of size $n = 6$:

Index (1-based):	1	2	3	4	5	6
$S =$	2	7	5	3	1	10
$S_{\text{sorted}} =$	1	2	3	5	7	10

From the order statistics perspective:

- For $k = 1$: the target is the absolute **minimum** of the collection:

$$S_{(1)} = S_{\text{sorted}}[1] = 1. \tag{6.2}$$

- For $k = n = 6$: the target is the absolute **maximum** of the collection:

$$S_{(6)} = S_{\text{sorted}}[6] = 10. \tag{6.3}$$

- For $k = 3$: the third order statistic corresponds to $S_{(3)} = 3$.
- For $k = \lceil n/2 \rceil = 3$ or $k = \lfloor n/2 \rfloor + 1 = 4$: we obtain the **median** of the collection (3 or 5).

6.1.2 Boundary Cases: Minimum and Maximum Selection

For the boundary cases $k = 1$ and $k = n$, locating the target element requires no advanced data structures. A single linear scan storing the current candidate in a temporary register suffices:

- Initialize a temporary register min $\leftarrow S[1]$.
- Sequentially scan the remaining $n-1$ entries from $i = 2$ to n, performing the comparison $S[i] <$ min: if true, update min $\leftarrow S[i]$.
- The total number of comparisons performed is exactly $n - 1$.

Property 6.1.1 (Asymptotic Optimality for $k = 1$ and $k = n$)**.** *In the comparison-based model of computation, determining the minimum (or maximum) of an unsorted collection of n elements requires at least $n - 1$ comparisons in the worst case. Hence, the linear scan algorithm achieves optimal complexity:*

$$T(n) = \Theta(n), \quad \textit{with auxiliary space } \mathcal{O}(1). \tag{6.4}$$

Adversary Argument. Each comparison between two distinct elements a and b can eliminate at most one candidate from being the minimum (the larger element). Starting with n potential candidates and requiring exactly one survivor upon termination, at least $n-1$ candidates must be eliminated. Since each comparison removes at most one candidate, at least $n - 1$ comparisons are necessary. □

Property 6.1.2 (Simultaneous Minimum and Maximum Search)**.** *When both the minimum ($k = 1$) and maximum ($k = n$) must be located simultaneously, running two independent scans takes $2(n - 1)$ comparisons. By processing elements in pairs, the comparison count can be reduced:*

1. *Compare adjacent elements $S[2i]$ and $S[2i + 1]$ with each other (1 comparison).*
2. *Compare the smaller element against the current minimum (1 comparison).*
3. *Compare the larger element against the current maximum (1 comparison).*

This pair-based strategy takes $\lceil 3n/2 \rceil - 2$ comparisons, achieving a 25% reduction over the naive approach.

6.1.3 General Strategies for Arbitrary k

When the target index $k \in \{1, \ldots, n\}$ is arbitrary, single-register linear scanning is insufficient, as multiple potential candidates must be tracked concurrently. Three primary strategies are distinguished:

Table 6.1: Analytical comparison of classical selection strategies for arbitrary k.

Approach	Algorithm / Structure	Time Complexity	Architectural Mechanism and Description
1. Global Sorting	Merge Sort / Heap Sort	$\mathcal{O}(n \log_2 n)$	Fully sorts array S; returns $S_{\text{sorted}}[k]$ in $\mathcal{O}(1)$ time. Excessive overhead if $k \ll n$.
2. Partial Sorting	Insertion Sort Window	$\mathcal{O}(k \cdot n)$ (*Worst-Case*)	Maintains a sorted window of size k of the smallest elements. Each new element shifts up to k slots.
3. Bounded Priority Queue	Max-Heap of size k	$\mathcal{O}(n \log_2 k)$	Maintains a Max-Heap containing the k smallest elements seen so far. Root holds the current k-th candidate; filtered in $\mathcal{O}(\log_2 k)$.

Strategy 1: Global Sorting (*Full Sorting*)

The most straightforward approach consists in sorting the entire sequence S using an optimal comparison-based sorting algorithm, such as Merge Sort or Heap Sort, requiring $\mathcal{O}(n \log_2 n)$ operations in the worst case. Once the array is sorted, the k-th order statistic is retrieved in $\mathcal{O}(1)$ time at index k.

- **Advantages:** Trivial to implement; highly efficient when answering multiple subsequent selection queries across distinct indices k (*multi-query workload*).
- **Disadvantages:** Performs excessive superfluous work. If only a single rank is required (e.g. median or 95th percentile), ordering unrelated

partitions incurs preventable computational overhead.

Strategy 2: Partial Incremental Sorting (*Insertion Window*)

When k is small relative to n ($k \ll n$), full sorting is unnecessary. An auxiliary sorted buffer W of capacity k can be maintained, holding the k smallest elements encountered during the scan:

1. Place the first k elements into W and sort them in $\mathcal{O}(k \log_2 k)$ or $\mathcal{O}(k^2)$ time.
2. For each remaining element $x \in S[k+1 \dots n]$:
   - Compare x against the maximum element of W, located at $W[k]$.
   - If $x \geq W[k]$, x cannot belong to the set of k smallest elements; discard x in $\mathcal{O}(1)$ time.
   - If $x < W[k]$, x belongs to the new set of k smallest elements. Evict $W[k]$ and insert x into W via backward shifting as in *Insertion Sort*, costing $\mathcal{O}(k)$ operations.

In the worst case (e.g. a strictly decreasing sequence), every element triggers an insertion at the front of W, yielding:

$$T(n) = \mathcal{O}(k^2) + \sum_{i=k+1}^{n} \mathcal{O}(k) = \mathcal{O}(k \cdot n). \tag{6.5}$$

When k is a small constant (e.g. $k \in \{2, 3, 5\}$), this runs in strictly linear time $\mathcal{O}(n)$. However, when $k = \Theta(n)$ (such as the median $k = n/2$), the running time degenerates to $\mathcal{O}(n^2)$, performing worse than global sorting.

Strategy 3: Bounded Priority Queue (*Bounded Max-Heap*)

Strategy 3 overcomes the shifting overhead of partial sorting by replacing the sequential buffer with a **Max-Heap of fixed size** k.

Property 6.1.3 (Bounded Max-Heap Invariant)**.** *At any point during execution, having processed a prefix of size $m \geq k$ of the input array, the Max-Heap stores exactly the k smallest elements drawn from the subset $\{S[1], \dots, S[m]\}$. The root of the heap (top()) holds the maximum among these k elements, representing the current candidate for the k-th order statistic.*

The decision flow is illustrated in Figure 6.1.

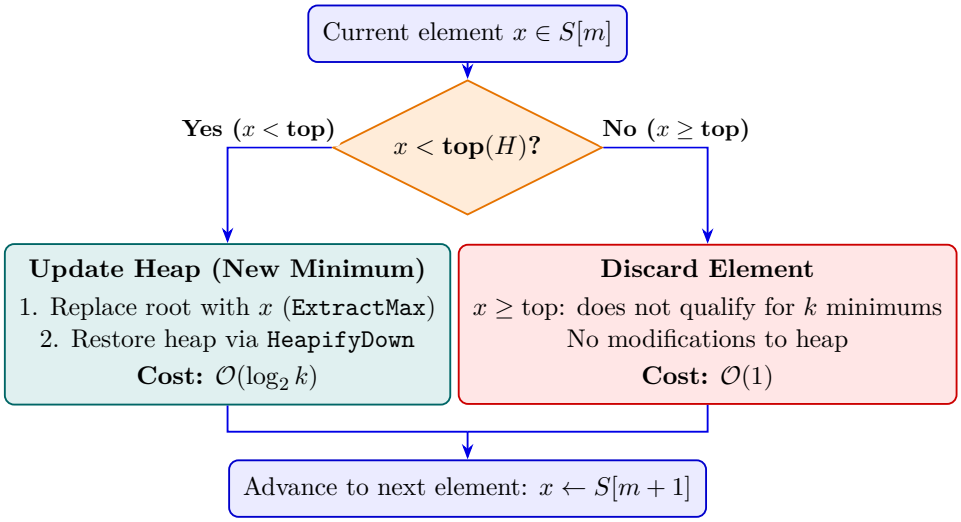


Figure 6.1: Decision flow using a bounded Max-Heap of size k. Elements that do not improve the current k minimums are discarded in constant time $\mathcal{O}(1)$.

The formal algorithm is given in algorithm 5.

Algorithm 5: Bounded Max-Heap Selection (*BoundedHeapSelect*)

Input: Array S of size n, target index $k \in \{1, \ldots, n\}$
Output: The value of the k-th order statistic $S_{(k)}$

```
Initialize an empty Max-Heap H of capacity k
for i ← 1 to k do
    H.Insert(S[i])                          // Initial build in O(k) time
for i ← k + 1 to n do
    if S[i] < H.Top() then
        H.ExtractMax()
        H.Insert(S[i])                      // Evict and restore in O(log_2 k) time
return H.Top()                              // The root holds exactly S_(k)
```

Example 6.1.2 (Step-by-Step Bounded Max-Heap Trace)**.** We trace algorithm 5 on array $S = [2, 7, 5, 3, 1, 10]$ with $k = 3$ to locate the 3rd smallest element (expected value: 3).

Table 6.2: Detailed trace of heap state for each processed element ($k = 3$).

Step	$S[i]$	Comparison with Root	Action Taken	Heap State H (Top)
1	2	Initial heap build	Insert into H	$[2]$
2	7	Initial heap build	Insert into H	$[7, 2]$
3	5	Initial heap build	Insert into H	$[7, 2, 5]$ (Top = 7)
4	3	$3 < 7$ (Yes)	Evict 7, Insert 3	$[5, 2, 3]$ (Top = 5)
5	1	$1 < 5$ (Yes)	Evict 5, Insert 1	$[3, 2, 1]$ (Top = 3)
6	10	$10 < 3$ (No)	Discard 10 in $\mathcal{O}(1)$	$[3, 2, 1]$ (Top = 3)

Upon scanning all n entries, the root of the Max-Heap contains 3, precisely matching $S_{(3)} = S_{\text{sorted}}[3]$.

Property 6.1.4 (Time and Space Complexity Analysis)**.** *Initial heap construction over the first k elements takes linear time $\mathcal{O}(k)$ via Floyd's algorithm (`BuildMaxHeap`). For each of the remaining $n - k$ elements, a constant-time comparison $\mathcal{O}(1)$ is performed; if an update is required, restoration takes $\mathcal{O}(\log_2 k)$ time. The total worst-case time is:*

$$T(n) = \mathcal{O}(k) + (n - k) \cdot \mathcal{O}(\log_2 k) = \mathcal{O}(n \log_2 k). \tag{6.6}$$

*Auxiliary space is bounded by $\Theta(k)$ memory cells, making this approach particularly well suited for **data streaming** environments where $n \gg k$ and the entire dataset cannot be held in memory.*

6.2 The Multi-Pivot Dilemma (*Why Not Many Pivots?*)

In distribution-based partitioning paradigms and divide-and-conquer sorting algorithms, choosing the number of partitioning splitters (*pivots*) is a fundamental design decision. Intuitively, dividing a problem of size n into many subproblems decreases the height of the recursion tree.

6.2.1 The Illusion of Multi-Way Trees

Suppose we generalize standard Quicksort by selecting $p \geq 1$ distinct, sorted pivots:

$$p_1 < p_2 < \cdots < p_p. \tag{6.7}$$

These splitters induce $p+1$ disjoint partitions:

$$(-\infty, p_1), \quad [p_1, p_2), \quad [p_2, p_3), \quad \ldots, \quad [p_p, +\infty). \tag{6.8}$$

Assuming balanced partitions of size approximately $n/(p+1)$, the recursion depth reduces to:

$$h(n) \approx \log_{p+1} n = \frac{\ln n}{\ln(p+1)}. \tag{6.9}$$

If p could be increased arbitrarily (e.g. $p = \sqrt{n}$ or $p = \log_2 n$), recursion depth would collapse. However, practical algorithm engineering raises the fundamental trade-off:

"Why not many pivots?" $\implies$ Increase cost of sorting and partitioning.

6.2.2 Pivot Selection Cost

Achieving balanced partitions across $p+1$ intervals requires selecting pivots that accurately represent the statistical distribution of the input:

- If p pivots are sampled uniformly at random, they must be pre-sorted among themselves, taking $\mathcal{O}(p \log_2 p)$ comparisons.
- If exact percentile guarantees are demanded, deterministic selection routines (such as the Median-of-Medians algorithm by Blum et al. [6]) must be invoked, whose overhead constants dwarf the benefits of multi-way partitioning.

6.2.3 Internal Element Classification Overhead

The primary limiting factor lies in the number of comparisons required to route each element into its designated bucket among the $p+1$ intervals:

1. **Sequential Linear Classification:** Comparing an element sequentially against $p_1, p_2, \ldots, p_p$ takes up to p comparisons per element. The total number of comparisons across the entire tree is:

$$C_{\text{tot}}(n) \approx n \cdot p \cdot \frac{\ln n}{\ln(p+1)} = n \ln n \cdot \left(\frac{p}{\ln(p+1)} \right). \tag{6.10}$$

   The coefficient $f(p) = \frac{p}{\ln(p+1)}$ is **strictly increasing** for all integers $p \geq 1$:

- For $p = 1$: $f(1) = \frac{1}{\ln 2} \approx 1.442$.
- For $p = 2$: $f(2) = \frac{2}{\ln 3} \approx 1.820$ (+26% comparisons relative to $p = 1$).
- For $p = 7$: $f(7) = \frac{7}{\ln 8} \approx 3.366$ (+133% comparisons).

As p grows, the total comparison count increases significantly, negating the shallower recursion tree.

2. **Binary Search Classification:** Performing binary search over the p sorted pivots requires $\lceil \log_2(p+1) \rceil$ comparisons per element. Multiplying by tree depth:

$$C_{\text{tot}}(n) \approx n \cdot \log_2(p+1) \cdot \frac{\log_2 n}{\log_2(p+1)} = n \log_2 n. \tag{6.11}$$

Asymptotically, the total number of comparisons **remains completely unchanged**, matching single-pivot Quicksort.

6.2.4 Microarchitectural and Hardware Bottlenecks

On modern microarchitectures, algorithmic execution time is governed not merely by comparison counts, but by cache line locality, register availability, and branch prediction efficiency:

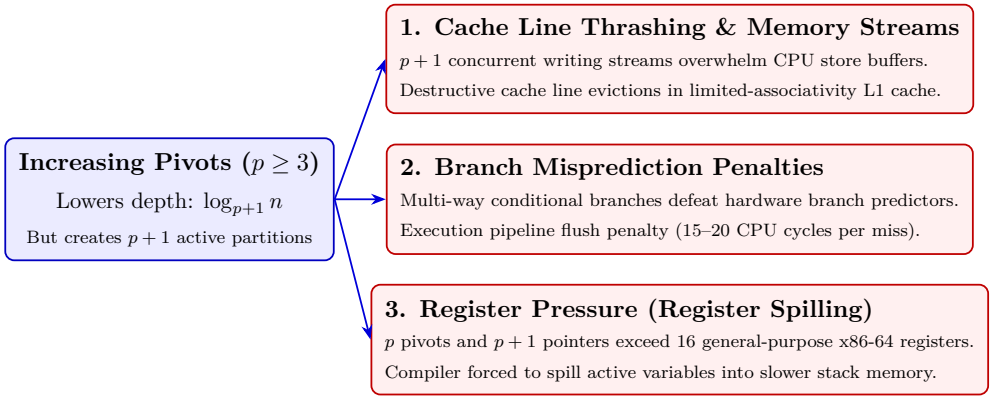


Figure 6.2: Microarchitectural degradation mechanisms caused by increasing the pivot count p.

- **Write Streams and Cache Lines:** A single pivot uses two continuous writing streams (from left and right ends), exploiting write-combining buffers efficiently. With p pivots, $p+1$ concurrent writing streams are

required. Because L1 caches feature limited associativity (commonly 8-way), having $p + 1 \geq 4$ active streams induces frequent cache line eviction conflicts.

- **Branch Prediction:** Binary branches ($x < p$) are predicted with high accuracy by modern dynamic branch predictors. In contrast, multi-way branch constructs produce unpredictable jump paths: each branch misprediction triggers a 15–20 cycle pipeline flush penalty.

- **Register Pressure:** x86-64 processors provide 16 general-purpose registers. Storing p pivot values, $p + 1$ boundary indices, base pointers, and iteration counters exceeds available registers when $p \geq 3$, forcing the compiler to spill variables to the stack and inflating memory bus traffic.

Property 6.2.1 (Why Dual-Pivot ($p = 2$) Excels in Practice)**.** *Yaroslavskiy's Dual-Pivot Quicksort [5] (adopted in the standard libraries of Java and C++) strikes an optimal trade-off: although it performs approximately* 5% *more comparisons on average (*$1.9n \ln n$ *vs* $2n \ln n$*), it* ***reduces memory write and swap operations by*** 20%*, offering sequential memory access patterns that align cleanly with CPU cache hierarchies. For* $p \geq 3$*, hardware penalties overwhelm any theoretical gains.*

6.3 Quicksort: 3-Way Partitioning and Recursive Implications

As established in Lecture 5 (section 5.3), a classical vulnerability of standard 2-way Quicksort is its performance collapse on inputs rich in duplicate keys: when all elements are identical, conventional schemes (such as Lomuto's partition) isolate only a single element per step, generating a degenerate linear recursion tree of depth $\Theta(n)$ and driving runtime to $\Theta(n^2)$.

The structural remedy is Dijkstra's 3-way partitioning scheme (*Dutch National Flag*) [3]. While Lecture 5 (subsection 5.3.2) provided a comprehensive breakdown of the three-pointer operational mechanics (lt, i, gt), the full in-place pseudocode, and a detailed step-by-step trace table (see Table 5.1 on page 52), this section investigates the **systemic role of 3-way partitioning within recursive execution**: its dimensional invariants, central partition pruning, direct integration into linear selection (*Quickselect*), and its connection to the average-case recurrence.

6.3.1 Region Topology and Dimensional Invariants

Given a sub-array $S[\text{lo} \dots \text{hi}]$ of size n and an initial pivot p, 3-way partitioning rearranges entries into three contiguous compartments demarcated by the final boundary pointers lt and gt, as illustrated in Figure 6.3.

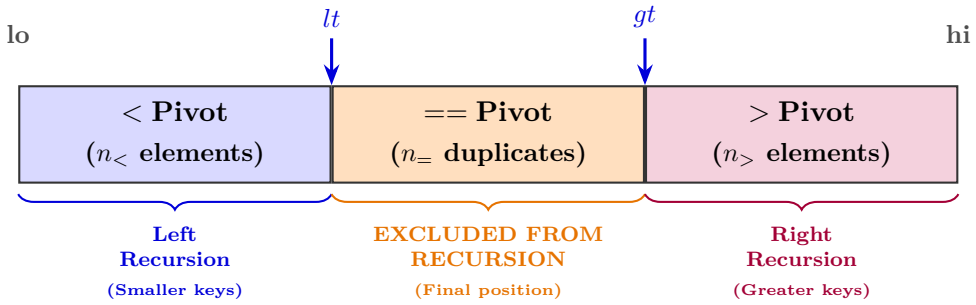


Figure 6.3: Topology of the three regions in 3-way partitioning (*Dutch National Flag*) and boundary pointers lt and gt.

Property 6.3.1 (Dimensional Invariants of 3-Way Partitioning)**.** *The sizes of the three regions satisfy the following invariants:*

1. $n_= \geq 1$*: the central partition always holds at least the chosen pivot.*
2. $0 \leq n_< \leq n-1$*: number of elements strictly less than the pivot.*
3. $0 \leq n_> \leq n-1$*: number of elements strictly greater than the pivot.*
4. $n_< + n_= + n_> = n$*: total array length conservation.*

Theorem 6.3.1 (Exclusion of the Central Region from Recursive Flow)**.** *Because all elements in the == Pivot band share the exact same key and are placed strictly between the smaller and larger elements, they already reside in their* ***final sorted positions****.*

In subsequent recursive Quicksort calls, the central region is ***entirely omitted****. Recursion proceeds exclusively on:*

- *The left sub-array of size $n_<$ ($S[lo \dots lt-1]$).*
- *The right sub-array of size $n_>$ ($S[gt+1 \dots hi]$).*

The aggregate size of remaining subproblems is:

$$n_< + n_> = n - n_= \leq n-1. \tag{6.12}$$

On an array where all elements are identical, $n_= = n$ and $n_< = n_> = 0$: the algorithm completes a single linear scan and terminates in optimal time $\mathcal{O}(n)$.

6.3.2 Application to Quickselect: Order Statistic Selection

The 3-way partitioning scheme integrates seamlessly with the order statistic selection problem explored in section 6.1.

In standard 2-way **Quickselect**, repeated duplicate keys can lead to highly unbalanced partitions, degenerating execution time to $\Theta(n^2)$. With 3-way partitioning, finding the k-th order statistic benefits from an immediate **early-exit property**:

- **If $k < lt$:** the target element is strictly smaller than the pivot; the algorithm recurses **exclusively into the left partition** $S[\text{lo} \dots lt-1]$ seeking rank k.
- **If $lt \leq k \leq gt$:** the target index k falls precisely within the central band of duplicate pivot elements. Since every element in this sub-array equals p, we conclude immediately that:

$$S_{(k)} = p. \tag{6.13}$$

  The algorithm **terminates immediately in $\mathcal{O}(1)$ time** returning p, without executing any further recursive calls.
- **If $k > gt$:** the target element is strictly greater than the pivot; the algorithm recurses **exclusively into the right partition** $S[gt+1 \dots \text{hi}]$ seeking rank k.

This early-exit behavior makes 3-way Quickselect extraordinarily robust: on datasets with low key cardinality (such as discretized sensor data or categorical values), the target is located in very few iterations, preserving optimal linear expected time $\mathcal{O}(n)$ and achieving $\mathcal{O}(n)$ worst-case time when all keys are identical.

6.3.3 Impact on the Average-Case Recurrence Relation

Pruning the central partition provides the mathematical justification for the recurrence relation evaluated in the upcoming section 6.4.

When input keys are distinct, the central partition contains only the pivot ($n_= = 1$), and the sum of recursive subproblem sizes is:

$$n_< + n_> = n - 1. \tag{6.14}$$

If the pivot splits the array into two partitions of sizes $i-1$ and $n-i$ (with $i \in \{1,\ldots,n\}$ uniformly distributed), the expected time $T(n)$ follows Hoare's classical recurrence:

$$T(n) = \frac{1}{n}\sum_{i=1}^{n}\Big(T(i-1)+T(n-i)\Big)+\Theta(n). \tag{6.15}$$

Whenever duplicate keys occur ($n_= > 1$), the combined size of the recursive subproblems shrinks to $n_< + n_> = n - n_= < n-1$. Each additional duplicate removes work from subsequent recursive stages at zero extra cost, accelerating asymptotic convergence. Hence, the worst-case configuration for randomized Quicksort corresponds to all keys being distinct ($n_= = 1$), whose complete mathematical resolution is developed formally in the following section.

6.4 Formal Mathematical Proof of Quicksort Average-Case ($\Theta(n \log_2 n)$)

The average-case analysis of Quicksort is an analytical landmark. We rigorously determine the expected execution time $T(n)$ under the assumption that all $n!$ input permutations are equally likely or, equivalently, that pivots are selected uniformly at random.

6.4.1 Recurrence Derivation

Let S be an array of n distinct elements. Partitioning requires comparing each element against the pivot, taking $c \cdot n$ elementary operations for some constant $c > 0$.

The chosen pivot is equally likely with probability $\frac{1}{n}$ to land at rank $i \in \{0,1,\ldots,n-1\}$ in the sorted array, splitting the problem into two subproblems:

- Left sub-array of size i.
- Right sub-array of size $n-1-i$.

The expected execution time satisfies:

$$T(n) = c\cdot n + \frac{1}{n}\sum_{i=0}^{n-1}\left(T(i)+T(n-1-i)\right). \tag{6.16}$$

Lemma 6.4.1 (Summation Symmetry)**.** *The two summation components contain the identical sequence of terms summed in reverse order:*

$$\sum_{i=0}^{n-1} T(n-1-i) = T(n-1) + T(n-2) + \cdots + T(0) = \sum_{i=0}^{n-1} T(i). \tag{6.17}$$

Substituting the lemma yields the classic Quicksort recurrence:

$$T(n) = c \cdot n + \frac{2}{n} \sum_{i=0}^{n-1} T(i) \quad \text{for } n \geq 2. \tag{6.18}$$

Base cases are $T(0) = 0$ and $T(1) = 0$.

6.4.2 Step-by-Step Algebraic Solution

We solve this full-history recurrence using consecutive difference subtraction and the integrating-factor method.

Step 1: Eliminating the Denominator via Multiplication by n

Multiply both sides of Equation 6.18 by n:

$$n\,T(n) = c\,n^2 + 2 \sum_{i=0}^{n-1} T(i). \tag{6.19}$$

Step 2: Expressing the Recurrence for Instance Size $n-1$

Substitute n with $n-1$ (valid for $n \geq 3$):

$$(n-1)\,T(n-1) = c\,(n-1)^2 + 2 \sum_{i=0}^{n-2} T(i). \tag{6.20}$$

Step 3: Member-by-Member Subtraction (Eq. 6.19 − Eq. 6.20)

Subtracting Equation 6.20 from Equation 6.19 cancels all historical summation terms except the last term $i = n-1$:

$$2 \sum_{i=0}^{n-1} T(i) - 2 \sum_{i=0}^{n-2} T(i) = 2\,T(n-1). \tag{6.21}$$

This yields:

$$\begin{aligned} n\,T(n) - (n-1)\,T(n-1) &= c\left[n^2 - (n-1)^2\right] + 2\,T(n-1) && (6.22)\\ &= c\left[n^2 - (n^2 - 2n + 1)\right] + 2\,T(n-1) && (6.23)\\ &= c\,(2n-1) + 2\,T(n-1). && (6.24) \end{aligned}$$

Reorganizing terms in $T(n-1)$:

$$n\,T(n) = (n-1)\,T(n-1) + 2\,T(n-1) + c\,(2n-1) \tag{6.25}$$
$$n\,T(n) = (n+1)\,T(n-1) + c\,(2n-1). \tag{6.26}$$

Since $2n-1 < 2n$ for all $n \geq 1$, we obtain the upper bound:

$$n\,T(n) \leq (n+1)\,T(n-1) + 2c\,n. \tag{6.27}$$

Step 4: Integrating Factor (Summation Factor)

To bring Equation 6.27 into telescopic form, divide both sides by $n(n+1)$:

$$\frac{n\,T(n)}{n(n+1)} \leq \frac{(n+1)\,T(n-1)}{n(n+1)} + \frac{2c\,n}{n(n+1)}. \tag{6.28}$$

Canceling common factors:

$$\frac{T(n)}{n+1} \leq \frac{T(n-1)}{n} + \frac{2c}{n+1}. \tag{6.29}$$

Let $A_n \coloneqq \frac{T(n)}{n+1}$. The relation reduces to a standard additive recurrence:

$$A_n \leq A_{n-1} + \frac{2c}{n+1}. \tag{6.30}$$

Step 5: Telescopic Expansion and Harmonic Numbers

Unrolling backwards to the base case A_1:

$$A_n \leq A_{n-1} + \frac{2c}{n+1} \tag{6.31}$$
$$\leq A_{n-2} + \frac{2c}{n} + \frac{2c}{n+1} \tag{6.32}$$
$$\leq A_1 + 2c \sum_{k=2}^{n} \frac{1}{k+1}. \tag{6.33}$$

Changing the index of summation $j = k+1$:

$$\sum_{k=2}^{n} \frac{1}{k+1} = \sum_{j=3}^{n+1} \frac{1}{j} \leq \sum_{j=1}^{n} \frac{1}{j}. \tag{6.34}$$

This partial sum defines the **n-th Harmonic Number** H_n:

$$H_n \coloneqq \sum_{j=1}^{n} \frac{1}{j} = 1 + \frac{1}{2} + \frac{1}{3} + \cdots + \frac{1}{n}. \tag{6.35}$$

Using the integral bounding technique (illustrated in Figure 6.4):

$$H_n = \ln n + \gamma + \mathcal{O}\left(\frac{1}{n}\right) = \ln n + \mathcal{O}(1), \tag{6.36}$$

where $\gamma \approx 0.577215$ is the Euler-Mascheroni constant.

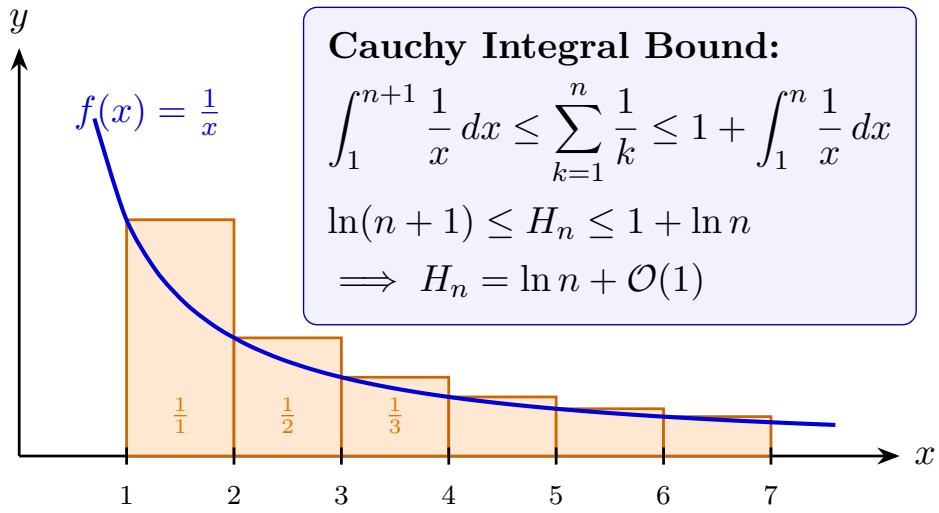


Figure 6.4: Integral approximation of the harmonic series $H_n = \sum_{k=1}^{n} \frac{1}{k} \approx \ln n$. The rectangle areas upper-bound the integral of the hyperbola $f(x) = \frac{1}{x}$.

Substituting into the bound for A_n:

$$A_n = \frac{T(n)}{n+1} \le 2c \ln n + \mathcal{O}(1). \tag{6.37}$$

Multiplying back by $(n+1)$:

$$T(n) \le 2c\,(n+1) \ln n + \mathcal{O}(n) = 2c\,n \ln n + \mathcal{O}(n). \tag{6.38}$$

6.4.3 Base-2 Conversion and Information-Theoretic Meaning

To express runtime in standard binary comparison units, convert the natural logarithm $\ln n$ to $\log_2 n$:

$$\ln n = \frac{\log_2 n}{\log_2 e} = (\ln 2) \cdot \log_2 n \approx 0.693147 \cdot \log_2 n. \tag{6.39}$$

Substituting into the expected cost:

$$T_{\text{avg}}(n) \leq 2c \cdot (\ln 2) \cdot n \log_2 n + \mathcal{O}(n) \tag{6.40}$$
$$\approx (2 \cdot 0.693147) \cdot c\, n \log_2 n \tag{6.41}$$
$$\approx 1.386294 \cdot c\, n \log_2 n = \Theta(n \log_2 n). \tag{6.42}$$

Theorem 6.4.1 (Information-Theoretic Lower Bound Comparison)**.** *The information-theoretic lower bound for comparison sorting dictates that any algorithm requires at least:*

$$\log_2(n!) \approx n \log_2 n - n \log_2 e \approx n \log_2 n - 1.4427\, n \quad \textit{comparisons.} \tag{6.43}$$

For randomized Quicksort, the expected comparison count is:

$$2 \ln 2 \cdot n \log_2 n \approx 1.386\, n \log_2 n. \tag{6.44}$$

Property 6.4.1 (Engineering Efficiency of Quicksort)**.** *On average, Quicksort performs merely* $\approx$ **38.6**% ***more comparisons*** *than the theoretical optimum. Because the partitioning inner loop is a compact, cache-friendly, in-place scan without dynamic allocation overhead, its constant factor* c *is so small that Quicksort routinely outperforms algorithms performing fewer theoretical comparisons (such as Merge Sort) in main memory.*

6.5 Comparative Synthesis of Results

To consolidate the comparative study of all analyzed methods, Table 6.3 summarizes asymptotic complexities, auxiliary space demands, and key architectural characteristics.

Table 6.3: Comparative synthesis of selection and sorting algorithms.

Algorithm	Best Case	Average Case	Worst Case	Space	Architectural Characteristic
Min / Max Search	$\Theta(n)$	$\Theta(n)$	$\Theta(n)$	$\mathcal{O}(1)$	Single sequential scan; optimal $n-1$ comparisons.
Bounded Max-Heap (k)	$\Theta(n)$	$\Theta(n \log_2 k)$	$\Theta(n \log_2 k)$	$\Theta(k)$	Ideal for data streams and $k \ll n$; discards non-qualifying keys in $\mathcal{O}(1)$.
Quickselect (Hoare)	$\Theta(n)$	$\Theta(n)$	$\Theta(n^2)$	$\mathcal{O}(\log_2 n)$	Single-branch divide-and-conquer; expected linear time for any k.
Standard Quicksort	$\Theta(n \log_2 n)$	$\Theta(n \log_2 n)$	$\Theta(n^2)$	$\Theta(n)$	Collapses to $\Theta(n^2)$ on duplicate keys or unbalanced pivots.
3-Way Quicksort	$\Theta(n)$	$\Theta(n \log_2 n)$	$\Theta(n^2)$	$\Theta(\log_2 n)$	Linear $\Theta(n)$ when all keys are equal; excludes central partition.
Dual-Pivot Quicksort	$\Theta(n \log_2 n)$	$\Theta(n \log_2 n)$	$\Theta(n^2)$	$\Theta(\log_2 n)$	20% fewer writes for +5% comparisons; excellent cache reuse.
Bounded Quicksort (BQS)	$\Theta(n \log_2 n)$	$\Theta(n \log_2 n)$	$\Theta(n^2)$	$\Theta(\log_2 n)$	Tail recursion elimination on larger partition; strictly logarithmic stack.

6.5.1 Practical Selection Guidelines

From the engineering analysis developed throughout the lecture:

1. **Extremal Selection ($k = 1$ or $k = n$):** Use a single-register linear scan; for simultaneous min/max, use pairwise comparison to minimize branch mispredictions.

2. **Small Window Selection ($k \ll n$ in Data Streams):** Deploy a **Bounded Max-Heap** of capacity k. Processes unbounded streams without loading all n items into RAM, filtering outliers in $\mathcal{O}(1)$.

3. **General Selection on Static Arrays ($k = \Theta(n)$):** Deploy **Quickselect** with 3-way partitioning, ensuring expected linear time $\mathcal{O}(n)$ without sorting unnecessary segments.

4. **High-Performance In-Memory Sorting:** Use advanced Quicksort variants combining 3-way partitioning (or Dual-Pivot), tail recursion elimination, and a small cutoff threshold ($M \approx 16$–32) delegated to *Insertion Sort* for maximum L1 cache utilization.

Chapter 7

External Distribution Sort, Over-Sampling, and Random Sampling Foundations

Lecture 7

External Distribution Sort, Over-Sampling Technique, and Foundations of Random Sampling

Date: September 30, 2026

7.1 The External Sorting Problem and Multi-Way Partitioning

7.1.1 The Computational I/O Model (Secondary Memory)

In systems processing massive datasets, the size of the dataset to be sorted S far exceeds the capacity of random access internal memory (RAM). We adopt the standard two-level secondary memory model introduced by Aggarwal and Vitter [2]:

- N (or n): total number of records in sequence $S = \langle S[1], \ldots, S[n] \rangle$;
- M: internal memory (RAM) capacity, measured in number of records ($M \ll n$);
- B: disk transfer block size, measured in number of records transferred per single Input/Output (I/O) operation.

The number of blocks that can reside simultaneously in internal memory is $\frac{M}{B}$.

To sort the dataset S with optimal I/O complexity, a standard binary recursive partition algorithm (such as classical Quicksort) is insufficient, as it yields a recursion tree of depth $\Theta(\log_2(n/B))$, which is asymptotically too deep and incurs excessive random disk seeks.

Instead, algorithms rely on multi-way partitioning (**Multi-Way Partitioning** or **External Distribution Sort**), dividing the dataset recursively into k disjoint partitions (called *buckets*), where the partition fan-out is calibrated to available memory:

$$k = \mathcal{O}\left(\frac{M}{B}\right). \tag{7.1}$$

7.1.2 Formal Definition of Buckets and Pivots

Given the unsorted array $S[1 \dots n]$, the goal is to select an ordered set of $k-1$ sentinel elements, called **pivots**:

$$s_1 \leq s_2 \leq \cdots \leq s_{k-1}. \tag{7.2}$$

The notation s emphasizes that these elements originate from a sampling procedure (*sampled*).

For analytical convenience and to ensure boundary completeness, we introduce two conceptual sentinel pivots:

$$s_0 \coloneqq -\infty, \qquad s_k \coloneqq +\infty. \tag{7.3}$$

Definition 7.1.1 (Buckets B_i)**.** The k buckets $B_1, B_2, \dots, B_k$ are defined as the disjoint partitions formed by all elements of S that fall within the respective half-open intervals determined by adjacent pivots:

$$B_i \coloneqq \Big\{ S[j] \in S \;\mid\; s_{i-1} < S[j] \leq s_i \Big\}, \qquad \forall i \in \{1, 2, \dots, k\}. \tag{7.4}$$

The resulting partition topology is illustrated in Figure 7.1.

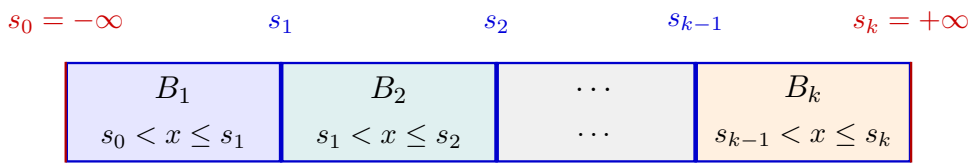


Figure 7.1: Partitioning of dataset S into k buckets using $k-1$ sorted pivots.

7.1.3 Optimal Balancing Condition

For the sorting algorithm to maintain optimal I/O complexity, bucket sizes must not deviate excessively from the ideal average size $\frac{n}{k}$.

Property 7.1.1 (Formal Balancing Objective)**.** *All buckets must have asymptotically uniform cardinality:*

$$|B_i| = \Theta\left(\frac{n}{k}\right), \qquad \forall i \in \{1, 2, \ldots, k\}. \tag{7.5}$$

The architectural significance of this balance is two-fold:

1. **Asymptotic Optimality of Recursion Depth:** When all buckets remain balanced across recursion levels, tree depth is bounded by:

$$\mathcal{O}\left(\log_k \frac{n}{B}\right) = \mathcal{O}\left(\frac{\log(n/B)}{\log(M/B)}\right). \tag{7.6}$$

   Distributing elements among k buckets at each level requires a single linear scan taking $\mathcal{O}\left(\frac{n}{B}\right)$ I/Os (allocating one output write buffer for each of the $k = \mathcal{O}(M/B)$ buckets). Total I/O complexity matches Aggarwal and Vitter's lower bound:

$$\mathcal{O}\left(\frac{n}{B} \log_{M/B} \frac{n}{B}\right) \text{ I/Os.} \tag{7.7}$$

2. **Prevention of Memory Overflow:** No bucket exceeds internal memory capacity during the base case where individual buckets are loaded into RAM and sorted locally.

7.2 The Over-Sampling Technique

7.2.1 Shortcomings of Naive Sampling

Directly extracting only $k - 1$ random pivots from S yields high statistical variance in the spacing between successive pivot keys. Consequently, with high probability, dense clusters and empty gaps emerge, creating pathological buckets with:

$$|B_i| \gg \frac{n}{k}. \tag{7.8}$$

This severe skew destroys the optimal I/O bounds of external distribution sort.

To sharply suppress this variance and mathematically guarantee that no bucket exceeds a target threshold, we employ **Over-Sampling**: we draw substantially more than k samples, sort them in RAM, and select the final pivots at equidistant regular strides.

7.2.2 Formalization of Over-Sampling Parameters

We define an over-sampling parameter a:

$$a = \Theta(\log_2 k). \tag{7.9}$$

Specifically, to align analytical constants with exponential concentration bounds, we calibrate:

$$a + 1 \coloneqq 12 \ln k \qquad (\text{for } k \geq 2). \tag{7.10}$$

The total size of the random sample pool $A \subset S$ is:

$$|A| = (a + 1)k - 1. \tag{7.11}$$

7.2.3 Pivot Selection Algorithm via Over-Sampling

The selection workflow proceeds through three sequential phases:

1. **Random Extraction:** Draw $(a+1)k-1$ elements uniformly at random from S, forming sample set A.

2. **Sample Sorting:** Sort set A in internal memory (incurring 0 extra disk I/Os since $|A| \leq M$):

$$A = [A[1], A[2], \ldots, A[|A|]], \qquad \text{where } A[1] \leq A[2] \leq \cdots \leq A[|A|]. \tag{7.12}$$

3. **Equidistant Pivot Selection:** For each index $i \in \{1, 2, \ldots, k-1\}$, pivot s_i is chosen as the element occupying the exact multiple position of $(a + 1)$ in the sorted array A:

$$s_i \coloneqq A\Big[(a+1) \cdot i\Big], \qquad \forall i \in \{1, 2, \ldots, k-1\}. \tag{7.13}$$

Structural Layout of the Sorted Sample Vector A

The sorted array A is partitioned into k logical segments containing a items each, separated by sentinel pivots:

- Exactly a samples of A lie strictly between s_{i-1} and s_i.
- Including the right boundary s_i, exactly $a+1$ samples of A fall into the half-open interval $(s_{i-1}, s_i]$.

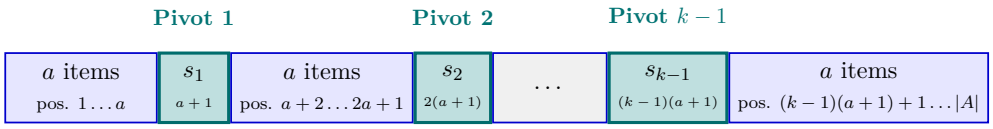


Figure 7.2: Structural segmentation of sorted sample array A into k blocks of size a spaced by pivots.

Algorithm 6: Pivot Selection via Over-Sampling

Input: Unsorted dataset $S[1 \dots n]$, fan-out $k \geq 2$, parameter $a + 1 = 12 \ln k$

Output: Sorted pivot vector $s = \langle s_1, s_2, \dots, s_{k-1} \rangle$

```
m ← (a + 1)k − 1
A ← RandomSample(S, m)        // Draw m items uniformly at random
Sort(A)        // Internal sort in RAM: A[1] <= ...  <= A[m]
for i ← 1 to k − 1 do
    s_i ← A[(a + 1) · i]      // Equidistant pivots at stride a+1
return s
```

7.3 Probabilistic Failure Analysis and Chernoff Bound

7.3.1 Formal Definition of Failure Event

Our algorithmic goal is to prevent any bucket from growing excessively large. We establish the critical failure threshold at twice the average bucket capacity $\frac{2n}{k}$.

Definition 7.3.1 (Failure Event (Failure))**.** Failure occurs if at least one of the k buckets exceeds the critical threshold:

$$\text{Failure} \iff \exists i \in \{1, 2, \dots, k\} \text{ such that } |B_i| \geq \frac{2n}{k}. \tag{7.14}$$

We aim to prove that the failure probability is bounded by a constant strictly less than 1:

$$\mathbb{P}(\text{Failure}) \leq \frac{1}{2}. \tag{7.15}$$

Property 7.3.1 (Practical Relevance of $\mathbb{P}(\text{Failure}) \leq \frac{1}{2}$)**.** *A failure probability $\leq \frac{1}{2}$ is ideal in the analysis of **Las Vegas** randomized algorithms.*

If the algorithm verifies bucket balance a posteriori (costing a linear $\frac{n}{B}$ I/O scan) and restarts sampling upon failure, the expected number of iterations is geometrically distributed with mean:

$$\mathbb{E}[\mathit{trials}] = \frac{1}{1 - \mathbb{P}(\mathit{Failure})} \leq \frac{1}{1 - 1/2} = 2. \tag{7.16}$$

Furthermore, after r independent attempts, the probability of remaining unbalanced decays exponentially to $\leq \left(\frac{1}{2}\right)^r$, ensuring extreme operational robustness.

7.3.2 Ideal Sample Space S_{sort} and Fixed-Block Partitioning

To analyze this stochastic behavior, consider the ideal fully sorted sequence of all n elements in S, denoted S_{sort}:

$$S_{\text{sort}} = [x_1, x_2, \ldots, x_n], \qquad \text{with } x_1 \leq x_2 \leq \cdots \leq x_n. \tag{7.17}$$

Because every bucket B_i gathers all elements in the interval $(s_{i-1}, s_i]$, the contents of each bucket B_i occupy a contiguous subarray inside S_{sort}.

We conceptually partition the sorted sequence S_{sort} into fixed, disjoint blocks t_j of uniform size $\frac{2n}{k}$. The total count of such fixed blocks is:

$$\text{Number of blocks } t_j = \frac{n}{\frac{2n}{k}} = \frac{k}{2}. \tag{7.18}$$

We denote these $\frac{k}{2}$ blocks as $t_1, t_2, \ldots, t_{k/2}$.

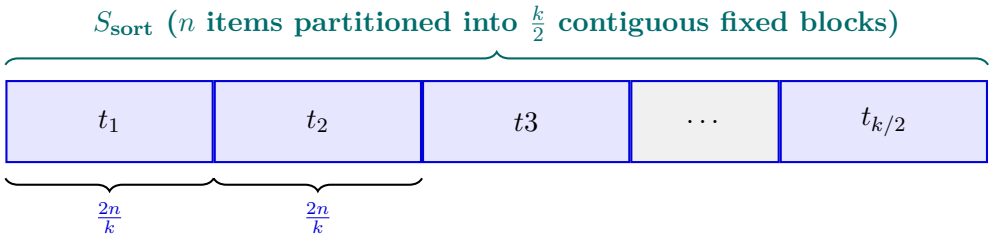


Figure 7.3: Theoretical partitioning of S_{sort} into $\frac{k}{2}$ fixed blocks t_j of size $\frac{2n}{k}$.

7.3.3 Logical Implication of the Failure Event

Examine the geometric configuration when failure occurs for an arbitrary bucket B_i, meaning $|B_i| \geq \frac{2n}{k}$:

1. Since B_i forms a contiguous interval in S_{sort} of length at least $\frac{2n}{k}$, it must completely contain (or span across) at least one fixed block t_j:
$$\exists j \in \left\{1, \ldots, \frac{k}{2}\right\} \text{ such that } t_j \subseteq B_i. \tag{7.19}$$
2. The delimiting pivots s_{i-1} and s_i sit strictly outside block t_j (to the left and right, respectively).
3. By construction of over-sampling, the entire bucket B_i contains at most a interior samples, plus boundary pivot s_i. Thus, the total count of samples from A falling within B_i is at most $a + 1$.
4. Since $t_j \subseteq B_i$ and pivot s_i lies outside t_j, fixed block t_j can contain strictly fewer than $a + 1$ samples of A:
$$|t_j \cap A| < a + 1. \tag{7.20}$$

Thus, the logical implication holds:
$$X \implies Y, \tag{7.21}$$
where:
$$X : \exists B_i \text{ such that } |B_i| \geq \frac{2n}{k}, \tag{7.22}$$
$$Y : \exists t_j \text{ such that } t_j \text{ contains } < a + 1 \text{ samples of } A. \tag{7.23}$$

By fundamental probability laws:
$$X \implies Y \implies \mathbb{P}(X) \leq \mathbb{P}(Y). \tag{7.24}$$
We upper-bound the failure probability by analyzing the likelihood that at least one fixed block receives too few samples:
$$\mathbb{P}(\text{Failure}) \leq \mathbb{P}\left(\exists j \in \left\{1, \ldots, \frac{k}{2}\right\} : |t_j \cap A| < a + 1\right). \tag{7.25}$$

7.3.4 Union Bound

Applying Boole's inequality (Union Bound) across all $\frac{k}{2}$ fixed blocks:
$$\mathbb{P}\left(\bigcup_{j=1}^{k/2} \Big\{|t_j \cap A| < a + 1\Big\}\right) \leq \sum_{j=1}^{k/2} \mathbb{P}\Big(|t_j \cap A| < a + 1\Big). \tag{7.26}$$

Because each block t_j has identical length $|t_j| = \frac{2n}{k}$, the probability of an individual sample landing in t_j is identical for every j. Selecting an arbitrary reference block t^*:
$$\mathbb{P}(\text{Failure}) \leq \frac{k}{2} \cdot \mathbb{P}\Big(t^* \text{ contains } < a + 1 \text{ samples of } A\Big). \tag{7.27}$$

7.3.5 Expectation Computation

Fix block t^*. Let X be the random variable counting how many sample items from A land in t^*. Define indicator variables for each drawn sample $r \in \{1, \ldots, |A|\}$:

$$X_r \coloneqq \begin{cases} 1 & \text{if sample } r \text{ of } A \text{ belongs to } t^*, \\ 0 & \text{otherwise.} \end{cases} \tag{7.28}$$

Total random count is:

$$X = \sum_{r=1}^{|A|} X_r. \tag{7.29}$$

The marginal probability that an arbitrary uniform sample from S falls into t^* is:

$$\mathbb{P}(\text{sample } \in t^*) = \frac{|t^*|}{n} = \frac{\frac{2n}{k}}{n} = \frac{2}{k}. \tag{7.30}$$

By linearity of expectation:

$$\mathbb{E}[X] = \mathbb{E}\left[\sum_{r=1}^{|A|} X_r\right] = \sum_{r=1}^{|A|} \mathbb{E}[X_r] = \sum_{r=1}^{|A|} \mathbb{P}(X_r = 1) = |A| \cdot \frac{2}{k}. \tag{7.31}$$

Substituting total sample size $|A| = (a+1)k - 1$:

$$\begin{aligned} \mathbb{E}[X] &= \Big[(a+1)k - 1\Big] \cdot \frac{2}{k} \\ &= 2(a+1) - \frac{2}{k}. \end{aligned} \tag{7.32}$$

Since $k \geq 2$, the subtracted fraction satisfies:

$$\frac{2}{k} \leq \frac{2}{2} = 1. \tag{7.33}$$

We lower-bound the expectation:

$$\mathbb{E}[X] \geq 2(a+1) - 1. \tag{7.34}$$

Since $a \geq 1$ (as $a + 1 = 12 \ln k \geq 12 \ln 2 \approx 8.31$ for all $k \geq 2$):

$$\begin{aligned} 2(a+1) - 1 &= \frac{3}{2}(a+1) + \underbrace{\frac{1}{2}(a+1) - 1}_{\geq 0} \\ &\geq \frac{3}{2}(a+1). \end{aligned} \tag{7.35}$$

We establish the key inequality relating expectation to threshold:

$$\mathbb{E}[X] \geq \frac{3}{2}(a+1) \iff a + 1 \leq \frac{2}{3}\mathbb{E}[X]. \tag{7.36}$$

7.3.6 Chernoff Bound Application (Lower Tail)

The Chernoff lower-tail concentration bound establishes that for a sum $X = \sum X_r$ of independent random variables bounded in $[0, 1]$, for any relative deviation $\delta \in (0, 1)$:

$$\mathbb{P}\Big(X < (1-\delta)\mathbb{E}[X]\Big) \leq e^{-\frac{\delta^2}{2}\mathbb{E}[X]}. \tag{7.37}$$

Property 7.3.2 (Intuitive Interpretation of Chernoff Bound)**.** *Random variable X concentrates sharply around its mean $\mathbb{E}[X]$. The probability of falling far below the expectation decays exponentially with respect to deviation square δ^2.*

We match $\mathbb{P}(X < a + 1)$ with Chernoff's format. From inequality (7.36), $a + 1 \leq \frac{2}{3}\mathbb{E}[X]$. Setting relative deviation:

$$1 - \delta = \frac{2}{3} \implies \delta = 1 - \frac{2}{3} = \frac{1}{3}. \tag{7.38}$$

Consequently:

$$\mathbb{P}(X < a+1) \leq \mathbb{P}\left(X < \frac{2}{3}\mathbb{E}[X]\right) = \mathbb{P}\left(X < \left(1 - \frac{1}{3}\right)\mathbb{E}[X]\right). \tag{7.39}$$

Applying Chernoff bound (7.37) with $\delta = \frac{1}{3}$:

$$\mathbb{P}\left(X < \left(1 - \frac{1}{3}\right)\mathbb{E}[X]\right) \leq e^{-\frac{(1/3)^2}{2}\mathbb{E}[X]} = e^{-\frac{1/9}{2}\mathbb{E}[X]} = e^{-\frac{1}{18}\mathbb{E}[X]}. \tag{7.40}$$

Substituting our expectation lower bound $\mathbb{E}[X] \geq \frac{3}{2}(a + 1)$:

$$\begin{aligned} e^{-\frac{1}{18}\mathbb{E}[X]} &\leq e^{-\frac{1}{18}\cdot\frac{3}{2}(a+1)} \\ &= e^{-\frac{3}{36}(a+1)} \\ &= e^{-\frac{1}{12}(a+1)}. \end{aligned} \tag{7.41}$$

7.3.7 Exact Calibration of a and Conclusion

Substituting our chosen parameter $a + 1 = 12 \ln k$:

$$e^{-\frac{1}{12}(a+1)} = e^{-\frac{1}{12}\cdot(12\ln k)} = e^{-\ln k} = \frac{1}{k}. \tag{7.42}$$

Therefore, for any individual reference block t^*:

$$\mathbb{P}(t^* \text{ contains } < a + 1 \text{ samples}) \leq \frac{1}{k}. \tag{7.43}$$

Multiplying by factor $\frac{k}{2}$ from the Union Bound (7.27):

$$\mathbb{P}(\text{Failure}) \leq \frac{k}{2} \cdot \mathbb{P}\Big(t^* \text{ contains } < a+1 \text{ samples}\Big) \leq \frac{k}{2} \cdot \frac{1}{k} = \frac{1}{2}. \tag{7.44}$$

Theorem 7.3.1 (Over-Sampling Balance Theorem)**.** *With success probability at least* $1 - \frac{1}{2} = \frac{1}{2}$*, over-sampling yields a partition where all buckets* B_i *strictly satisfy size bound* $|B_i| < \frac{2n}{k}$*:*

$$\mathbb{P}\left(\forall i \in \{1, \ldots, k\} \colon |B_i| < \frac{2n}{k}\right) \geq \frac{1}{2}. \tag{7.45}$$

This establishes uniform balancing and closes the I/O efficiency analysis of External Distribution Sort.

7.4 Introduction to Random Sampling and Non-Streaming Limits

Following the analysis of distribution sort, we must study the extraction primitive itself: how to formally draw m random elements from a universe S of size n?

7.4.1 Formal Problem Definition

Given an indexed array $S[1 \ldots n]$ containing n elements, extract subset $D \subseteq S$ of target cardinality m satisfying:

- **Marginal Uniformity:** Each item $i \in S$ shares identical inclusion probability:
$$\mathbb{P}(i \in D) = \frac{m}{n}. \tag{7.46}$$
- **Exact Cardinality Without Replacement:** The sample contains exactly m distinct items:
$$|D| = m. \tag{7.47}$$

We assume access to an integer random generator primitive:

$$\text{Rand}(a, b) \to r \in [a, b], \qquad \text{where } r \text{ is drawn uniformly.} \tag{7.48}$$

7.4.2 Algorithm 1: Rejection Sampling (Naive Insertion Sampling)

The simplest scheme draws random candidate indices, inserting new ones into a dynamic set while discarding duplicates until reaching size m.

Algorithm 7: Rejection Sampling via Dynamic Set Insertion

Input: Index universe $[1 \dots n]$, target sample size $m \leq n$
Output: Sorted set D containing m distinct indices

```
D ← ∅
while |D| < m do
    p ← Rand(1, n)
    if p ∉ D then
        D ← D ∪ {p}
Sort(D)
return D
```

Collision Analysis and Failure Probability

At each step of the `while` loop, the probability that candidate p is already in D (causing a collision and a wasted attempt) is:

$$\mathbb{P}(\text{collision}) = \mathbb{P}(p \in D) = \frac{|D|}{n}. \tag{7.49}$$

Across execution, $|D|$ rises from 0 to $m - 1$. The collision probability at any step is bounded by:

$$\mathbb{P}(\text{failure}) < \frac{m}{n}. \tag{7.50}$$

Property 7.4.1 (Operating Regimes)**.** *Efficiency depends heavily on the ratio of m to n:*

- ***Sparse Regime ($m \leq \frac{n}{2}$):*** *Step failure probability satisfies $\mathbb{P}(\text{failure}) < \frac{1}{2}$. Expected draws before success is less than 2, ensuring fast execution. Smaller m/n ratios yield even fewer collisions.*
- ***Dense Regime ($m \to n$):*** *As m nears n, drawing an unselected element becomes increasingly improbable, degenerating into the classical* ***Coupon Collector's Problem*** *and inflating expected iterations to:*

$$\Theta(n \ln m). \tag{7.51}$$

Impact of Underlying Data Structures

As highlighted in lecture:

- **Set Management for D:** Implementing D as a **hash table** enables membership checks $p \notin D$ and insertions in expected $\mathcal{O}(1)$ time. An unsorted array would require linear search taking $\mathcal{O}(|D|) = \mathcal{O}(m)$ per draw.
- **Final Sorting:** Operation Sort(D) consumes $\mathcal{O}(m \log m)$ CPU time to return sorted positions.

7.4.3 Handling Variable-Length Records (Strings)

How do we sample when elements of S are arbitrary variable-length records or text strings?

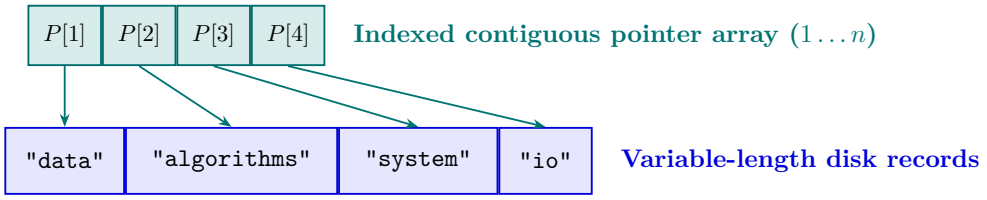


Figure 7.4: Mapping variable-length string records using a contiguous indexed pointer array.

In a raw sequential file of variable-length strings, scalar index $p \in [1, n]$ from Rand$(1, n)$ has no immediate $\mathcal{O}(1)$ byte offset.

To solve this, we construct a contiguous **array of pointers** $P[1 \dots n]$, where pointer $P[i]$ references the exact byte offset of the i-th record. Random draws operate directly over pointer indices in $\mathcal{O}(1)$ time, granting instant access to the targeted record.

7.4.4 Why Algorithm 1 Fails in the Streaming Model

The lecture concludes with a vital structural observation bridging into data streams:

Property 7.4.2 (Professor's Critical Observation: *"Our algorithm is not Streaming!"*)**.** *Algorithm 7 suffers from fundamental constraints preventing its application to data streams:*

1. ***Arbitrary Random Access:*** *It requires querying any arbitrary index $p \in [1, n]$ on demand at any iteration.*

2. ***Violation of Single-Pass Constraint:*** *Stream elements arrive in strict sequential order; rewinding or random hopping is strictly prohibited.*

3. ***Requirement of Known*** n***:*** *Primitive* $\text{Rand}(1, n)$ *requires knowing total cardinality* n *upfront before processing begins.*

This limitation motivates the development of **Random Sampling Algorithms on Data Streams** (examined in chapter 8), where cardinality n may be unknown and admission decisions must be finalized *on-the-fly* as elements arrive.

Chapter 8

Random Sampling Algorithms on Data Streams

Lecture 8

Random Sampling Algorithms on Data Streams: Streaming Model, Uniform Sampling, Algorithm S, Priority Sampling, and Reservoir Sampling

Date: October 6, 2026

In previous lectures, we thoroughly examined data processing under the *External Memory Model*, focusing on massive datasets stored on secondary storage through multiway external merge sort and distribution sort algorithms. In those scenarios, although access costs were dominated by block I/O transfers, multiple scans (*multi-pass*) over the stored dataset were still permissible.

In this chapter, we explore a fundamental paradigm shift: the **Data Streaming Model**. In this setting, data arrives as a continuous, massive, or potentially infinite sequence at high arrival rates, subject to stringent working memory and per-item processing constraints. Among the fundamental stream processing tasks, **Uniform Random Sampling** plays a critical role in constructing statistical synopses, network monitoring, high-volume log debugging, and online aggregate estimation.

We investigate both principal categories of sampling without replacement: when stream length n is known in advance (analyzing single-item selection and Knuth's **Algorithm S**, supported by a complete combinatorial derivation) and when cardinality n is unknown ahead of time (**Priority Sampling** with Min-Heaps and the Waterman–Knuth **Reservoir Sampling** framework). We prove the correctness of Reservoir Sampling by mathematical induction, present comprehensive step-by-step traces, and examine engineering optimizations (including Vitter's **Algorithm Z**).

8.1 Foundations of the Streaming Model and Random Sampling

8.1.1 The Data Streaming Computation Model

In traditional computational paradigms (both in RAM and secondary storage), datasets remain available for repeated scans. Under the *Data Streaming Model*, conversely, the algorithm can neither control data ordering nor request data retransmission.

The streaming scenario is formally characterized by four architectural pillars:

1. **Data Stream S:** a time-ordered sequence of discrete items, potentially infinite or of massive size n ($n \to \infty$ or $n \gg M$):
$$S = \langle i_1, i_2, \ldots, i_j, \ldots, i_n \rangle. \tag{8.1}$$

2. **Strict Working Memory Constraint ($M \ll n$):** the fast internal working memory M available to the algorithm satisfies $M \ll n$. Typically, storage is strictly bounded by:
$$M = \mathcal{O}(m) \quad \text{or} \quad M = \mathcal{O}(\text{polylog}(n)) \text{ words of memory}, \tag{8.2}$$
where m denotes the desired sample size. Consequently, storing the entire stream or any substantial fraction thereof is impossible.

3. **Single-Pass Sequential Access:** elements arrive one by one in chronological order. Once an item i_j has passed, if the algorithm chooses not to store it in working memory, it is **lost forever** and can never be re-examined.

4. **Strictly Bounded Per-Item Processing Time:** to avoid input buffer exhaustion and packet drops on high-speed feeds (such as backbone routers or real-time trading systems), the processing time allocated to each arriving item i_j must remain strictly constant:
$$T(i_j) = \mathcal{O}(1) \quad \text{or at most} \quad T(i_j) = \mathcal{O}(\log m). \tag{8.3}$$

8.1.2 Formal Definition of the Uniform Random Sampling Problem

Given a continuous stream S, our objective is to select and maintain a subset of m items that represents a **Simple Random Sample without Replacement** (SRSWOR).

Formally, at any step n (or upon stream completion if n is finite), two probabilistic uniformity criteria must hold:

Definition 8.1.1 (Marginal Uniformity)**.** Every single element $i_j \in S$ (for $1 \leq j \leq n$) must possess an identical marginal inclusion probability:

$$\mathbb{P}(i_j \in \text{Sample}) = \frac{m}{n}. \tag{8.4}$$

Definition 8.1.2 (Joint Uniformity)**.** Every possible subset $S^* \subseteq S$ of cardinality m must have an identical joint extraction probability:

$$\mathbb{P}(\text{Sample} = S^*) = \frac{1}{\binom{n}{m}}, \qquad \forall S^* \subseteq S \text{ such that } |S^*| = m. \tag{8.5}$$

Depending on prior knowledge of stream cardinality n, sampling algorithms are classified into two primary regimes:

- **Case A (n known a priori):** the total stream length n is known before sampling begins (e.g., sequentially reading an unindexed n-line log file from tape or storage).
- **Case B (n unknown a priori):** cardinality n is unavailable upfront, and the stream may terminate arbitrarily or continue indefinitely. The algorithm must guarantee that whenever the stream terminates at any step $t \geq m$, internal memory stores a valid uniform random sample drawn from the first t items.

8.2 Sampling with Known Stream Length (n Known)

8.2.1 Single-Item Selection ($m = 1$)

Consider the elementary case of extracting exactly one element ($m = 1$) from a stream of known length n, ensuring each item has uniform probability $\frac{1}{n}$ of being selected while deciding online whether to accept or reject the current item i_j.

Algorithm Formulation

The algorithm sequentially inspects elements for $j = 1, 2, \ldots, n$:

1. At step j, draw a random variable $p \sim \text{Uniform}(0, 1)$, meaning $p \in [0, 1)$.

2. Evaluate the selection condition:

$$\text{If } p \leq \frac{1}{n-j+1}, \quad \text{select } i_j \text{ and terminate the algorithm.} \tag{8.6}$$

3. If $p > \frac{1}{n-j+1}$, item i_j is permanently discarded, and processing proceeds to step $j+1$.

Algorithm 8: Online Single-Item Sampling ($m = 1$, n known)

Input: Stream $S = \langle i_1, \ldots, i_n \rangle$ of known length n
Output: A single item $x \in S$ sampled uniformly with $\mathbb{P}(x = i_j) = \frac{1}{n}$

```
for j ← 1 to n do
    p ← Uniform(0, 1)
    if p ≤ 1/(n−j+1) then
        return i_j
```

Rigorous Proof by Induction

Theorem 8.2.1. *Algorithm 8 ensures that every element $i_j \in S$ has exact probability $\frac{1}{n}$ of being chosen:*

$$\mathbb{P}(\textit{select } i_j) = \frac{1}{n}, \qquad \forall j \in \{1, 2, \ldots, n\}. \tag{8.7}$$

Proof. We proceed by induction on stream index j.

Base step ($j = 1$):
At step 1, for item i_1, the selection threshold is:

$$\frac{1}{n-1+1} = \frac{1}{n}. \tag{8.8}$$

Since $p \sim \text{Uniform}(0, 1)$, the probability that $p \leq \frac{1}{n}$ matches the interval length:

$$\mathbb{P}(\text{pick } i_1) = \frac{1}{n}. \tag{8.9}$$

The base case holds.

Inductive hypothesis:
Assume the property holds for all preceding indices $k \in \{1, 2, \ldots, j-1\}$, meaning:

$$\mathbb{P}(\text{pick } i_k) = \frac{1}{n}. \tag{8.10}$$

Because the algorithm halts immediately upon selecting an element, the events $\{\text{pick } i_k\}$ are mutually disjoint. Thus, the cumulative probability that at least one of the first $j-1$ elements was selected is:

$$\mathbb{P}\left(\bigcup_{k=1}^{j-1}\{\text{pick } i_k\}\right) = \sum_{k=1}^{j-1}\mathbb{P}(\text{pick } i_k) = \sum_{k=1}^{j-1}\frac{1}{n} = \frac{j-1}{n}. \tag{8.11}$$

The complementary probability that *no* element among the first $j-1$ was selected is:

$$\mathbb{P}(\text{no element selected in } \{i_1,\ldots,i_{j-1}\}) = 1 - \frac{j-1}{n} = \frac{n-j+1}{n}. \tag{8.12}$$

Inductive step:
For element i_j to be chosen at step j, two joint events must occur:

1. No element among $i_1,\ldots,i_{j-1}$ was selected earlier.
2. At step j, the random draw p satisfies $p \le \frac{1}{n-j+1}$.

$$\begin{aligned}\mathbb{P}(\text{pick } i_j) &= \mathbb{P}\left(p \le \frac{1}{n-j+1} \,\middle|\, \text{no element chosen previously}\right) \\ &\quad \cdot \mathbb{P}(\text{no element chosen previously}) \\ &= \frac{1}{n-j+1}\cdot\left(\frac{n-j+1}{n}\right) = \frac{1}{n}.\end{aligned} \tag{8.13}$$

The factor $(n-j+1)$ cancels exactly, yielding $\frac{1}{n}$. The assertion holds for all $j \in \{1,\ldots,n\}$. □

Geometric Interpretation

The probabilistic mechanism underlying Algorithm 8 admits a clear geometric interpretation illustrated in Figure 8.1.

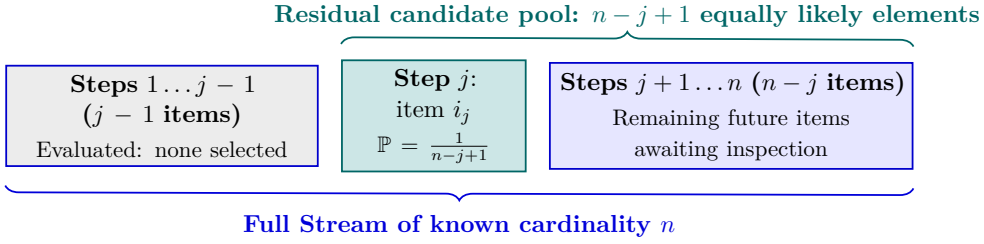


Figure 8.1: Stream space partition at step j for single-element sampling ($m=1$).

If no element was selected across the first $j-1$ steps, the target item must reside within the remaining $n-j+1$ elements. By uniformity, all remaining items share equal rights to selection, so the conditional probability of choosing i_j immediately is the reciprocal of the remaining count: $\frac{1}{n-j+1}$.

8.2.2 Multi-Item Selection: Algorithm S (Fan, Muller, Rezucha / Knuth)

When the requested sample size is $m > 1$ and total stream size n is known upfront, this principle generalizes to **Algorithm S** (introduced by Fan, Muller, and Rezucha [7] and popularized by Knuth [1]), which maintains a dynamic counter of selected elements.

State Variable Definitions

The algorithm tracks the following quantities during sequential traversal:

- n: total number of stream elements;
- m: target sample size;
- $j \in \{1, 2, \ldots, n\}$: time index of current candidate element i_j;
- s: number of items already chosen into the sample across the first $j-1$ steps ($0 \leq s \leq m$);
- $m - s$: remaining items *still needed* to complete the sample;
- $n - j + 1$: total remaining items in the stream yet to be evaluated (including i_j).

Selection Rule and Boundary Cases

At each iteration $j \in \{1, \ldots, n\}$:

1. Draw $p \sim \text{Uniform}(0, 1)$.
2. Compare p against the dynamic threshold $\frac{m-s}{n-j+1}$:
$$p \leq \frac{m-s}{n-j+1}. \tag{8.14}$$
3. If satisfied: insert i_j into sample and increment counter:
$$s \leftarrow s + 1. \tag{8.15}$$
Otherwise, discard i_j and keep s unchanged.

The algorithm handles boundary conditions automatically:

- **Sample Completed ($s = m$):** the threshold numerator becomes $m - s = 0$. Selection probability falls to zero ($\frac{0}{n-j+1} = 0$), preventing further additions and enabling early termination.
- **Forced Inclusion of All Remaining Items ($m - s = n - j + 1$):** the remaining quota matches the remaining elements. The threshold becomes $\frac{m-s}{n-j+1} = 1$, deterministically forcing inclusion of all remaining elements since $p < 1$ with probability 1.

Algorithm 9: Algorithm S (Selection Sampling by Fan, Muller, Rezucha / Knuth)

Input: Stream $S = \langle i_1, \ldots, i_n \rangle$ of known length n, sample size $m \leq n$
Output: Set C containing m items sampled uniformly

```
C ← ∅
s ← 0                          // Counter of already sampled items
for j ← 1 to n do
    if s = m then
        break                  // Sample full: terminate early
    p ← Uniform(0, 1)
    if p ≤ (m − s)/(n − j + 1) then
        C ← C ∪ {i_j}
        s ← s + 1
return C
```

Combinatorial Derivation of the Threshold

The threshold $\frac{m-s}{n-j+1}$ stems directly from combinatorial principles and the hypergeometric distribution.

Theorem 8.2.2. *Given that s elements have already been selected across the first $j-1$ steps, the conditional probability that the current item i_j belongs to a uniform sample of size m drawn from the remaining $n - j + 1$ elements is:*

$$\mathbb{P}(i_j \in \textit{Sample} \mid s \textit{ already sampled}) = \frac{m-s}{n-j+1}. \tag{8.16}$$

Proof. The conditional probability is the ratio of favorable combinations to total configurations:

$$\mathbb{P}(\text{pick } i_j \mid s) = \frac{\text{Favorable cases (complete sample with } i_j)}{\text{Possible cases (choose } m-s \text{ from residual elements)}}. \tag{8.17}$$

Fixing i_j accounts for 1 item; thus, $m - s - 1$ items must be chosen from the subsequent $n - j$ available stream items. The number of ways is given by the binomial coefficient:

$$\binom{n-j}{m-s-1}. \tag{8.18}$$

Conversely, the unconstrained combinations for choosing $m - s$ items from all $n - j + 1$ remaining candidates is:

$$\binom{n-j+1}{m-s}. \tag{8.19}$$

Computing the ratio and expanding factorials:

$$\begin{aligned}\frac{\binom{n-j}{m-s-1}}{\binom{n-j+1}{m-s}} &= \frac{\dfrac{(n-j)!}{(m-s-1)!\,((n-j)-(m-s-1))!}}{\dfrac{(n-j+1)!}{(m-s)!\,((n-j+1)-(m-s))!}} \\ &= \frac{\dfrac{(n-j)!}{(m-s-1)!\,(n-j-m+s+1)!}}{\dfrac{(n-j+1)!}{(m-s)!\,(n-j-m+s+1)!}}.\end{aligned} \tag{8.20}$$

The factor $(n - j - m + s + 1)!$ in the denominator of both fractions cancels completely. Regrouping:

$$= \frac{(n-j)!}{(n-j+1)!} \cdot \frac{(m-s)!}{(m-s-1)!}. \tag{8.21}$$

Applying the recursive factorial definition ($k! = k \cdot (k-1)!$):

$$(n-j+1)! = (n-j+1) \cdot (n-j)!, \tag{8.22}$$

$$(m-s)! = (m-s) \cdot (m-s-1)!. \tag{8.23}$$

Substituting into each term:

$$= \frac{1}{n-j+1} \cdot (m-s) = \frac{m-s}{n-j+1}. \tag{8.24}$$

Thus, the local transition probability exactly matches the global hypergeometric uniform distribution. □

8.2.3 Step-by-Step Walkthrough: Algorithm S Example

To examine how stochastic thresholds operate in practice, we trace the example from lecture notes.

Example 8.2.1 (Algorithm S Execution Trace)**.** Consider the following input configuration:

- Input stream: $S = \langle a, b, c, d, e, f, g, h \rangle \implies n = 8$.
- Target sample size: $m = 2$.
- Sequence of pseudo-random draws $p \in [0, 1)$ drawn per step:

$$p = \langle 0.5,\ 0.5,\ 0.1,\ 0.5,\ 0.5,\ 0.2,\ 0.9,\ 0.3 \rangle. \tag{8.25}$$

The step-by-step state progression is presented in Table 8.1.

Table 8.1: Step-by-step execution trace of Algorithm S ($n = 8$, $m = 2$).

j	i_j	s	$m-s$	$n-j+1$	Threshold $\frac{m-s}{n-j+1}$	p	Condition $p \leq$ Threshold	Outcome	New s
1	a	0	2	8	$\frac{2}{8} = 0.250$	0.5	$0.5 \leq 0.250$ (False)	Discarded (×)	0
2	b	0	2	7	$\frac{2}{7} \approx 0.286$	0.5	$0.5 \leq 0.286$ (False)	Discarded (×)	0
3	c	0	2	6	$\frac{2}{6} \approx 0.333$	0.1	$0.1 \leq 0.333$ (True)	**Selected (✓)**	1
4	d	1	1	5	$\frac{1}{5} = 0.200$	0.5	$0.5 \leq 0.200$ (False)	Discarded (×)	1
5	e	1	1	4	$\frac{1}{4} = 0.250$	0.5	$0.5 \leq 0.250$ (False)	Discarded (×)	1
6	f	1	1	3	$\frac{1}{3} \approx 0.333$	0.2	$0.2 \leq 0.333$ (True)	**Selected (✓)**	2
7	g	2	0	2	$\frac{0}{2} = 0.000$	0.9	$0.9 \leq 0.000$ (False)	Discarded / STOP	2
8	h	2	0	1	$\frac{0}{1} = 0.000$	0.3	$0.3 \leq 0.000$ (False)	Discarded / STOP	2

Outcome: At step $j = 6$, the counter reaches $s = m = 2$, completing the sample $C = \{c, f\}$. At steps 7 and 8, the selection threshold drops to zero, systematically discarding subsequent elements immediately.

8.3 Sampling with Unknown Stream Length (n Unknown)

In realistic streaming environments—including network telemetry, server logging, and sensor feeds—the stream size n is unknown upfront and data may cease arriving at any arbitrary time. Streaming algorithms must therefore maintain a valid sample at all moments.

8.3.1 Priority Sampling with Min-Heap (Random Keys Sampling)

A natural strategy for sampling without knowing n relies on **Random Keys Sampling** (or *Priority Sampling*).

Functional Principle

Each arriving item i_j receives an independent and identically distributed continuous random key:

$$\pi_j \sim \text{Uniform}(0, 1). \tag{8.26}$$

The algorithm dynamically tracks the m elements with the highest keys observed so far. Because keys are continuous and i.i.d., every subset of size m from the first j items has equal likelihood of possessing the m largest keys.

Management via Bounded Min-Heap

To confine memory consumption to $\mathcal{O}(m)$, we maintain a **Min-Heap** H of fixed capacity m, storing pairs $\langle \pi_j, i_j \rangle$ keyed by π_j:

- **First m elements ($j \leq m$):** insert pairs directly into H. Total initial construction time is $\mathcal{O}(m)$.
- **Subsequent elements ($j > m$):** for each candidate i_j with key π_j:
  1. Inspect the heap root, which holds the current minimum priority among the m sample elements:
  $$\pi_{\min} = \text{min_key}(H). \tag{8.27}$$
  2. **If $\pi_j \leq \pi_{\min}$:** the key fails to enter the top m. Item i_j is discarded immediately in $\mathcal{O}(1)$ time.
  3. **If $\pi_j > \pi_{\min}$:** the element holding $\pi_{\min}$ is ejected. We replace the root (`Replace-Min`) with $\langle \pi_j, i_j \rangle$ and restore heap order via `Heapify-Down` in $\mathcal{O}(\log m)$ time.

The functional architecture of Priority Sampling is illustrated in Figure 8.2.

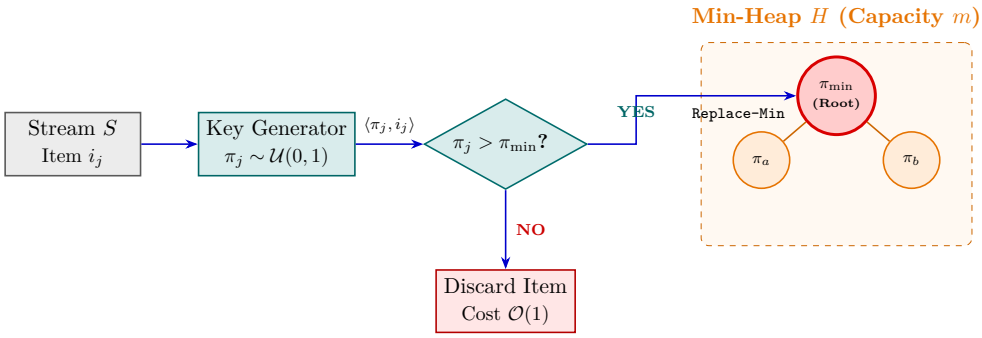


Figure 8.2: Functional flow of streaming Priority Sampling using a bounded Min-Heap of capacity m.

Property 8.3.1 (Complexity Analysis of Priority Sampling)**.** *Memory space is bounded by $\mathcal{O}(m)$ words. Worst-case processing time per element is $\mathcal{O}(\log m)$. However, because the probability that element j qualifies into the top m is $\frac{m}{j}$, the expected number of heap updates across n stream items is:*

$$\sum_{j=m+1}^{n} \frac{m}{j} = m \sum_{j=m+1}^{n} \frac{1}{j} = \mathcal{O}(m \log(n/m)). \tag{8.28}$$

The expected amortized cost per item is:

$$T_{avg} = \mathcal{O}\left(1 + \frac{m \log m \log(n/m)}{n}\right) = \mathcal{O}(1) \quad \text{for } n \gg m. \tag{8.29}$$

Nevertheless, Priority Sampling requires generating a pseudo-random key for every *stream item, alongside tree management overhead.*

8.3.2 Reservoir Sampling (Algorithm R by Waterman / Knuth)

To achieve strict $\mathcal{O}(1)$ worst-case time per item without priority heap overhead, the canonical solution is **Reservoir Sampling** (originally developed by Alan G. Waterman and formalized as **Algorithm R** by Knuth [1]).

Operational Mechanism

The algorithm maintains a simple linear array $R[1 \dots m]$, called the **reservoir**. Execution proceeds in two phases:

1. **Phase 1 (Deterministic Initialization):** the first m stream items are unconditionally assigned to their matching slots in R:

$$R[j] \leftarrow i_j, \qquad \forall j \in \{1, 2, \ldots, m\}. \tag{8.30}$$

   No pseudo-random numbers are generated during this initial phase.

2. **Phase 2 (Competitive Online Replacement):** for every subsequent item i_j arriving at time $j = m + 1, m + 2, \ldots$:

   (a) Draw a uniform discrete random integer k within the inclusive range $[1, j]$:

$$k \sim \text{rand}(1, j). \tag{8.31}$$

   (b) Evaluate admission:

   - If $k \leq m$: item i_j enters the reservoir, **replacing** slot k:

$$R[k] \leftarrow i_j. \tag{8.32}$$

     The item formerly residing at $R[k]$ is permanently evicted.
   - If $k > m$: current item i_j is discarded immediately, leaving R unchanged.

Algorithm 10: Reservoir Sampling (Algorithm R by Waterman / Knuth)

Input: Streaming sequence $S = \langle i_1, i_2, \ldots \rangle$, reservoir capacity m
Output: Array $R[1 \ldots m]$ containing a uniform sample at all times

```
for j ← 1 to m do
    R[j] ← i_j                    // Fill initial reservoir without RNG
j ← m + 1
while Stream continues and new item i_j arrives do
    k ← RandomInteger(1, j)
    if k ≤ m then
        R[k] ← i_j                // Replace slot k with probability m/j
    j ← j + 1
return R
```

The replacement mechanism is illustrated in Figure 8.3.

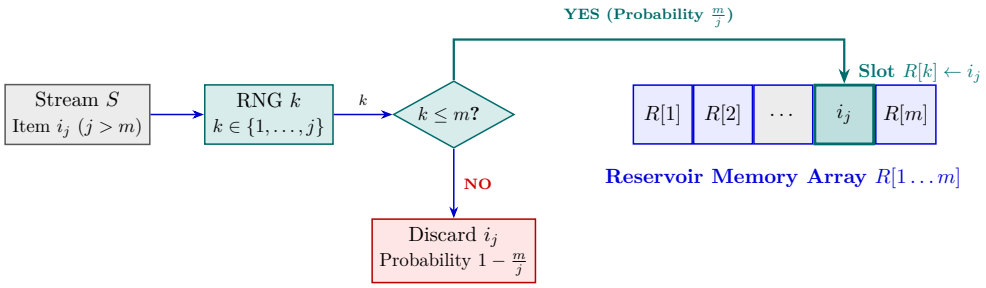


Figure 8.3: Operational replacement flow of Reservoir Sampling (Algorithm R).

8.3.3 Step-by-Step Walkthrough: Reservoir Sampling Example

We trace the step-by-step state progression on a sample stream.

Example 8.3.1 (Reservoir Sampling Execution Trace)**.** Consider the sequence:

$$S = \langle a, b, c, d, e, f, g, h, i \rangle, \qquad \text{with capacity } m = 3. \tag{8.33}$$

- **Initialization** $(j = 1, 2, 3)$**:** the first $m = 3$ elements are saved into R:

$$R = [a,\ b,\ c]. \tag{8.34}$$

- **Online updates** $(j = 4, 5, 6, 7)$**:** state transitions are detailed in Table 8.2.

Table 8.2: Step-by-step trace of Reservoir Sampling ($m = 3$).

Step j	Item i_j	Random Integer k	$k \leq m$	Executed Action	Reservoir State $R[1 \ldots 3]$
$1 \ldots 3$	a, b, c	—	—	Deterministic initialization	$[a, b, c]$
4	d	$k = 2 \in \{1 \ldots 4\}$	True $(2 \leq 3)$	Replace slot $R[2] \leftarrow d$	$[a, d, c]$
5	e	$k = 5 \in \{1 \ldots 5\}$	False $(5 > 3)$	Discard item e immediately	$[a, d, c]$
6	f	$k = 1 \in \{1 \ldots 6\}$	True $(1 \leq 3)$	Replace slot $R[1] \leftarrow f$	$[f, d, c]$
7	g	$k = 4 \in \{1 \ldots 7\}$	False $(4 > 3)$	Discard item g immediately	$[f, d, c]$

8.4 Formal Proof by Induction of Reservoir Sampling

The mathematical correctness of Reservoir Sampling ensures that upon stream termination at any point, every observed element has an identical inclusion probability.

Theorem 8.4.1 (Correctness of Reservoir Sampling)**.** *For any stream S, at any step $n \geq m$, every element i_j seen so far ($1 \leq j \leq n$) belongs to the reservoir R with probability:*

$$\mathbb{P}(i_j \in R) = \frac{m}{n}. \tag{8.35}$$

Proof. We proceed by mathematical induction on observed stream length n, keeping capacity m fixed.

Base step ($n = m$):
After the first m steps, elements $i_1, \ldots, i_m$ are deterministically placed into reservoir R:

$$R = [i_1, i_2, \ldots, i_m]. \tag{8.36}$$

Therefore, for every $j \in \{1, \ldots, m\}$:

$$\mathbb{P}(i_j \in R) = 1 = \frac{m}{m}. \tag{8.37}$$

The base case holds.

Inductive hypothesis:
Assume the property holds at step $n-1$, so for every i_j with $1 \leq j \leq n-1$:

$$\mathbb{P}(i_j \in R \text{ at step } n-1) = \frac{m}{n-1}. \tag{8.38}$$

Inductive step (transition from $n-1$ to n):
Upon arrival of the n-th element i_n, the algorithm draws:

$$k \sim \text{rand}(1, n). \tag{8.39}$$

We evaluate two mutually exclusive cases:

- **Case 1: Newly arrived element ($j = n$):**
  Element i_n enters reservoir R if and only if $k \leq m$. Because k is drawn uniformly from $\{1, 2, \ldots, n\}$:

  $$\mathbb{P}(i_n \in R) = \mathbb{P}(k \leq m) = \frac{m}{n}. \tag{8.40}$$

  The assertion holds for $j = n$.

- **Case 2: Any earlier element ($j < n$):**
  Consider an arbitrary previous element i_j ($1 \leq j \leq n-1$). For i_j to remain in R after step n, two conditions must hold:

  1. i_j was present in reservoir R at step $n-1$;

2. i_j **survives** step n, meaning it is not overwritten by i_n.

Applying the conditional probability law:

$$\mathbb{P}(i_j \in R \text{ at step } n) = \mathbb{P}(i_j \in R \text{ at step } n-1) \cdot \mathbb{P}(i_j \text{ survives step } n \mid i_j \in R \text{ at step } n-1). \tag{8.41}$$

By the inductive hypothesis (8.38), the first factor is $\frac{m}{n-1}$.

We evaluate survival event E_{surv}, meaning i_j is not overwritten at step n. An element in R survives under two disjoint scenarios:

- *Scenario A:* The new element i_n does not enter R ($k > m$), occurring with probability:

$$\mathbb{P}(k > m) = 1 - \frac{m}{n}. \tag{8.42}$$

  Here, no slot is modified, and i_j survives with certainty.

- *Scenario B:* The new element i_n enters R ($k \leq m$, probability $\frac{m}{n}$), **BUT** the replacement slot chosen is distinct from the one occupied by i_j. Because slot k is chosen uniformly among the m slots, the probability of selecting one of the other $m-1$ slots is:

$$\frac{m-1}{m}. \tag{8.43}$$

Summing probabilities of these disjoint outcomes gives total survival probability:

$$\begin{aligned}
\mathbb{P}(E_{\text{surv}}) &= \left(1 - \frac{m}{n}\right) + \left(\frac{m}{n} \cdot \frac{m-1}{m}\right) \\
&= \frac{n-m}{n} + \frac{m-1}{n} \\
&= \frac{n-m+m-1}{n} \\
&= \frac{n-1}{n}.
\end{aligned} \tag{8.44}$$

Combining the survival probability (8.44) with the inductive hypothesis:

$$\begin{aligned}
\mathbb{P}(i_j \in R \text{ at step } n) &= \left(\frac{m}{n-1}\right) \cdot \left(\frac{n-1}{n}\right) \\
&= \frac{m}{n}.
\end{aligned} \tag{8.45}$$

The factor $(n-1)$ cancels algebraically, confirming the result for all prior items $j < n$.

Having established the assertion for both $j = n$ and all $j < n$, the inductive step is complete:

$$\mathbb{P}(i_j \in R) = \frac{m}{n}, \qquad \forall j \in \{1, 2, \ldots, n\}. \tag{8.46}$$

The correctness of Reservoir Sampling is established. □

8.5 Comparative Synthesis

To consolidate our theoretical findings, Table 8.3 summarizes the operational profiles, working space, and computational costs of the three stream sampling paradigms examined.

Table 8.3: Comparative synthesis of stream sampling strategies.

Algorithm	**Knowledge of n**	**Working Space**	**Cost per Item**	**RNG Calls**	**Data Structure**
Algorithm S (Knuth)	Known a priori	$\mathcal{O}(1)$	$\mathcal{O}(1)$	n (1 per item)	Static array for sample
Priority Sampling	Unknown	$\mathcal{O}(m)$	$\mathcal{O}(\log m)$	n (1 per item)	Bounded Min-Heap of size m
Reservoir Sampling (Alg. R)	Unknown	$\mathcal{O}(m)$	$\mathcal{O}(1)$	$n - m$ (0 for first m)	Reservoir array of size m

Algorithm Engineering Remark (Optional Insight: Practical RNG Bottleneck): In high-throughput industrial streaming workloads ($n \gg 10^9$), classical Algorithm R incurs a subtle computational penalty: although operating in $\mathcal{O}(1)$ worst-case time per item, it queries the pseudo-random number generator (RNG) for each of the $n - m$ subsequent stream items. Because admission probability $\frac{m}{j}$ rapidly approaches zero, nearly all RNG invocations yield $k > m$, wasting substantial CPU cycles on discarded items. To circumvent this bottleneck, **Vitter's Algorithm Z** [8] models the number of elements skipped before the next reservoir replacement as a random variable termed the *skip distance*. By sampling this skip distance directly, the algorithm skips entire blocks of unselected stream elements in bulk without per-item RNG queries, drastically cutting total random draws from $\mathcal{O}(n)$ to $\mathcal{O}(m(1 + \log(n/m)))$.

Chapter 9

Randomized Data Structures: The Dictionary Problem and Skip Lists

Lecture 9

Randomized Data Structures: The Dictionary Problem, Block Decomposition, and Skip Lists with Backward Analysis

Date: October 12, 2026

In previous lectures, we investigated hierarchical memory models (*External Memory Model*) and high-velocity data streams (*Data Streams*), designing algorithms capable of minimizing I/O transfers and sampling elements under severe memory constraints. Across all these domains, algorithmic efficiency relies critically on maintaining dynamic collections of elements through associative structures that support search, insertion, and deletion queries in sublinear or logarithmic time.

In this chapter, we explore a foundational architectural principle in algorithm engineering: **randomization applied to data structures**. Instead of enforcing rigid deterministic balance invariants (such as complex tree rotations in AVL trees or re-coloring cases in Red-Black trees), we introduce pseudo-random coin tosses directly inside the algorithms. We demonstrate how randomization decouples asymptotic execution performance from input data distributions, achieving logarithmic guarantees $\mathcal{O}(\log n)$ both in expectation and *with high probability* (w.h.p.), while enabling simpler, lightweight implementations ideal for multi-core concurrent environments.

We trace the evolution from deterministic binary search over linked lists to the **Skip List** introduced by William Pugh, formalizing its structural height and conducting a rigorous probabilistic search cost derivation via

Backward Analysis. In the subsequent chapter (Chapter 10), we extend this randomized paradigm directly to binary search trees by examining the **Treap** and its geometric applications.

9.1 Introduction and Motivation: The Dictionary Problem

9.1.1 Formalization of the Dictionary Problem

In theoretical computer science and high-performance systems engineering, the **dictionary problem** models the dynamic maintenance of a finite subset S drawn from a totally ordered universe $\mathcal{U}$, where each item is identified by a unique key $k \in \mathcal{U}$.

Definition 9.1.1 (Dynamic Dictionary Problem)**.** Let $\mathcal{U}$ be a totally ordered universe equipped with a comparison relation $\leq$, and let $S \subseteq \mathcal{U}$ be a dynamic subset of cardinality $|S| = n$. A dictionary data structure maintains S subject to an arbitrary sequence of the following core operations:

1. Search(k): determines whether an item with key k exists in S. If present, it returns a pointer/reference to the corresponding element; otherwise, it returns a null sentinel value (NIL).

2. Insert(x): adds a new item x with key $k = \text{Key}(x)$ into the set, updating $S \leftarrow S \cup \{x\}$.

3. Delete(x): removes item x from S, updating $S \leftarrow S \setminus \{x\}$.

Beyond these three fundamental operations, real-world systems (such as DBMS storage engines and indexing layers) require efficient support for **order-related queries**:

4. Predecessor(k): returns the element in S with the greatest key strictly smaller than k:

$$\text{pred}(k) = \text{argmax}_{x \in S}\{\text{Key}(x) \mid \text{Key}(x) < k\}. \tag{9.1}$$

5. Successor(k): returns the element in S with the smallest key strictly greater than k:

$$\text{succ}(k) = \text{argmin}_{x \in S}\{\text{Key}(x) \mid \text{Key}(x) > k\}. \tag{9.2}$$

9.1.2 Comparison of Traditional Deterministic Data Structures

Classical algorithm design provides distinct trade-offs to resolve the dictionary problem. Table 9.1 summarizes the asymptotic worst-case computational complexity of the primary deterministic data structures.

Data Structure	Search	Insert	Delete	Pred/Succ	Implementation Notes
Sorted Array	$\mathcal{O}(\log n)$	$\mathcal{O}(n)$	$\mathcal{O}(n)$	$\mathcal{O}(\log n)$	Excellent binary search on static data; mutations require linear shifts of contiguous memory.
Sorted Linked List	$\mathcal{O}(n)$	$\mathcal{O}(1)^*$	$\mathcal{O}(1)^*$	$\mathcal{O}(n)$	Local $\mathcal{O}(1)$ updates given a pointer; search is strictly sequential.
Balanced BSTs (AVL, Red-Black)	$\mathcal{O}(\log n)$	$\mathcal{O}(\log n)$	$\mathcal{O}(\log n)$	$\mathcal{O}(\log n)$	Optimal worst-case bounds, but high structural overhead: color/height bits, multiple rotations, coarse locking.

Table 9.1: Asymptotic worst-case comparison of deterministic data structures (*local cost given pointer).

9.1.3 Why Introduce Randomization?

The deterministic paradigm exhibits two critical architectural bottlenecks in modern systems:

1. **Restructuring Overhead and Implementation Complexity:** Enforcing strict balance invariants requires topological checks, extra metadata per node (balance factors in AVL trees, color bits in Red-Black trees), and cascading single/double rotations upon mutations.

2. **Concurrency Bottlenecks in Multi-Threaded Engines:** In deterministic balanced trees, a single insertion or deletion can trigger a cascade of rotations propagating from leaves all the way up to the tree root. In concurrent multi-core architectures, this demands acquiring

exclusive write locks over large subtrees or even the root itself, severely degrading parallel scalability.

Introducing **randomization** revolutionizes this design space by letting internal randomized decisions (such as unbiased coin flips or independent uniform random priorities) govern structure topology:

- **Input Independence:** Asymptotic performance does not depend on key orderings or adversarial inputs, but solely on internal pseudo-random choices.
- **Logarithmic Guarantees With High Probability:** The $\mathcal{O}(\log n)$ running times are achieved not merely in expectation $\mathbb{E}[T]$, but **with high probability** (w.h.p.), meaning with failure probability at most $\mathcal{O}(n^{-c})$ for any chosen constant $c \geq 1$.
- **Lightweight and Lock-Free Concurrency:** Randomized structures rely on horizontal and vertical link pointers, making lock-free concurrent designs with atomic Compare-And-Swap (CAS) primitives straightforward.

Industrial Applications.

- **Skip Lists:** Used as the primary in-memory table (*MemTable*) in log-structured merge-tree (LSM-Tree) storage engines, including Google's **LevelDB** and Meta's **RocksDB**. They are also used in **Redis** for *Sorted Sets* (`ZSET`) [9].
- **Treaps:** Deployed in geometric indexing, ring-based dynamic routing protocols, and spatial range search systems [10].

9.2 From Binary Search on Lists to Skip Lists

In 1990, William Pugh tackled the challenge of replicating binary search efficiency directly over linked lists while preserving fast pointer updates [9].

9.2.1 The Base Linked List (Level 0)

Consider a singly linked sorted list delimited by sentinels $-\infty$ and $+\infty$, storing the set $S = \{3, 6, 7, 10, 15, 16\}$:

In this structure, any search requires $\mathcal{O}(n)$ time because pointers only advance sequentially one step at a time. Direct access to the median element in constant time is impossible.



Figure 9.1: Base sorted singly linked list with $-\infty$ and $+\infty$ sentinels.

9.2.2 The Deterministic Two-Level Model

To accelerate search, we introduce an upper **express lane** (Level L_1) that samples items at fixed step intervals from the base list (Level L_0):

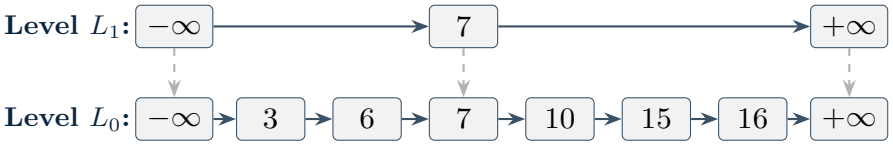


Figure 9.2: Deterministic two-level list: express lane L_1 skips groups of items.

Searching for a key k (e.g., $k = 15$) proceeds in two phases:

1. Scan horizontally along L_1 from $-\infty$ until finding the greatest item $\leq k$ (here, node 7).
2. Drop down to level L_0 at node 7 and continue linear local scanning ($7 \to 10 \to 15$).

The Deterministic Dilemma. While search is expedited, insertions and deletions disrupt the uniform spacing between express pointers. Maintaining rigid equal intervals requires an expensive $\mathcal{O}(n)$ global reorganization, negating any speedup.

9.2.3 Square Root Decomposition

Let us analytically formalize the query cost under a 2-level deterministic block scheme:

- n: total number of stored elements;
- k: number of blocks (and express pointers at level L_1);
- $\frac{n}{k}$: number of elements per block at the base level L_0.

The worst-case search cost $T(k)$ is the sum of steps along the express lane plus steps inside a block:

$$T(k) = \mathcal{O}(k) + \mathcal{O}\left(\frac{n}{k}\right) = \mathcal{O}\left(k + \frac{n}{k}\right). \tag{9.3}$$

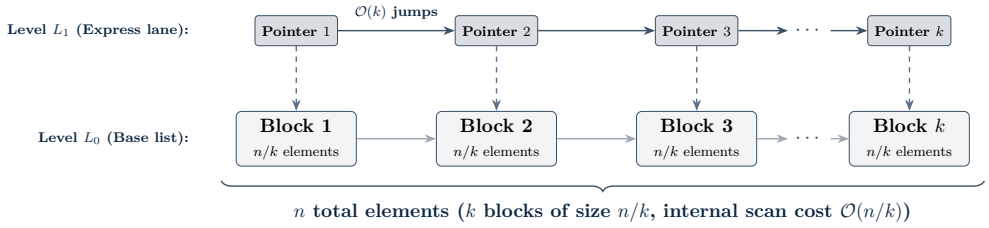


Figure 9.3: Deterministic square root decomposition into k blocks of size n/k.

Theorem 9.2.1 (Optimal Block Size)**.** *The cost function $f(k) = k + \frac{n}{k}$ for $k > 0$ has a unique global minimum at:*

$$k^* = \sqrt{n}. \tag{9.4}$$

At this optimum, the worst-case search complexity is:

$$T(\sqrt{n}) = \mathcal{O}(\sqrt{n}). \tag{9.5}$$

Proof. Differentiating $f(k) = k + \frac{n}{k}$ with respect to k:

$$\frac{df}{dk} = 1 - \frac{n}{k^2}. \tag{9.6}$$

Setting the first derivative to zero yields:

$$1 - \frac{n}{k^2} = 0 \iff k^2 = n \iff k = \sqrt{n}. \tag{9.7}$$

The second derivative is strictly positive for all $k > 0$:

$$\frac{d^2 f}{dk^2} = \frac{2n}{k^3} > 0. \tag{9.8}$$

Thus, $f(k)$ is strictly convex and achieves its minimum at $k^* = \sqrt{n}$, giving $f(\sqrt{n}) = 2\sqrt{n} = \mathcal{O}(\sqrt{n})$. □

> **The Key Insight of Skip Lists:** Avoid rigid deterministic reorganization by assigning each node's level via independent probabilistic coin tosses!

9.3 Skip Lists: Structure and Formal Probabilistic Analysis

9.3.1 Formal Definition

A Skip List is a sequence of sorted linked lists $\{L_0, L_1, \ldots, L_h\}$ satisfying:

Definition 9.3.1 (Skip List)**.** Let S be a set of n distinct keys. A Skip List for S satisfies:

1. Level L_0 contains all n keys in ascending order, bounded by sentinels $-\infty$ and $+\infty$.
2. For each level $i \geq 0$, $L_{i+1} \subset L_i$.
3. **Independent Geometric Growth:** An element in L_i is promoted to L_{i+1} independently with probability $p = \frac{1}{2}$ (a fair coin flip yielding **Heads**). If the flip yields **Tails**, its tower stops.

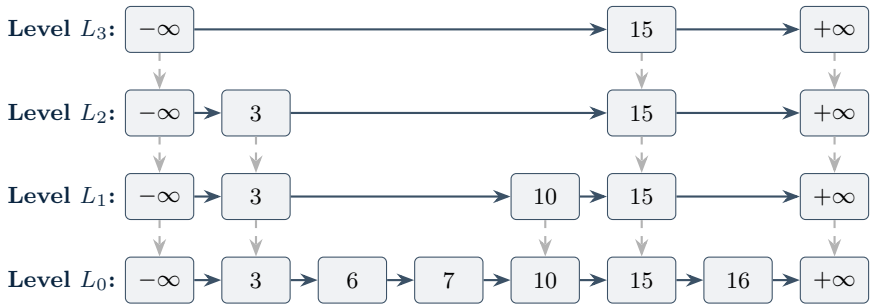


Figure 9.4: Multi-level Skip List representation across 4 levels (L_0 through L_3).

9.3.2 Core Algorithms

1. Search (Search(k))

The search algorithm (Algorithm 11) starts at the highest level h at $-\infty$. If the forward key is $\leq k$, it moves right; otherwise, it drops down one level.

2. Insertion (Insert(x))

To insert key k:

1. Search for the key's position, storing predecessors in array update$[0 \ldots h]$.
2. Sample a tower height l by simulating independent coin flips:

$$l \leftarrow 0; \quad \textbf{while } \text{random}() < 0.5 \textbf{ do } l \leftarrow l + 1.$$

3. Allocate the new node of height l and splice pointers for each level $i \in [0, l]$:

$$\text{node.next}[i] \leftarrow \text{update}[i].\text{next}[i], \qquad \text{update}[i].\text{next}[i] \leftarrow \text{node}. \quad (9.9)$$

Algorithm 11: Skip List Search (SEARCH(k))

Input: Skip List SL with maximum height h, search key $k \in \mathcal{U}$
Output: Node containing k, or NIL if not present

```
curr ← head of Skip List at level h
for i ← h to 0 do
    while curr.next[i] ≠ NIL and curr.next[i].key ≤ k do
        curr ← curr.next[i]
if curr.key == k then
    return curr
return NIL
```

3. Deletion (Delete(x))

Find predecessors across all levels and bypass: update[i].next[i] $\leftarrow x$.next[i], then deallocate.

9.3.3 Operational Costs

$$\text{Cost}(\textsc{Search}) = \#\text{vertical pointers} + \#\text{horizontal pointers}, \tag{9.10}$$

$$\text{Cost}(\textsc{Insert}) = \text{Cost}(\textsc{Search}) + \text{height(key)}, \tag{9.11}$$

$$\text{Cost}(\textsc{Delete}) = \text{Cost}(\textsc{Search}) + \#\text{pointer updates}. \tag{9.12}$$

9.3.4 Proof: Maximum Height of a Skip List

Theorem 9.3.1 (Logarithmic Height Bound)**.** *Let S be a set of n keys in a Skip List with $p = \frac{1}{2}$, and let $H = \max_{k \in S}\{\text{level}(k)\}$. For any constant $c > 1$:*

$$\mathbb{P}(H \geq c \log_2 n) \leq \frac{1}{n^{c-1}} = n^{-(c-1)}. \tag{9.13}$$

Thus, Skip List height is $\mathcal{O}(\log n)$ with high probability (w.h.p.).

Proof. The proof proceeds in three analytical steps:

Step 1: Single element tower height. Let level(k) be the level assigned to an item with key k. An element reaches a level $\geq l$ if and only if it obtains at least l consecutive successes (Heads) in independent coin tosses:

$$\mathbb{P}(\text{level}(k) \geq l) = \left(\frac{1}{2}\right)^l. \tag{9.14}$$

Step 2: Union bound across all n keys. The overall Skip List height H satisfies $H \geq l$ if and only if at least one key reaches level $\geq l$:

$$\mathbb{P}(H \geq l) = \mathbb{P}\left(\bigcup_{k \in S} \{\text{level}(k) \geq l\}\right) \leq \sum_{k=1}^{n} \mathbb{P}(\text{level}(k) \geq l) = n\left(\frac{1}{2}\right)^{l} = \frac{n}{2^l}. \tag{9.15}$$

Step 3: Setting logarithmic height threshold $l = c \log_2 n$. For any constant $c > 1$:

$$\mathbb{P}(H \geq c \log_2 n) \leq \frac{n}{2^{c \log_2 n}} = \frac{n}{(2^{\log_2 n})^c} = \frac{n}{n^c} = \frac{1}{n^{c-1}} = n^{-(c-1)}. \tag{9.16}$$

□

9.3.5 Search Cost: Backward Analysis

To determine horizontal traversal steps, we trace the search path **in reverse**, starting from the target at L_0 up to the head sentinel.

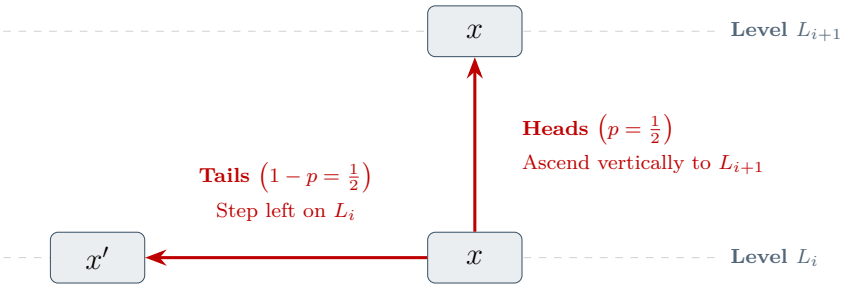


Figure 9.5: Transitions in backward analysis: vertical ascent (Heads) vs horizontal left step (Tails).

At each step backward from node x at level i:

1. If x was promoted to L_{i+1} (Heads, $p = 1/2$), the search path entered from L_{i+1}; we step vertically up.
2. If x was not promoted (Tails, $1 - p = 1/2$), the search path came from the left on L_i; we step left.

Theorem 9.3.2 (Expected Horizontal Steps per Level)**.** *The expected number of leftward steps before ascending is:*

$$\mathbb{E}[\#tails] = \sum_{h=0}^{\infty} h\left(\frac{1}{2}\right)^{h} = 2. \tag{9.17}$$

Proof. For $|x| < 1$, the geometric series evaluates to:

$$\sum_{h=0}^{\infty} x^h = \frac{1}{1-x}. \tag{9.18}$$

Differentiating term by term with respect to x:

$$\sum_{h=1}^{\infty} h x^{h-1} = \frac{1}{(1-x)^2}. \tag{9.19}$$

Multiplying by x:

$$\sum_{h=0}^{\infty} h x^h = \frac{x}{(1-x)^2}. \tag{9.20}$$

Evaluating at $x = \frac{1}{2}$:

$$\sum_{h=0}^{\infty} h \left(\frac{1}{2}\right)^h = \frac{\frac{1}{2}}{\left(1-\frac{1}{2}\right)^2} = \frac{1/2}{1/4} = 2. \tag{9.21}$$

□

Since the maximum height is $\mathcal{O}(\log n)$, the total expected search cost is:

$$\mathbb{E}[\text{Search Cost}] \leq 2 \cdot \mathcal{O}(\log n) + \mathcal{O}(\log n) = \mathcal{O}(\log n). \tag{9.22}$$

9.3.6 Advanced Application: Vector Databases and HNSW Graphs

Modern AI vector search applies the Skip List principle to multi-dimensional graphs via **HNSW** (*Hierarchical Navigable Small World*) [11]. Dense vectors reside at L_0, while upper layers contain sparser subgraphs with long-range links acting as express lanes for logarithmic Approximate Nearest Neighbor Search.

9.4 Summary and Outlook: Toward Randomized Search Trees

In this chapter we explored how internal algorithmic randomization eliminates the structural rigidity and maintenance overhead of classical balanced search structures:

- Starting from the sequential bottleneck of standard linked lists, we derived hierarchical express lanes (transitioning from deterministic $\sqrt{n}$ block partitioning to geometric towers governed by coin-flip promotion probability $p = 1/2$);
- The **Skip List** emulates binary search probabilistically, guaranteeing $\mathcal{O}(n)$ expected space and $\mathcal{O}(\log n)$ runtime with high probability for search, insertion, and deletion;
- The **Backward Analysis** technique inverts the analytical perspective by traversing the query path in reverse from the target key back to the top sentinels, establishing that the expected number of horizontal shifts per level is strictly bounded by $1/p = 2$.

While Skip Lists solve the dictionary problem through multi-level linked sequences, an important architectural question arises: *can the same randomized elegance be applied directly to binary search trees, eliminating rigid deterministic balance invariants and complex tree rotations?* Chapter 10 addresses this question by presenting the **Treap** (Randomized Search Tree), exploring its structural uniqueness, rotation mechanics, the Seidel-Aragon probabilistic analysis, and multi-dimensional geometric applications.

Chapter 10

Randomized Search Trees: Treaps, Dynamic Operations, and Geometric Applications

Lecture 10

Randomized Search Trees (Treaps): Topological Uniqueness, Tree Rotations, Insert, Delete, Split, and Merge Operations, Seidel-Aragon Probabilistic Analysis, Cartesian Trees, and 3-Sided Range Queries

Date: October 13, 2026

In previous lectures, we investigated how randomization can overcome the rigid constraints of deterministic data structures, studying the Skip List as a probabilistic generalization of binary search on linked lists. In this chapter, we extend the probabilistic paradigm to **binary search trees**, addressing one of the most elegant and versatile data structures in algorithm engineering: the **Treap** (*Tree + Heap*), formally introduced by Raimund Seidel and Cecilia Aragon [10].

The fundamental principle of the Treap is to guarantee automatic logarithmic balance without storing balancing factors or color bits, and without triggering cascading rebalancing operations as in AVL or Red-Black trees. By assigning independent continuous random priorities to nodes, the Treap mirrors the topological distribution of a binary search tree constructed by inserting keys according to a uniform random permutation.

In this formal treatment, we analyze:

- The formal definition of the Treap and the proof of the **Structural Uniqueness Theorem**;

- The mechanics of **tree rotations** (right and left) and invariant preservation;
- Complete dynamic algorithms: SEARCH, INSERT (with bubble-up), DELETE (with trickle-down), SPLIT, and MERGE;
- The rigorous probabilistic analysis by Seidel and Aragon: proving that the expected depth of each node is $\mathcal{O}(\log n)$ and that the **expected number of rotations per update is** $\mathcal{O}(1)$;
- The geometric isomorphism with Vuillemin's **Cartesian Trees** [12], deterministic linear construction $\mathcal{O}(n)$ using a monotonic stack, and optimal resolution of **3-Sided Range Queries** with *output-sensitive* complexity $\mathcal{O}(\log n + k_{\text{out}})$.

10.1 Introduction and Formal Treap Definition

10.1.1 Motivation and Core Concept

In classical binary search trees (BST), the efficiency of search, insert, and delete operations is strictly governed by the tree height h:

$$\text{Cost}(\text{OPERATION}) = \mathcal{O}(h). \tag{10.1}$$

In the best case, a perfectly balanced tree guarantees $h = \lfloor \log_2 n \rfloor$. However, adversarial or sorted insertion sequences degrade the topology into a degenerate linked list, yielding worst-case height $h = \mathcal{O}(n)$.

Deterministic balanced trees (such as AVL trees [1] and Red-Black trees) restore logarithmic balance by enforcing rigid structural invariants:

- They require auxiliary memory per node (balance factors or color bits);
- They introduce complex single and double rotation case analyses during updates;
- They cause severe synchronization bottlenecks in concurrent multi-threaded environments, where rebalancing cascades can propagate up to the root, requiring extensive lock acquisitions.

The **Treap** resolves this challenge by cleanly decoupling the logical data domain (governed by **keys**) from the structural layout (governed by **priorities**).

Definition 10.1.1 (Treap Node)**.** Each node u in a Treap stores an ordered pair:

$$u = \langle k(u), p(u) \rangle, \tag{10.2}$$

where:

- $k(u) \in \mathcal{U}$ is the **key** drawn from a totally ordered universe (horizontal ordering);
- $p(u) \in \mathbb{R}$ is the **priority** drawn from a continuous totally ordered numeric domain (vertical ordering).

10.1.2 Fundamental Structural Invariants

A binary tree T is a valid Treap if and only if it simultaneously satisfies two orthogonal invariants for every node $u \in T$:

1. **Binary Search Tree (BST) Property on Keys:**
   The horizontal layout reflects key order:

   $$\forall v \in \text{left}(u) \implies k(v) < k(u), \tag{10.3}$$
   $$\forall w \in \text{right}(u) \implies k(w) > k(u). \tag{10.4}$$

   An in-order traversal yields the keys in strictly increasing order.

2. **Max-Heap Property on Priorities:**
   The vertical hierarchy reflects priority order:

   $$\forall v \in \{\text{left}(u), \text{right}(u)\}, \quad v \neq \text{NIL} \implies p(u) \geq p(v). \tag{10.5}$$

   The root of the tree holds the maximum priority in the entire Treap.

10.1.3 Structural Uniqueness Theorem

A central theorem establishes that the tree shape depends only on the set of pairs, not on their insertion history.

Theorem 10.1.1 (Treap Topological Uniqueness)**.** *Let S be a set of n pairs $S = \{\langle x_1, p_1 \rangle, \ldots, \langle x_n, p_n \rangle\}$ with distinct keys ($x_i \neq x_j$) and distinct priorities ($p_i \neq p_j$). Then there exists **one and only one** Treap T storing S.*

Proof. By mathematical induction on $n = |S|$.

- **Base case ($n = 0$ or $n = 1$):**
  For $n = 0$, the unique tree is empty (NIL). For $n = 1$, the only valid tree is a single isolated node containing $\langle x_1, p_1 \rangle$. Both cases are uniquely determined.

- **Inductive step ($n > 1$):**
  Assume the theorem holds for all sets of cardinality strictly less than n. Consider set S with $|S| = n$.

  Since all priorities $\{p_1, \ldots, p_n\}$ are distinct, there is a unique index $r \in \{1, \ldots, n\}$ such that:

$$p_r = \max_{1 \le i \le n} \{p_i\}. \tag{10.6}$$

  To satisfy the Max-Heap invariant globally, the root of T must be $\langle x_r, p_r \rangle$. No other node can serve as root.

  With the root fixed at $\langle x_r, p_r \rangle$, the BST property deterministically partitions the remaining $n - 1$ elements:

$$S_L = \{\langle x_i, p_i \rangle \in S \setminus \{\langle x_r, p_r \rangle\} \mid x_i < x_r\}, \tag{10.7}$$
$$S_R = \{\langle x_i, p_i \rangle \in S \setminus \{\langle x_r, p_r \rangle\} \mid x_i > x_r\}. \tag{10.8}$$

  Because keys are distinct, this partition is unique.

  Moreover, $|S_L| < n$ and $|S_R| < n$. By induction, there exists a unique valid Treap T_L for S_L and a unique valid Treap T_R for S_R. Setting T_L as the left child and T_R as the right child produces the unique binary tree satisfying both BST and Heap properties. The topology is therefore unique for all $n \ge 0$.

□

Corollary 10.1.1 (Equivalence to Random BST)**.** *When priorities p_i are drawn i.i.d. from a continuous uniform distribution $\mathcal{U}[0, 1]$, descending priority order induces a uniform random permutation $\pi \in \mathcal{S}_n$ on keys. Hence, the resulting Treap is* ***topologically identical to a BST constructed by inserting keys in the order of*** *π.*

10.2 Elementary Operations: Tree Rotations

10.2.1 Right and Left Rotations

To restore the Max-Heap invariant following insertions or deletions without disturbing the in-order key sequence, the Treap uses **tree rotations**. A

rotation swaps the parent-child relationship between two adjacent nodes while preserving the in-order key traversal:

- **Right Rotation (Rotate-Right(v)):**
  Applied when u is the left child of v. Node u ascends to replace v, v becomes the right child of u, and the right subtree of u (B) becomes the left subtree of v.
- **Left Rotation (Rotate-Left(u)):**
  Symmetric dual of right rotation. Applied when v is the right child of u. Node v ascends to replace u, u becomes the left child of v, and the left subtree of v (B) becomes the right subtree of u.

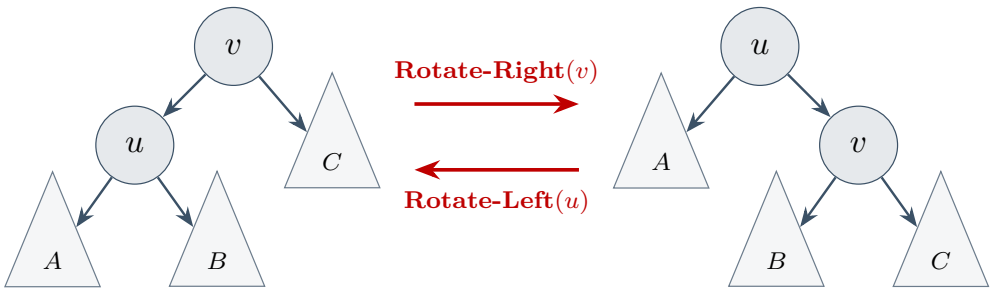


Figure 10.1: Algebraic mechanics of tree rotations. In both configurations, the in-order key traversal remains strictly invariant: $A < u < B < v < C$.

10.2.2 Rigorous Invariant Preservation

Property 10.2.1 (BST Ordering Invariance)**.** *Before right rotation:*

$$\forall a \in A,\ b \in B,\ c \in C \implies k(a) < k(u) < k(b) < k(v) < k(c). \tag{10.9}$$

After right rotation:

- *A is left child of u* $\implies \forall a \in A,\ k(a) < k(u)$;
- *v is right child of u* $\implies k(u) < k(v)$;
- *B is left child of v* $\implies \forall b \in B,\ k(u) < k(b) < k(v)$;
- *C is right child of v* $\implies \forall c \in C,\ k(v) < k(c)$.

The BST property is unconditionally preserved. Since each rotation updates only 3 pointer fields, its running time is strictly $\mathcal{O}(1)$.

10.3 Fundamental Dynamic Algorithms

10.3.1 Search (Search)

Search relies purely on keys and mirrors standard BST lookup:

Algorithm 12: Key Lookup in a Treap (SEARCH(u, k))

Input: Current root u, search key $k \in \mathcal{U}$
Output: Pointer to node containing k, or NIL

```
if u = NIL or k = k(u) then
    return u
if k < k(u) then
    return SEARCH(left(u), k)
else
    return SEARCH(right(u), k)
```

The runtime is $\mathcal{O}(h)$, which evaluates to expected $\mathcal{O}(\log n)$ as established in Section 10.4.

10.3.2 Insertion (Insert) via Bubble-Up

Inserting key k follows two steps:

1. **Leaf Insertion:** Sample $p \sim \mathcal{U}[0, 1]$, traverse down via standard BST search to locate the appropriate leaf slot, and allocate $z = \langle k, p \rangle$.
2. **Bubble-Up:** While $z \neq$ root and $p(z) > p(\text{parent}(z))$, rotate z upward:
   - If z is left child of parent, perform ROTATE-RIGHT(parent(z));
   - If z is right child of parent, perform ROTATE-LEFT(parent(z)).

Algorithm 13: Treap Insertion (INSERT(T, k))

Input: Treap root T, key k
Output: Updated root containing k

```
p ← RANDOMUNIFORM(0, 1)
z ← CREATENODE(k, p)
return INSERTNODE(T, z)
```

Algorithm 14: Recursive Insertion with Bubble-Up Rotations (InsertNode(u, z))

Input: Current subtree root u, new node z
Output: Updated subtree root

```
if u = NIL then
    return z
if k(z) < k(u) then
    left(u) ← InsertNode(left(u), z)
    if p(left(u)) > p(u) then
        u ← Rotate-Right(u)
else if k(z) > k(u) then
    right(u) ← InsertNode(right(u), z)
    if p(right(u)) > p(u) then
        u ← Rotate-Left(u)
return u
```

Step-by-Step Trace: Insertion with Bubble-Up

To trace the insertion mechanics, consider the lecture Treap storing the key set $\{(G, 3), (H, 8), (I, 6), (M, 9), (P, 2), (R, 5), (T, 7)\}$ and suppose we insert the new pair:

$$z = \langle K, 7.5 \rangle. \tag{10.10}$$

Execution proceeds in two sequential phases:

1. **Phase 1 (Standard BST Leaf Placement):**
   - Compare with root M: since $K < M$, descend left to H.
   - Compare with H: since $K > H$, descend right to I.
   - Compare with I: since $K > I$ and the right pointer of I is NIL, node $z = \langle K, 7.5 \rangle$ is attached as the right child of I.

   In this intermediate state (Figure 10.2(a)), the BST horizontal key ordering is preserved, but a local Max-Heap violation occurs between parent I and child K:

   $$p(K) = 7.5 > p(I) = 6. \tag{10.11}$$

2. **Phase 2 (Bubble-Up via Left Rotation on I):** Since K is the right child of I, the heap invariant is restored by invoking Rotate-Left(I):

- Node K is promoted to take I's place as the right child of H.
- Node I moves down to become the left child of K.
- Any left subtree of K (empty here) would become the right child of I.

3. **Termination Check at Parent Level:** The new parent of K is H. We check the heap condition:

$$p(H) = 8 \geq p(K) = 7.5. \tag{10.12}$$

Since the parent's priority dominates the child's, the Max-Heap invariant is satisfied across all ancestor levels. The insertion terminates successfully (Figure 10.2(b)).

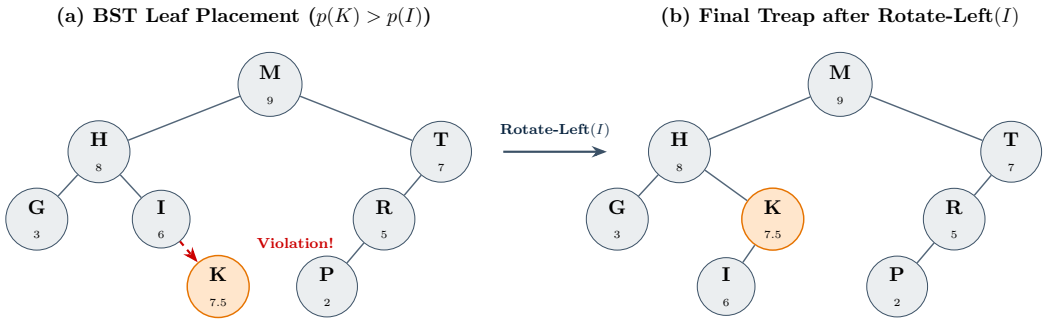


Figure 10.2: Trace of inserting $\langle K, 7.5\rangle$: (a) initial placement as a BST leaf violating heap priority at $I(6)$; (b) final valid configuration after left rotation on I.

10.3.3 Deletion (Delete) via Trickle-Down

To delete key k without violating Treap invariants, the target node is rotated downward until it reaches a leaf:

1. Locate node u with $k(u) = k$ using BST descent.
2. While u is not a leaf, select the child with **maximum priority**:
   - If $p(\text{left}(u)) > p(\text{right}(u))$, execute Rotate-Right($u$);
   - Otherwise, execute Rotate-Left(u).

   This rotation preserves the Max-Heap property above and demotes u by one level.

3. Once u becomes a leaf (both children NIL), deallocate it in $\mathcal{O}(1)$.

Algorithm 15: Treap Deletion (DELETE(u, k))

Input: Subtree root u, key k to remove
Output: Updated subtree root without k

```
1  if u = NIL then
2      return NIL
3  if k < k(u) then
4      left(u) ← DELETE(left(u), k)
5  else if k > k(u) then
6      right(u) ← DELETE(right(u), k)
7  else
8      if left(u) = NIL and right(u) = NIL then
9          DEALLOCATE(u)
10         return NIL
11     else if left(u) ≠ NIL and (right(u) = NIL or
         p(left(u)) > p(right(u))) then
12         u ← ROTATE-RIGHT(u)
13         right(u) ← DELETE(right(u), k)
14     else
15         u ← ROTATE-LEFT(u)
16         left(u) ← DELETE(left(u), k)
17 return u
```

Step-by-Step Trace: Deletion via Trickle-Down

Suppose we now remove internal node H from the Treap obtained after inserting K:

$$\text{DELETE}(T, H). \tag{10.13}$$

Instead of swapping values with the in-order predecessor or successor, the Treap tricks H down into a leaf via priority-guided rotations:

1. **Initial State of H:** Node H has two children: G on the left ($p(G) = 3$), and K on the right ($p(K) = 7.5$). The algorithm compares their priorities:

$$\max(p(G), p(K)) = \max(3, 7.5) = p(K) = 7.5. \tag{10.14}$$

Since the higher priority child is on the right (K), the algorithm executes ROTATE-LEFT(H):

- K is promoted to become the new left child of root M.
- H demotes to become the left child of K.
- The former left subtree of K ($I(6)$) becomes the new right child of H.

The children of H are now $G(3)$ (left) and $I(6)$ (right).

2. **Second Trickle-Down Step:** Node H still has two children. Comparing priorities:

$$\max(p(G), p(I)) = \max(3, 6) = p(I) = 6. \tag{10.15}$$

Since the child with higher priority is on the right (I), we apply a second left rotation Rotate-Left(H):

- I is promoted to become the left child of K.
- H demotes to become the left child of I.
- The former left child of I (NIL) becomes the right child of H.

Now H has only left child $G(3)$ and no right child.

3. **Third Trickle-Down Step (Right Rotation):** With only left child G, execute Rotate-Right(H):

- G rises to become the left child of I.
- H demotes to become the right child of G.

Now both child pointers of H are NIL: H is a **leaf**!

4. **Final Deallocation $\mathcal{O}(1)$:** Parent G's right child pointer is set to NIL, and H is freed. The resulting tree (Figure 10.3) satisfies all Treap invariants.

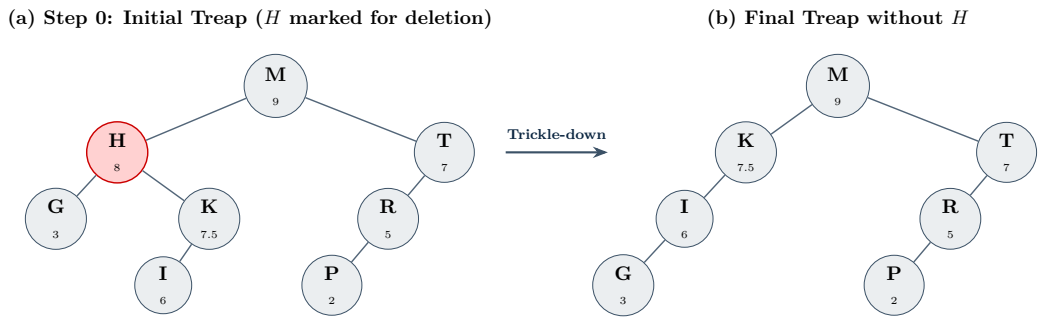


Figure 10.3: Trace of deleting node H: (a) initial configuration (Step 0) with target node H highlighted; (b) final valid Treap after trickle-down rotations and $\mathcal{O}(1)$ deallocation of leaf H.

10.3.4 Advanced Operations: Split and Merge

- **Split (Split(T, k)):**
  Partitions T into T_1 ($\leq k$) and T_2 ($> k$) in $\mathcal{O}(\log n)$ time by inserting dummy key $\langle k, +\infty \rangle$. Because its priority is $+\infty$, bubble-up brings it to the root. Its left subtree is T_1 and right subtree is T_2. Removing the dummy root yields the two partitioned trees (Figure 10.4(a)).

- **Merge (Merge(T_1, T_2)):**
  Given $\max(T_1) < \min(T_2)$, creates a dummy root $\langle k^*, -\infty \rangle$ with left child T_1 and right child T_2, then removes it via trickle-down in $\mathcal{O}(\log n)$ time (Figure 10.4(b)).

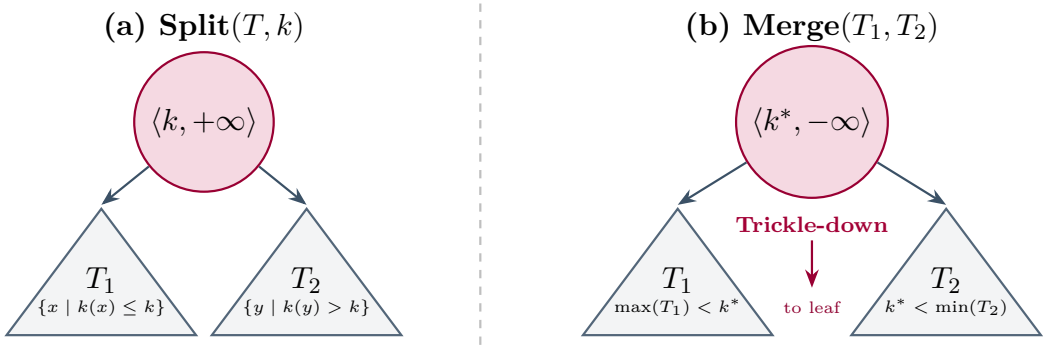


Figure 10.4: Constructive mechanics of Split and Merge: assigning extreme sentinel priorities ($+\infty$ and $-\infty$) leverages native Treap rotations to partition or join sorted sets in $\mathcal{O}(\log n)$ time.

10.4 Probabilistic Analysis (Seidel and Aragon)

10.4.1 Probability Space and Indicator Variables

Let $S = \{x_1, \ldots, x_n\}$ be sorted keys: $x_1 < \cdots < x_n$, with i.i.d. priorities $p_i \sim \mathcal{U}[0,1]$. Define the indicator variable:

$$A_{i,j} = \begin{cases} 1 & \text{if } x_i \text{ is an ancestor of } x_j \text{ in } T, \\ 0 & \text{otherwise.} \end{cases} \tag{10.16}$$

10.4.2 Ancestor Condition Theorem

Theorem 10.4.1 (Ancestor Condition)**.** *Node x_i is an ancestor of x_j if and only if p_i is the maximum priority in the range between x_i and x_j:*

$$A_{i,j} = 1 \iff p_i = \max\left\{p_k \mid k \in [\min(i,j), \max(i,j)]\right\}. \tag{10.17}$$

Lemma 10.4.1 (Ancestor Probability)**.**

$$\mathbb{P}(A_{i,j} = 1) = \frac{1}{|j-i|+1}. \tag{10.18}$$

Proof. Among the $|j-i|+1$ keys in $[\min(i,j), \max(i,j)]$, each element has equal probability $1/(|j-i|+1)$ of attaining the maximum priority. □

10.4.3 Expected Depth of a Key

Theorem 10.4.2 (Expected Node Depth)**.** *For any key x_i:*

$$\mathbb{E}[\text{Depth}(x_i)] = H_i + H_{n-i+1} - 2 = \mathcal{O}(\log n), \tag{10.19}$$

where $H_k = \sum_{m=1}^{k} \frac{1}{m}$ is the k-th harmonic number.

Proof. By linearity of expectation:

$$\begin{aligned} \mathbb{E}[\text{Depth}(x_i)] &= \sum_{j \neq i} \mathbb{P}(A_{j,i} = 1) \\ &= \sum_{j=1}^{i-1} \frac{1}{i-j+1} + \sum_{j=i+1}^{n} \frac{1}{j-i+1} \\ &= (H_i - 1) + (H_{n-i+1} - 1) \\ &= H_i + H_{n-i+1} - 2 \leq 2\ln n = \mathcal{O}(\log n). \end{aligned} \tag{10.20}$$

□

10.4.4 Expected Rotations per Update: $\mathcal{O}(1)$

Theorem 10.4.3 (Expected Rebalancing Rotations)**.** *The expected number of rotations per insertion or deletion is strictly bounded by a constant:*

$$\mathbb{E}[\#rotations] \leq 2. \tag{10.21}$$

10.5 Geometric Isomorphism and Multidimensional Applications

By associating the key x with the abscissa and the priority y with the ordinate of a 2D point, the Treap exhibits a direct isomorphism with **Cartesian Trees** [12] and **Priority Search Trees** [13]. This geometric perspective enables linear-time $\mathcal{O}(n)$ deterministic construction, $\mathcal{O}(1)$ Range Minimum Queries (RMQ), and optimal $\mathcal{O}(\log n + k)$ 3-sided orthogonal range searching.

10.5.1 Cartesian Trees and 2D Point Mapping

Definition 10.5.1 (Cartesian Tree Isomorphism)**.** Given a set of n points $P = \{(x_1, y_1), \ldots, (x_n, y_n)\} \subset \mathbb{R}^2$ with distinct coordinates, the Cartesian tree associated with P is the unique binary tree in which:

- The abscissae x_i serve as **BST keys** (horizontal ordering);
- The ordinates y_i serve as **Max-Heap priorities** (vertical ordering).

The point with maximum y-coordinate serves as the root; its x-coordinate partitions the plane into the left and right subtrees (Figure 10.5).

10.5.2 Linear-Time $\mathcal{O}(n)$ Construction via Monotonic Stack

When points are sorted by x-coordinate ($x_1 < x_2 < \cdots < x_n$), the Cartesian tree can be constructed in strictly linear deterministic time $\mathcal{O}(n)$ using an incremental **monotonic stack** algorithm (Algorithm 16):

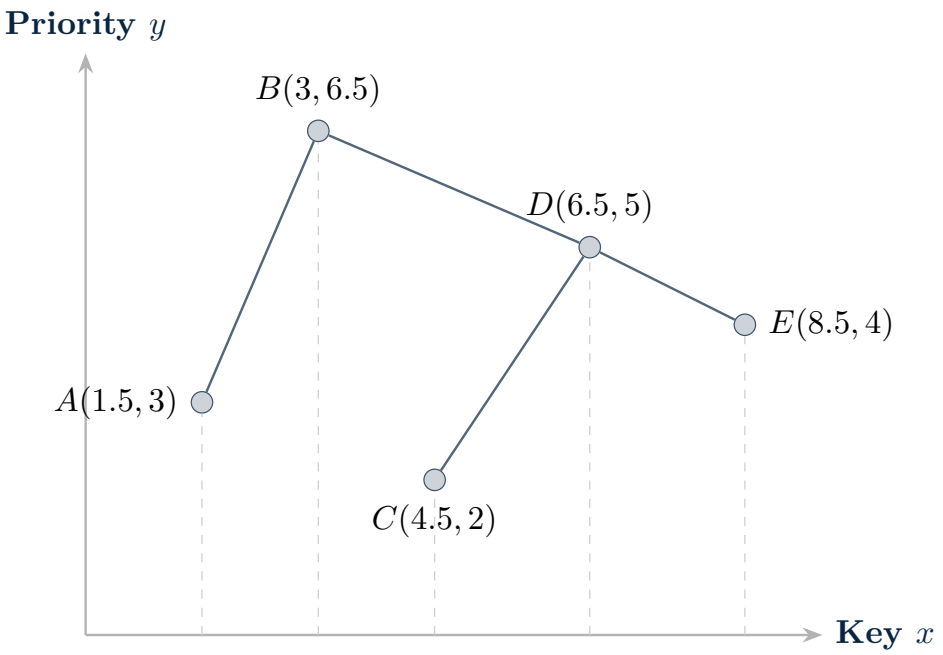


Figure 10.5: Geometric isomorphism of the Cartesian tree: B has maximum y and serves as root. Left subtree is A; right subtree contains $\{C, D, E\}$ rooted at local maximum D.

Algorithm 16: Linear Cartesian Tree Construction (BUILD-CARTESIAN($A[1 \dots n]$))

Input: Array $A[1 \dots n]$ pre-sorted by x-coordinate
Output: Root of Cartesian tree

```
Initialize empty stack S
for i ← 1 to n do
    last ← NIL
    while S ≠ ∅ and y(S.TOP()) < y(A[i]) do
        last ← S.POP()
    left(A[i]) ← last
    if S ≠ ∅ then
        right(S.TOP()) ← A[i]
    S.PUSH(A[i])
return bottom element of stack (the global root)
```

Amortized Complexity Analysis: The algorithm maintains the right spine of the current tree on the stack. At each step, point $A[i]$ pops all nodes with lower y-coordinates; the last popped node becomes its left child, while $A[i]$ becomes the right child of the new stack top. Because each of the n nodes is pushed onto the stack exactly once and popped at most once, the total number of stack operations is bounded by $2n$. The construction time is therefore strictly deterministic and linear: $\mathcal{O}(n)$.

10.5.3 Range Minimum Query (RMQ) in $\mathcal{O}(1)$

The Cartesian tree provides the foundational reduction linking range minimum searches on arrays to lowest common ancestor queries on trees:

$$\mathrm{RMQ}_A(i, j) = \arg\min_{i \le k \le j} A[k]. \tag{10.22}$$

By constructing a Cartesian tree (in Min-Heap form) over pairs $(k, A[k])$:

$$\mathrm{RMQ}_A(i, j) = \mathrm{LCA}_{\mathrm{CartesianTree}}(A[i], A[j]), \tag{10.23}$$

where LCA denotes the *Lowest Common Ancestor*. Because static LCA queries can be answered in $\mathcal{O}(1)$ time following $\mathcal{O}(n)$ preprocessing (via the Bender and Farach-Colton algorithm [14]), the Cartesian tree allows answering RMQ in optimal $\mathcal{O}(1)$ time.

10.5.4 3-Sided Range Queries

In geospatial applications, temporal databases, and computational geometry, a fundamental class of multidimensional queries is given by **3-sided range queries** (semi-infinite orthogonal rectangles):

$$Q = [x_{\min}, x_{\max}] \times [y_{\min}, +\infty). \tag{10.24}$$

Such queries seek all points $p = (x, y)$ satisfying simultaneously:

$$x_{\min} \leq x \leq x_{\max} \quad \wedge \quad y \geq y_{\min}. \tag{10.25}$$

In the context of Treaps, the horizontal axis represents the alphabetical or lexicographical ordering of keys (defining the query as a letter interval $[x_{\min}, x_{\max}]$), while the vertical axis measures numeric priorities (expressing the vertical constraint as a priority interval $[y_{\min}, +\infty)$), as illustrated in Figure 10.6.

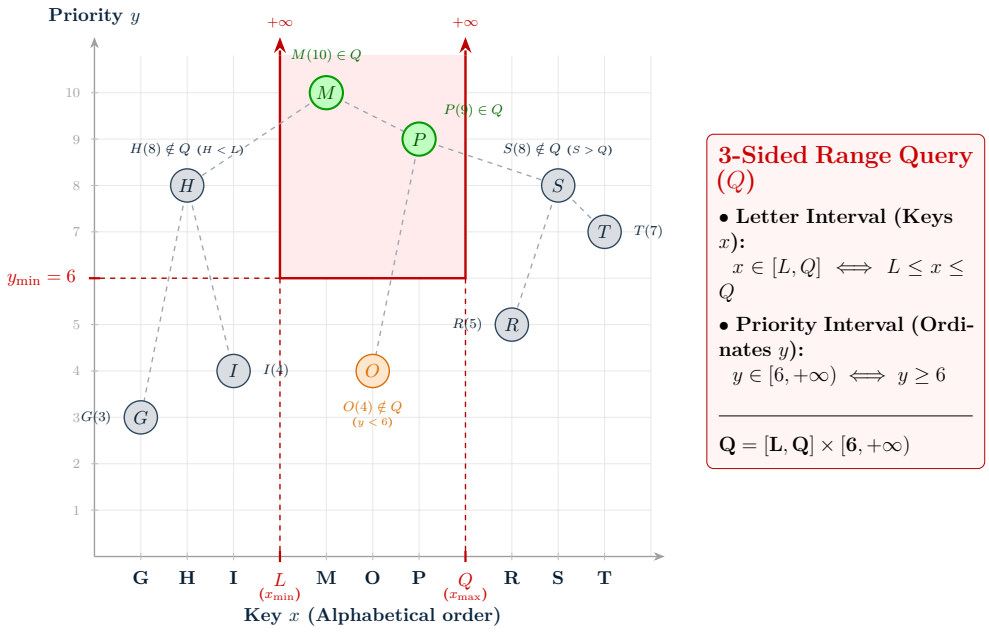


Figure 10.6: 3-Sided range query $Q = [L, Q] \times [6, +\infty)$ in the Cartesian plane: the x-axis follows alphabetical key order and the y-axis priorities. Points within the region are M and P. Points G, H, I and R, S, T fall outside the letter range, while O violates the minimum priority ($y < 6$). Dashed lines show the Treap structure.

The Instantaneous Pruning Principle ($\mathcal{O}(1)$ Pruning): Combining horizontal binary search ordering with vertical Max-Heap ordering provides an unbeatable pruning mechanism for filtering out out-of-range points (Figure 10.6):

- **Horizontal filter (BST on keys x):** bounds search to paths and subtrees intersecting $[x_{\min}, x_{\max}]$, branching down from the Lowest Common Ancestor (LCA) of $x_{\min}$ and $x_{\max}$.
- **Vertical filter (Max-Heap on priorities y):** for any node u, if its priority falls below the query threshold ($p(u) < y_{\min}$), then by the Max-Heap property **no descendant of u can have priority $\geq y_{\min}$**. Exploration of the entire subtree rooted at u halts immediately in $\mathcal{O}(1)$ time.

Two Algorithmic Paradigms: In algorithm engineering, 3-sided queries can be solved using two complementary approaches:

1. **Direct guided traversal:** search paths for $x_{\min}$ and $x_{\max}$ are traversed, visiting intermediate candidate subtrees and pruning any branch whose local root has $y < y_{\min}$.
2. **Sentinel split isolation:** two Split operations with sentinel nodes of priority $+\infty$ at $x_{\min}$ and $x_{\max}$ cleanly isolate a candidate subtree containing exactly the nodes with $x \in [x_{\min}, x_{\max}]$. Traversal on this candidate tree simplifies to a pure heap-guided filter governed by $y_{\min}$.

Theorem 10.5.1 (Output-Sensitive Complexity of 3-Sided Queries)**.** *Executing a 3-sided range query on a Treap of n elements returning k_{out} points takes expected time:*

$$T(n, k_{out}) = \mathcal{O}(\log n + k_{out}). \tag{10.26}$$

Proof. Visited nodes partition into two disjoint classes:

1. **Boundary nodes:** nodes traversed along search paths bounding $x_{\min}$ and $x_{\max}$. Since expected Treap height is $\mathcal{O}(\log n)$, the expected number of boundary nodes is $\mathcal{O}(\log n)$. In the sentinel-split paradigm, the two bubble-up rotations similarly take $\mathcal{O}(\log n)$ time.
2. **Internal candidate nodes and pruned leaves:** each visited node satisfying $p(u) \geq y_{\min}$ belongs to the solution set and is emitted, contributing $+k_{\text{out}}$. Unproductive branches are cut off at the very first

node with priority $< y_{\min}$; thus, the number of visited non-output nodes is at most the number of pruned leaf positions, bounded by $2k_{\text{out}} + 2 = \mathcal{O}(k_{\text{out}})$.

The total runtime is therefore optimal and output-sensitive: $\mathcal{O}(\log n + k_{\text{out}})$. □

10.5.5 Step-by-Step Example: Solving a 3-Sided Query via Split and Pruning

To illustrate the concrete mechanics of the structure, we walk through the complete execution of a 3-sided query using sentinel-based range isolation followed by priority pruning.

Query Instance Definition: Consider the 2D range query:

$$Q = [x_1, x_2] \times [y_1, +\infty) = [L, Q] \times [6, +\infty), \tag{10.27}$$

governed by:

- **Horizontal range:** alphabetic keys $x \in [L, Q]$, namely $L \leq x \leq Q$ in lexicographic order;
- **Vertical threshold:** numeric priorities $y \geq 6$.

Initial Treap Topology (INIT): Nodes store pairs $\frac{\text{Key}}{\text{Priority}} = \frac{x}{y}$, ordered alphabetically ($G < H < I < L < M < O < P < Q < R < S < T$) and constrained by the Max-Heap property on coordinates y. The initial tree contains 9 elements (Figure 10.7(a)):

- Global root: $M/10$;
- Left subtree: $H/8$, with children $G/3$ (left) and $I/4$ (right);
- Right subtree: $P/9$, with left child $O/4$ and right child $S/8$ (parent of $R/5$ left and $T/7$ right).

Phase 1: Lower Bound Isolation ($x < L$): To discard all keys strictly smaller than L, sentinel $(L, +\infty)$ is inserted:

- **Bubble-up to root:** Sentinel $(L, +\infty)$ enters as a leaf (right child of $I/4$). Its infinite priority triggers rotations promoting it to the tree root in $\mathcal{O}(\log n)$ expected time (Figure 10.7(b)).

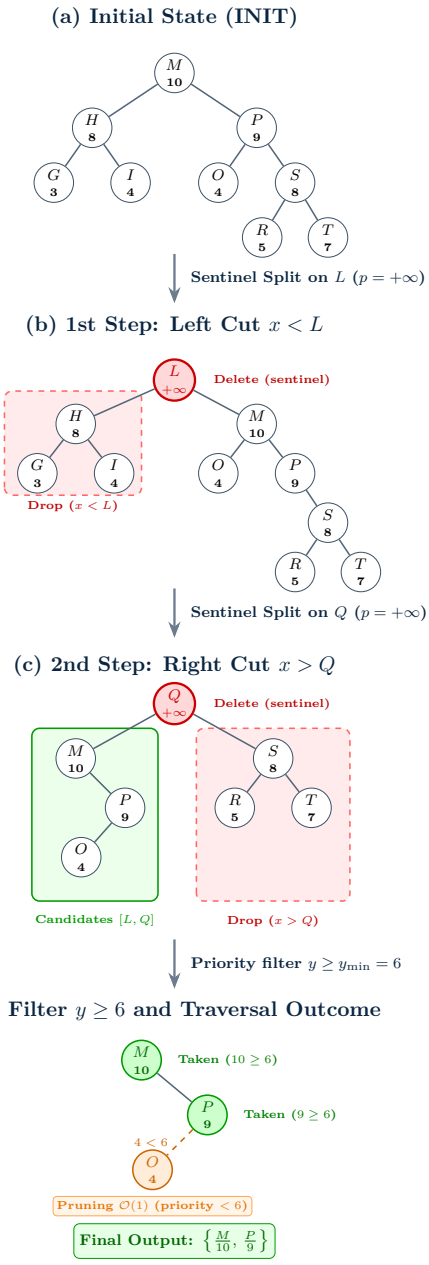


Figure 10.7: Step-by-step execution of the 3-sided range query $[L, Q] \times [6, +\infty)$ on a Treap, displayed vertically: (a) Initial state; (b) 1st Step: Left cut with sentinel $(L, +\infty)$ and DROP of $\{G, H, I\}$ $(x < L)$; (c) 2nd Step: Right cut with sentinel $(Q, +\infty)$ and DROP of $\{R, S, T\}$ $(x > Q)$, isolating candidate subtree for $[L, Q]$; (d) Guided traversal on candidate subtree with priority filter $y \geq 6$ and immediate $\mathcal{O}(1)$ pruning of $(O/4)$.

- **Left subtree drop (DROP):** By the BST property, the left subtree of L holds $\{G, H, I\}$. Because each key satisfies $x < L$, all fall outside $[L, Q]$ and are discarded in $\mathcal{O}(1)$ time.
- **Sentinel deletion:** Dummy node $(L, +\infty)$ is removed, leaving the right subtree $\{M, O, P, R, S, T\}$ as the active set for the next step.

Phase 2: Upper Bound Isolation ($x > Q$): On the remaining subtree, the upper bound is isolated by inserting sentinel $(Q, +\infty)$:

- **Bubble-up of Q to root:** Maximum priority rotates Q to the root of the remaining subtree in $\mathcal{O}(\log n)$ expected time (Figure 10.7(c)).
- **Right subtree drop (DROP):** The right subtree of Q contains $\{R, S, T\}$. Since $x > Q$, all violate the horizontal upper bound and are discarded in $\mathcal{O}(1)$ time.
- **Candidate subtree:** Removing dummy node $(Q, +\infty)$ leaves the left subtree containing exactly the candidate nodes with $x \in [L, Q]$:

$$\text{Candidates } [L, Q] = \left\{ \frac{M}{10}, \frac{P}{9}, \frac{O}{4} \right\}. \tag{10.28}$$

  This candidate subtree preserves the Max-Heap property on coordinates y.

Phase 3: Guided Traversal and Immediate Pruning ($y \geq 6$): Extracting solution points proceeds by recursive traversal from candidate root $(M/10)$ applying threshold filter $y \geq 6$ (Figure 10.7(d)):

1. **Node $(M/10)$:** Priority $y = 10 \geq 6$. Condition satisfied; emitted as output. Traversal continues to children (right child $P/9$).
2. **Node $(P/9)$:** Priority $y = 9 \geq 6$. Emitted as output. Traversal continues to children (left child $O/4$).
3. **Node $(O/4)$:** Priority $y = 4 < 6$. Violates minimum threshold.
   - $\mathcal{O}(1)$ **Pruning:** By the Max-Heap property, no descendant of $(O/4)$ can have priority greater than 4, so none can satisfy $y \geq 6$. Traversal on this branch halts immediately in $\mathcal{O}(1)$ time.

Computational Outcome and Cost Breakdown: The resulting solution set is:

$$\text{Output} = \left\{ \frac{M}{10}, \frac{P}{9} \right\}. \tag{10.29}$$

The entire procedure breaks down into:

- **Split / Isolation Phase:** two sentinel insertions rotating to root, taking $\mathcal{O}(\log n)$ expected time;
- **Traversal and Emission Phase:** visiting only output nodes (M and P) plus a single pruned leaf (O), taking time proportional to output size: $\mathcal{O}(k)$ with $k = 2$;
- **Total Runtime:**

  $$T(n) = \mathcal{O}(\log n + k), \tag{10.30}$$

  demonstrating the optimal output-sensitive efficiency achieved by marrying binary search trees with heaps.

Chapter 11

Posting Lists Intersection Algorithms in Information Retrieval

Lecture 11

Posting Lists Intersection Algorithms: Search Engine Architecture, Two-Pointer Merge-Based Scan, Binary Search, Mutual Partition with Information-Theoretic Lower Bound, Doubling Search with Jensen's Inequality, and Block Partitioning with Skip Pointers

Date: October 14, 2026

In previous lectures, we investigated advanced associative data structures and probabilistic paradigms such as Skip Lists and Treaps, examining how ordered key maintenance enables logarithmic search and range query operations. In real-world systems handling massive unstructured data collections, and specifically in web-scale **Information Retrieval** (IR), the dominant computational bottleneck does not stem from dynamic insertions of individual keys, but from the lightning-fast evaluation of **conjunctive queries** (boolean `AND` operations) executed over massive sorted arrays of document identifiers.

In this chapter, we address the fundamental algorithmic problem of **Set Intersection** (intersection of ordered sets) applied to **Posting Lists** (inverted lists) within a web search engine [15, 16]. We formally analyze the architecture of modern search engines and develop a systematic algorithmic hierarchy:

1. From elementary strategies of **two-pointer linear scanning** (*merge-based*) and **binary search**, deriving the analytical breakeven threshold determined by the imbalance in list sizes.

2. To the balanced decomposition algorithm **Mutual Partition** based on the *Divide and Conquer* paradigm, proving its asymptotic optimality against the decision tree lower bound (*Information-Theoretic Comparison Lower Bound*) [17, 18].

3. To the iterative, non-recursive **Doubling Search** (also known as *Exponential Search* or *Galloping Search* [19]), formalizing the step-by-step mathematical derivation of its optimal complexity via the application of **Jensen's Inequality** on concave telescoping series.

4. To the engineering foundation for hierarchical memory and disk storage: **Block Partitioning** with **Skip Pointers**, establishing through analytical optimization the optimal block size $L^* = \sqrt{n/m}$ and the resulting sublinear complexity $\mathcal{O}(\sqrt{n \cdot m})$ with cache-friendly sequential access.

11.1 Search Engine Architecture and Problem Formulation

11.1.1 Core Modules of a Web Search Engine

A modern web search engine processes collections comprising tens of billions of heterogeneous text documents, answering complex user queries with latency below a few tens of milliseconds. To achieve this, the retrieval infrastructure is organized into five coordinated macro-modules, illustrated in Figure 11.1:

1. **Web Crawler (24/7):** An autonomous, continuously active daemon navigating the World Wide Web hyperlink graph. It discovers new URLs, downloads updated HTML web pages, and stores raw content into a distributed local repository.

2. **Parser & Indexer:** Processes raw downloaded text streams. It performs markup stripping, tokenization, linguistic normalization (case folding, punctuation removal), high-frequency uninformative word removal (*stop-words elimination*), and morphological reduction (*stemming* or *lemmatization*). Upon completing textual analysis, it builds the core retrieval structure: the **Inverted Index**.

3. **Inverted Index:** The primary data structure for text indexing. It consists of two interconnected components:

   - **Dictionary (Vocabulary):** Maintains the ordered collection of unique terms $\mathcal{V} = \{t_1, t_2, \ldots, t_{|\mathcal{V}|}\}$ extracted across the corpus. For

each term, it stores global metadata such as document frequency (df_t) and a pointer/offset to its corresponding posting list. The dictionary typically resides entirely in main RAM (using hash tables, tries, or B-Trees).

- **Posting Lists (Inversion Lists):** For each term $t \in \mathcal{V}$, a strictly increasing ordered sequence of unique integer document identifiers (`DocID`) where the term appears at least once.

4. **Retriever:** Receives submitted user queries (in the simplest case, conjunctive boolean queries such as $t_1 \wedge t_2$). It queries the dictionary, retrieves the posting lists associated with the search terms, and executes **set intersection** to identify the candidate set of documents containing all queried terms.

5. **Ranker:** Receives the candidate document set from the Retriever. It computes a numerical relevance score for each candidate using statistical retrieval models (e.g., TF-IDF, Okapi BM25) or modern machine learning frameworks (*Learning to Rank*). It sorts results descending by score and returns the Top-k results to the user.

11.1.2 Formal Problem Formulation (Set Intersection)

Given a conjunctive user query:

$$Q = t_1 \ \wedge \ t_2, \tag{11.1}$$

the Retriever queries the inverted index and extracts the corresponding posting lists A and B:

$$A = [a_1, a_2, \ldots, a_n], \quad |A| = n, \tag{11.2}$$
$$B = [b_1, b_2, \ldots, b_m], \quad |B| = m. \tag{11.3}$$

Property 11.1.1 (Properties of Inversion Lists)**.** *The two identifier lists satisfy the following structural axioms:*

1. ***Strict Monotonic Ordering:*** *Elements within each list are positive integers strictly increasing, representing document identifiers assigned during corpus ingestion:*

$$a_1 < a_2 < \cdots < a_n \quad \textit{and} \quad b_1 < b_2 < \cdots < b_m. \tag{11.4}$$

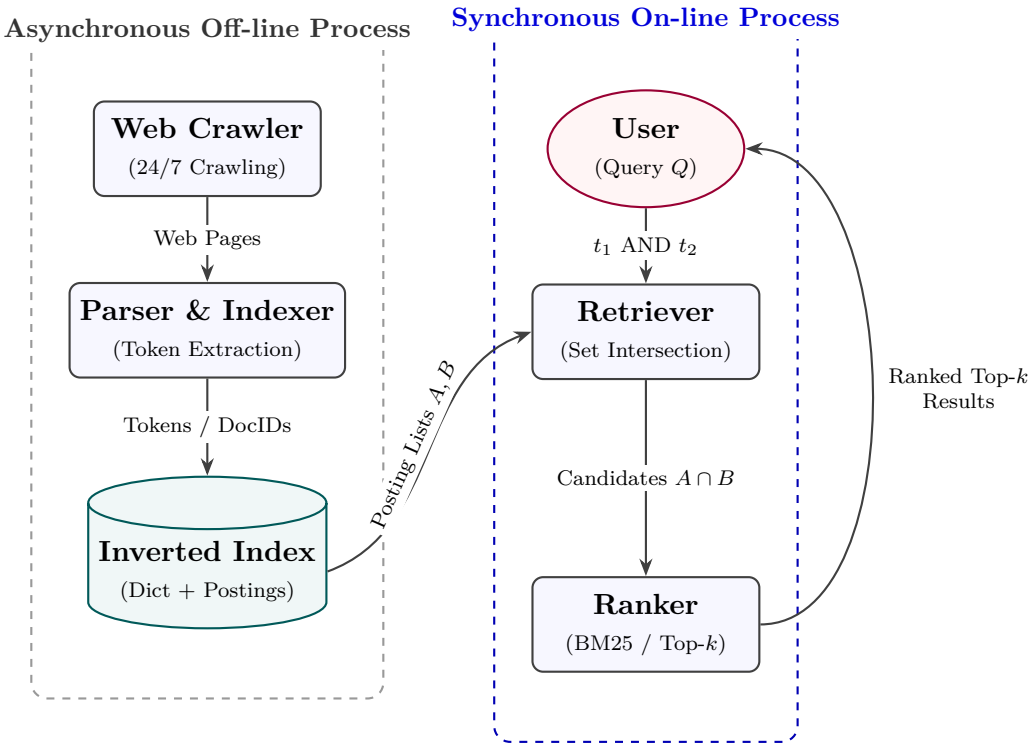


Figure 11.1: Modular architecture of a web search engine: asynchronous offline crawling and indexing pipeline vs synchronous online retrieval and ranking pipeline.

2. ***Asymmetry Assumption (W.L.O.G.):*** *we assume list B is the shorter sequence:*

$$m \leq n. \tag{11.5}$$

In practical search workloads, the imbalance ratio n/m can span multiple orders of magnitude (e.g., when t_1 is a frequent term such as "computer" with $n \approx 10^7$ and t_2 is a specialized keyword such as "quicksort" with $m \approx 10^3$).

Algorithmic Goal: Compute the set intersection:

$$A \cap B = \{x \in \mathbb{N} \mid x \in A \wedge x \in B\} \tag{11.6}$$

in minimum computational time, minimizing comparisons while maximizing cache and memory hierarchy efficiency.

11.2 Fundamental Algorithms and Comparative Analysis

11.2.1 Brute Force Approach

The naive method ignores the sorted property of the lists:

- For each of the m elements $b_j \in B$, scan linearly through array A from index 1 to n, stopping when reaching a match $a_i == b_j$ or an element $a_i > b_j$.
- In the worst case, every element of B requires inspecting all n elements of A.
- **Time Complexity:** $\mathcal{O}(n \cdot m)$.
- **Engineering Evaluation:** Unusable on real-world indices: with $n = 10^7$ and $m = 10^4$, this requires up to 10^{11} operations for a single query.

11.2.2 Merge-Based: Two-Pointer Linear Scanning

The **Merge-based** algorithm systematically exploits the strictly ascending order of both sequences. Maintaining two cursors i (for A) and j (for B) initialized at the head of each list, it mirrors the merge step of MergeSort (algorithm 17):

Algorithm 17: Two-Pointer Merge-Based Set Intersection

Input: Two sorted lists $A = [a_1, \ldots, a_n]$ and $B = [b_1, \ldots, b_m]$ with $m \leq n$

Output: Sorted list of common document identifiers $C = A \cap B$

```
1  i ← 1; j ← 1
2  C ← ∅
3  while i ≤ n and j ≤ m do
4  |  if A[i] == B[j] then
5  |  |  C.Append(A[i])
6  |  |  i ← i + 1
7  |  |  j ← j + 1
8  |  end
9  |  else if A[i] < B[j] then
10 |  |  i ← i + 1
11 |  end
12 |  else
   |  |  // A[i] > B[j]
13 |  |  j ← j + 1
14 |  end
15 end
16 return C
```

Theorem 11.2.1 (Complexity of Merge-Based Scanning)**.** *The two-pointer intersection algorithm performs at most $n + m - 1$ comparisons and runs in time:*

$$T_{merge}(n, m) = \mathcal{O}(n + m). \tag{11.7}$$

Proof. In each iteration of the **while** loop, at least one comparison between $A[i]$ and $B[j]$ takes place. Regardless of the comparison outcome $(<, =, >)$, at least one cursor (i or j) is incremented by exactly 1. Because execution terminates as soon as $i > n$ or $j > m$, the total number of iterations cannot exceed $(n + m - 1)$. □

Strengths and Limitations of Linear Scanning.

- **Strengths:** The algorithm reads elements in strictly **sequential** order. This pattern maximizes CPU cache spatial locality, takes full advantage of hardware prefetchers, and enables pipelined streaming directly from secondary storage without random seeks.
- **Weaknesses:** When $m \ll n$, linear scanning incurs significant wasted work: it inspects every single element of the long list A, unable to skip over large swathes of non-matching documents.

11.2.3 Binary Search Intersection

To overcome the overhead of linear scanning when list B is much shorter than A, binary search is employed:

- Iterate through the m elements of the short list B.
- For each $b_j \in B$, perform standard binary search (BinarySearch) across the entire array A.
- Because binary search over an array of size n requires at most $\lfloor \log_2 n \rfloor + 1$ comparisons:

$$T_{\text{binary}}(n, m) = \mathcal{O}(m \log n). \tag{11.8}$$

11.2.4 Analytical Trade-Off: Merge-Based vs Binary Search

We determine the breakeven point by comparing the two complexity bounds:

$$T_{\text{binary}}(n, m) < T_{\text{merge}}(n, m) \iff m \log n < n + m. \tag{11.9}$$

Solving for the cardinality m:

$$\begin{aligned} m \log n - m &< n \\ m(\log n - 1) &< n \\ m &< \frac{n}{\log n - 1} \approx \frac{n}{\log n}. \end{aligned} \tag{11.10}$$

Corollary 11.2.1 (Asymptotic Efficiency Regimes)**.** *From relation* (11.10)*, two operational regimes emerge:*

1. ***Sparse Regime*** *($m < \frac{n}{\log n}$):* *List B is sufficiently sparse relative to A. Binary search is asymptotically faster than linear scanning, as the total cost of logarithmic lookups remains strictly sublinear with respect to n.*

2. ***Dense Regime*** *($m \geq \frac{n}{\log n}$):* *List B has density comparable to A. Two-pointer linear scanning is asymptotically and practically superior, since the small constant factors of sequential cache reads easily outperform non-local bisection jumps.*

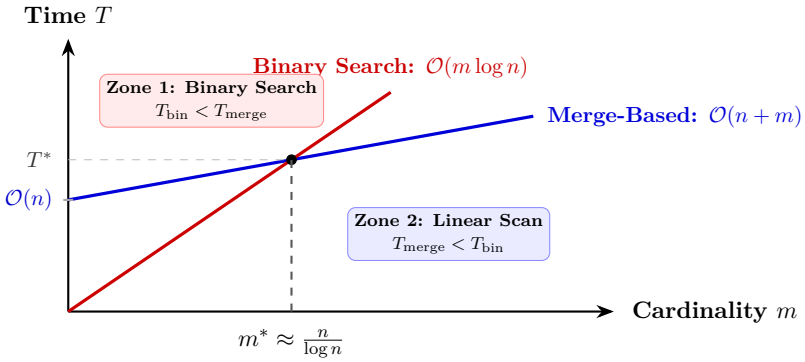


Figure 11.2: Qualitative performance and computational breakeven point $m^* \approx \frac{n}{\log n}$ between two-pointer linear scanning and binary search as short list cardinality m increases (with long list size n fixed).

11.3 Mutual Partition (Divide and Conquer)

11.3.1 Algorithmic Mechanism

The **Mutual Partition** algorithm overcomes the rigidity of independent binary searches by applying balanced simultaneous bisection via the **Divide and Conquer** paradigm [17]:

1. **Median Pivot Selection on A:** Select the median element of the long list A:
$$pivot = A\left[\left\lfloor \frac{n}{2} \right\rfloor\right]. \tag{11.11}$$

2. **Pivot Localization on B:** Using a single binary search on the short list B, locate the position where *pivot* falls (or would fall) in B.

3. **Decomposition into Disjoint Subproblems:** The *pivot* element naturally divides both sorted sets into two partitions:
$$A = A_1 \cup \{pivot\} \cup A_2, \quad \text{with } A_1 < pivot \text{ and } A_2 > pivot, \tag{11.12}$$
$$B = B_1 \cup (\{pivot\} \cap B) \cup B_2, \quad \text{with } B_1 < pivot \text{ and } B_2 > pivot. \tag{11.13}$$
If binary search finds *pivot* in B at position p_B ($B[p_B] = pivot$), it belongs to the intersection ($pivot \in A \cap B$) and is immediately emitted to the result set C. Otherwise ($pivot \notin B$), p_B marks the split point such that $B[p_B - 1] < pivot < B[p_B]$.

4. **Distributive Property:** Because all elements in A_1 and B_1 are strictly $< pivot$, and all elements in A_2 and B_2 are strictly $> pivot$, no element of A_1 can intersect B_2, and vice versa. Global intersection decomposes into the disjoint union of two independent intersections:
$$A \cap B = (A_1 \cap B_1) \cup (\{pivot\} \cap B) \cup (A_2 \cap B_2). \tag{11.14}$$
The two disjoint sub-instances (A_1, B_1) and (A_2, B_2) are solved recursively in parallel without re-inserting the pivot into either subproblem.

11.3.2 Complexity and Decision Tree Lower Bound

Theorem 11.3.1 (Complexity of Mutual Partition)**.** *Mutual Partition computes the intersection of list A of size n and list B of size m ($m \leq n$) in time:*
$$T(n, m) = \Theta\left(m\left(1 + \log \frac{n}{m}\right)\right). \tag{11.15}$$

Verification of Boundary Cases. Formula (11.15) gracefully interpolates the entire spectrum of relative list lengths:

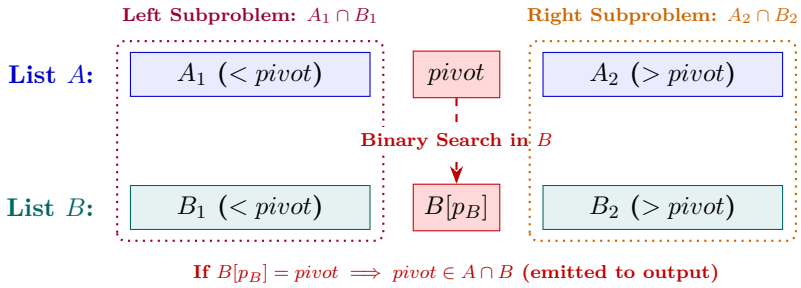


Figure 11.3: Conceptual schema of Mutual Partition: median element $pivot = A[n/2]$ is searched in B via binary search; if found at position p_B, it is emitted to the output intersection. Both sets decompose into disjoint independent subproblems $A_1 \cap B_1$ and $A_2 \cap B_2$ solved recursively in parallel.

- **Balanced Case ($n = m$):**

$$T(m, m) = \Theta\left(m\left(1 + \log\frac{m}{m}\right)\right) = \Theta(m(1+0)) = \mathcal{O}(m) = \mathcal{O}(n+m). \tag{11.16}$$

  The complexity simplifies to optimal linear merge scanning.

- **Severely Asymmetric Case ($m = \mathcal{O}(1)$):**

$$T(n, 1) = \Theta\left(1 \cdot \left(1 + \log\frac{n}{1}\right)\right) = \mathcal{O}(\log n). \tag{11.17}$$

  The complexity matches a single binary search.

11.3.3 Information-Theoretic Lower Bound

We prove that the bound achieved by Mutual Partition is asymptotically unbeatable by any comparison-based algorithm.

Theorem 11.3.2 (Information-Theoretic Lower Bound for Set Intersection)**.** *Any comparison-based algorithm computing the intersection of two sorted sequences of sizes n and m ($m \leq n$) requires in the worst case:*

$$\#CMP \geq \Omega\left(m \log\frac{n}{m}\right) \quad \textit{comparisons.} \tag{11.18}$$

Proof. Consider the problem of determining the relative interleaving of m sorted elements of B within the n sorted slots of A. The number of possible distinct interleavings is given by the binomial coefficient:

$$\#\text{Possible Configurations} = \binom{n}{m} = \frac{n!}{m!\,(n-m)!}. \tag{11.19}$$

In a ternary decision tree model where each internal node represents a comparison $<, =, >$ and each leaf corresponds to a consistent output configuration, the minimum tree height h required to distinguish all valid leaves satisfies:

$$\#\text{CMP} \geq \log_2(\#\text{Configurations}) \geq \log_2 \binom{n}{m}. \tag{11.20}$$

Using the combinatorial lower bound for binomial coefficients:

$$\binom{n}{m} \geq \left(\frac{n}{m}\right)^m, \tag{11.21}$$

taking the base-2 logarithm on both sides yields:

$$\log_2 \binom{n}{m} \geq \log_2 \left(\left(\frac{n}{m}\right)^m\right) = m \log_2 \left(\frac{n}{m}\right). \tag{11.22}$$

Therefore, no comparison-based algorithm can have asymptotic complexity lower than $\Omega\left(m \log \frac{n}{m}\right)$. Mutual Partition achieves this theoretical bound, proving it is asymptotically optimal. □

11.3.4 Practical Bottleneck of Mutual Partition

Despite its theoretical optimality, Mutual Partition exhibits a critical drawback in practical systems engineering:

"This is good, but it is recursive!"

Its recursive nature incurs substantial stack frame overhead. Furthermore, non-local median jumping leads to erratic memory access patterns, destroying CPU cache line locality on large arrays.

11.4 Doubling Search (Exponential Search / Galloping Search)

11.4.1 Motivation and Intuition of Exponential Jumps

Doubling Search (known in classical literature as **Exponential Search** or **Galloping Search**, formalized by Bentley and Yao [19]) was designed to achieve the same optimal complexity $\mathcal{O}(m \log \frac{n}{m})$ as Mutual Partition while remaining **purely iterative and non-recursive**.

The core intuition exploits spatial correlation between consecutive elements of B. Suppose we just located element $b_{j-1} \in B$ at index i_{j-1} in array A:

- To find the subsequent element $b_j \in B$, we neither restart from index 1 nor bisect the entire remaining suffix of A.
- Because B is sorted ($b_j > b_{j-1}$), the matching index i_j must satisfy $i_j \geq i_{j-1}$.
- Rather than advancing by single steps or halving the remaining array, we step forward from i_{j-1} with **exponentially increasing powers of 2**:

$$+2^0,\ +2^1,\ +2^2,\ +2^3,\ \ldots,\ +2^k. \tag{11.23}$$

- As soon as we find the smallest exponent k such that $A[i_{j-1} + 2^k] \geq b_j$, we restrict binary search strictly to the bounded local window between the last two jumps:

$$\text{Binary Search Window} = \left[i_{j-1} + 2^{k-1},\ i_{j-1} + 2^k\right]. \tag{11.24}$$

Example 11.4.1 (Execution Trace of Doubling Search)**.** Let A be the stored posting list. Suppose the previous match for $b_{j-1} = 12$ occurred at index i_{j-1}. We now search for $b_j = 41$:

- Probe indices by powers of 2:

$$\begin{aligned} A[i_{j-1} + 1] &= 14 < 41, \\ A[i_{j-1} + 2] &= 16 < 41, \\ A[i_{j-1} + 4] &= 20 < 41, \\ A[i_{j-1} + 8] &= 34 < 41, \\ A[i_{j-1} + 16] &= 60 \geq 41. \end{aligned}$$

- Since $34 < 41 \leq 60$, value 41 must lie inside $[i_{j-1} + 8,\ i_{j-1} + 16]$.
- Perform standard binary search on this local window of width $16 - 8 = 8$, locating 41 in very few comparisons.

11.4.2 Step-by-Step Mathematical Derivation

We formalize the complexity analysis of Doubling Search across four rigorous steps.

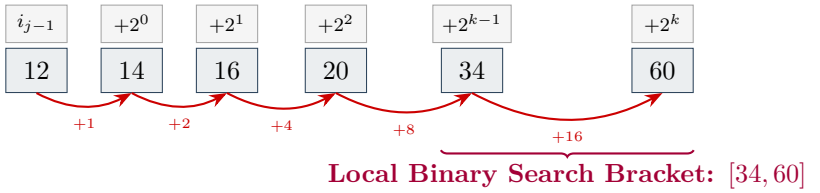


Figure 11.4: Progression of exponential jumps in Doubling Search (Galloping): doubling stride until bracketing the target, followed by local binary search.

Step 1: Determining the Number of Jumps k. Let i_j be the matching position of b_j in A. The distance traveled between consecutive matches is $(i_j - i_{j-1})$. Exponential stepping stops as soon as the stride overshoots this distance:

$$2^{k-1} < i_j - i_{j-1} \leq 2^k. \tag{11.25}$$

Examining the bounds separately:

1. From the lower bound:

$$2^{k-1} < i_j - i_{j-1} \implies k-1 < \log_2(i_j - i_{j-1}) \implies k_1 = \mathcal{O}(\log_2(i_j - i_{j-1})). \tag{11.26}$$

2. From the upper bound:

$$i_j - i_{j-1} \leq 2^k \implies 2^k \geq i_j - i_{j-1} \implies k_2 = \Omega(\log_2(i_j - i_{j-1})). \tag{11.27}$$

Hence, the number of jumping probes is proportional to the logarithm of the distance:

$$k = k_1 + k_2 = \Theta(\log_2(i_j - i_{j-1})). \tag{11.28}$$

Step 2: Local Binary Search Cost. Binary search is then performed in $[i_{j-1} + 2^{k-1},\ i_{j-1} + 2^k]$. The window width is:

$$\Delta = 2^k - 2^{k-1} = 2^{k-1}. \tag{11.29}$$

The computational cost for binary search over an interval of size 2^{k-1} is:

$$T_{\text{local-BS}} = \mathcal{O}(\log_2(2^{k-1})) = \mathcal{O}(k-1) = \mathcal{O}(k) = \mathcal{O}(\log_2(i_j - i_{j-1})). \tag{11.30}$$

The local binary search cost asymptotically matches the galloping phase cost.

Step 3: Total Cost Across All m Elements. Summing jump cost and local binary search cost for element $b_j \in B$:

$$T_j = \mathcal{O}(k) + \mathcal{O}(k) = \mathcal{O}(\log_2(i_j - i_{j-1})). \tag{11.31}$$

Aggregating across all m elements of B:

$$T_{\text{total}} = \sum_{j=1}^{m} T_j = \mathcal{O}\left(\sum_{j=1}^{m} \log_2(i_j - i_{j-1})\right). \tag{11.32}$$

Step 4: Application of Jensen's Inequality. To maximize summation (11.32) and establish the worst-case bound, observe that $f(x) = \log_2 x$ is **strictly concave** on $\mathbb{R}^+$:

$$f'(x) = \frac{1}{x \ln 2}, \qquad f''(x) = -\frac{1}{x^2 \ln 2} < 0 \quad \forall x > 0. \tag{11.33}$$

Theorem 11.4.1 (Jensen's Inequality for Concave Functions)**.** *For a concave function $f : \mathbb{R}^+ \to \mathbb{R}$ and m points $x_1, \ldots, x_m$:*

$$\frac{1}{m} \sum_{j=1}^{m} f(x_j) \leq f\left(\frac{1}{m} \sum_{j=1}^{m} x_j\right). \tag{11.34}$$

Setting $f(x) = \log_2 x$ and $x_j = i_j - i_{j-1}$:

$$\sum_{j=1}^{m} \log_2(i_j - i_{j-1}) \leq m \cdot \log_2\left(\frac{1}{m} \sum_{j=1}^{m} (i_j - i_{j-1})\right). \tag{11.35}$$

The inner sum constitutes a **telescoping series**:

$$\sum_{j=1}^{m} (i_j - i_{j-1}) = (i_1 - i_0) + (i_2 - i_1) + \cdots + (i_m - i_{m-1}) = i_m - i_0 \leq n. \tag{11.36}$$

Substituting this upper bound into Jensen's inequality (11.35):

$$\sum_{j=1}^{m} \log_2(i_j - i_{j-1}) \leq m \log_2\left(\frac{n}{m}\right). \tag{11.37}$$

Therefore, the overall complexity is:

$$T_{\text{Doubling}}(n, m) = \mathcal{O}\left(m \log_2 \frac{n}{m}\right). \tag{11.38}$$

Doubling Search matches the theoretical optimum of Mutual Partition while executing as a strictly forward iterative procedure.

11.4.3 Hardware Limitations of Doubling Search

Despite its optimal complexity, Doubling Search is rarely applied directly in its basic form on storage-backed systems:

- Exponential stepping causes non-contiguous memory access patterns.
- On magnetic disks, random seek latency (5–10 ms per seek) dwarfs the cost of sequential block reads.
- Even on RAM and solid-state drives, random probing triggers cache misses and TLB churn.

11.5 Block Partitioning / Skip Pointers (Practical Block Approach)

11.5.1 Block Index Architecture and Skip Pointers

To combine the asymptotic skipping advantages with the peak sequential throughput of physical memory and storage, production search systems (including Apache Lucene and Elasticsearch) deploy **Block Partitioning** with **Skip Pointers** [16, 15].

Definition 11.5.1 (Block Partitioning of a Posting List)**.** Let A be a posting list of size n. Given a fixed integer block size $L \geq 1$:

1. Sequence A is partitioned into $\frac{n}{L}$ contiguous blocks $A_1, A_2, \ldots, A_{n/L}$, each holding L elements:
$$A_k = [a_{(k-1)L+1},\ a_{(k-1)L+2},\ \ldots,\ a_{kL}]. \tag{11.39}$$
2. A compact auxiliary sentinel list A' of size $\frac{n}{L}$ is constructed, known as the **Skip List of Maximums**:
$$A' = \Big[\max(A_1),\ \max(A_2),\ \ldots,\ \max(A_{n/L})\Big] = [a_L,\ a_{2L},\ \ldots,\ a_n]\,. \tag{11.40}$$
Each entry in A' stores the block's maximum value and an offset pointer to the start of the block (*skip pointer*).

11.5.2 Algorithmic Procedure and Cost Analysis

Intersecting short list B with block-partitioned list A proceeds in two sequential stages:

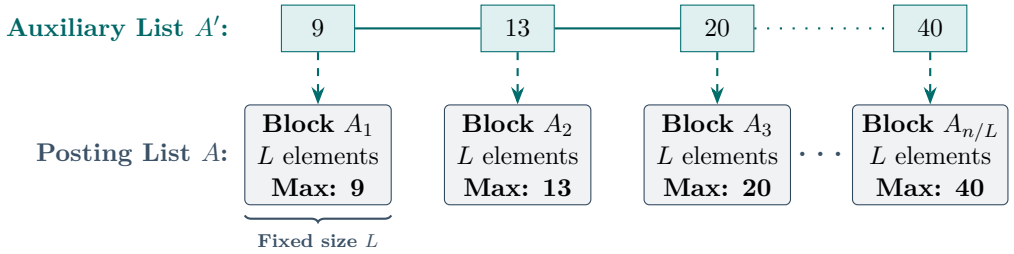


Figure 11.5: Organization of Block Partitioning with Skip Pointers: auxiliary list A' stores block maximums and enables jumping over unneeded blocks.

Stage 1: Routing Elements of B into Blocks. Linearly merge list B (size m) and auxiliary maximums list A' (size $\frac{n}{L}$). Since both lists are sorted, a two-pointer merge routes each element $b \in B$ into its candidate block:

- Element b maps to block A_k if and only if $\max(A_{k-1}) < b \leq \max(A_k)$.
- List B is partitioned into disjoint subsets:

$$B = B_1 \cup B_2 \cup \cdots \cup B_{n/L}, \quad \text{where } B_k = \{b \in B \mid b \in \text{range}(A_k)\}. \tag{11.41}$$

- Time complexity of Stage 1:

$$T_{\text{stage 1}} = \mathcal{O}\left(m + \frac{n}{L}\right). \tag{11.42}$$

Stage 2: Local Intersection on Active Blocks. Iterate over the blocks of A:

- If $B_k = \emptyset$, block A_k is **skipped entirely** without fetching or decoding.
- If $B_k \neq \emptyset$ (active block), perform linear merge intersection between A_k (size L) and B_k (size $|B_k|$):

$$T_{\text{block } k} = \mathcal{O}(|A_k| + |B_k|) = \mathcal{O}(L + |B_k|). \tag{11.43}$$

- At most m blocks can be active ($|\{k \mid B_k \neq \emptyset\}| \leq m$).
- Summing across all active blocks:

$$T_{\text{stage 2}} = \sum_{k:B_k \neq \emptyset} (L + |B_k|) \leq L \cdot m + m. \tag{11.44}$$

Overall Computational Cost. Combining the execution times of both stages, the total time complexity of Block Partitioning as a function of block size L is:

$$\begin{aligned} T_{\text{total}}(L) &= T_{\text{stage 1}} + T_{\text{stage 2}} \\ &= \mathcal{O}\left(m + \frac{n}{L}\right) + \mathcal{O}(L \cdot m + m) \\ &= \mathcal{O}\left(\frac{n}{L} + m \cdot L\right), \end{aligned} \tag{11.45}$$

where the term $\mathcal{O}(m)$ is absorbed since $m \leq m \cdot L$ for any integer block parameter $L \geq 1$.

Bibliography

[1] Donald E. Knuth. *The Art of Computer Programming*, volume 1. Addison-Wesley, Reading, MA, 3 edition, 1997.

[2] Alok Aggarwal and Jeffrey S. Vitter. The input/output complexity of sorting and related problems. *Communications of the ACM*, 31(9):1116–1127, 1988.

[3] Edsger W. Dijkstra. *A Discipline of Programming*. Prentice-Hall, 1976.

[4] Robert Sedgewick. Implementing quicksort programs. *Communications of the ACM*, 21(10):847–857, 1978.

[5] Vladimir Yaroslavskiy. Dual-pivot quicksort algorithm. *Research Disclosure*, 2009. Available at http://iaroslavski.narod.ru/quicksort/DualPivotQuicksort.pdf.

[6] Manuel Blum, Robert W. Floyd, Vaughan Pratt, Ronald L. Rivest, and Robert E. Tarjan. Time bounds for selection. *Journal of Computer and System Sciences*, 7(4):448–461, 1973.

[7] C. T. Fan, Mervin E. Muller, and Ivan Rezucha. Development of sampling plans by using sequential (item by item) selection techniques and digital computers. *Journal of the American Statistical Association*, 57(298):387–402, 1962.

[8] Jeffrey S. Vitter. Random sampling with a reservoir. *ACM Transactions on Mathematical Software*, 11(1):37–57, 1985.

[9] William Pugh. Skip lists: A probabilistic alternative to balanced trees. *Communications of the ACM*, 33(6):668–676, 1990.

[10] Raimund Seidel and Cecilia R. Aragon. Randomized search trees. *Algorithmica*, 16(4-5):464–497, 1996.

[11] Yury A. Malkov and Dmitry A. Yashunin. Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 42(4):824–836, 2020.

[12] Jean Vuillemin. A unifying look at data structures. *Communications of the ACM*, 23(4):229–239, 1980.

[13] Edward M. McCreight. Priority search trees. *SIAM Journal on Computing*, 14(2):257–276, 1985.

[14] Michael A. Bender and Martin Farach-Colton. The LCA problem revisited. In *Proceedings of the 4th Latin American Symposium on Theoretical Informatics (LATIN)*, pages 88–94. Springer, 2000.

[15] Christopher D. Manning, Prabhakar Raghavan, and Hinrich Schütze. *Introduction to Information Retrieval*. Cambridge University Press, Cambridge, UK, 2008.

[16] Alistair Moffat and Justin Zobel. Self-indexing inverted files for fast text retrieval. *ACM Transactions on Information Systems*, 14(4):349–379, 1996.

[17] Ricardo Baeza-Yates. A fast set intersection algorithm for sorted sequences. In *Combinatorial Pattern Matching (CPM)*, volume 3109 of *Lecture Notes in Computer Science*, pages 400–408. Springer, 2004.

[18] Erik D. Demaine, Alejandro López-Ortiz, and J. Ian Munro. Adaptive set intersection, union, and difference of ordered sets. In *Proceedings of the 11th Annual ACM-SIAM Symposium on Discrete Algorithms (SODA)*, pages 743–752, 2000.

[19] Jon Louis Bentley and Andrew Chi-Chih Yao. An almost optimal algorithm for unbounded searching. *Information Processing Letters*, 5(3):82–87, 1976.